

Cross-Chain Content Rights Management Service

A Polkadot-Based Unified Rights Token for Subscription, Pay-Per-View, and Permanent Ownership

Ryc Brownrigg

Supervised by Dr Joshua Ellul

Centre for Distributed Ledger Technologies
Faculty of ICT
University of Malta

September, 2026

A dissertation submitted in partial fulfilment of the requirements for the degree of M.Sc. in Blockchain and Distributed Ledger Technologies (ICT).



Copyright ©2026 University of Malta

WWW.UM.EDU.MT

Fifth edition, October 5, 2026



**L-Università
ta' Malta**

Declaration by Postgraduate Students

(a) Authenticity of Dissertation

I hereby declare that I am the legitimate author of this Dissertation and that it is my original work.

No portion of this work has been submitted in support of an application for another degree or qualification of this or any other university or institution of higher education.

I hold the University of Malta harmless against any third party claims with regard to copyright violation, breach of confidentiality, defamation and any other third party right infringement.

(b) Research Code of Practice and Ethics Review Procedures

I declare that I have abided by the University's Research Ethics Review Procedures.

As a Master's student, as per Regulation 58 of the General Regulations for University Postgraduate Awards, I accept that should my dissertation be awarded a Grade A, it will be made publicly available on the University of Malta Institutional Repository.

Faculty/Institute/Centre/School	Faculty of ICT
Degree	M.Sc. in Blockchain and Distributed Ledger Technologies (ICT)
Title	Cross-Chain Content Rights Management Service
Candidate (Id.)	Ryc Brownrigg (2104001)

Signature of Student

Date

October 5, 2026

Acknowledgements

I express my deepest gratitude to my supervisor, Dr Joshua Ellul, for his insightful guidance, patience, and unwavering support throughout this dissertation.

I am also grateful to the faculty of the Centre for Distributed Ledger Technologies for creating the academic environment in which this work was produced.

I thank the Polkadot and Kusama communities for their documentation, tooling, and open-source contributions that supported this work. I also thank Parity Technologies for the Polkadot SDK, comprehensive documentation, and tooling that made this implementation feasible.

I used SuperWhisper, a dictation tool, to compose most of this dissertation by voice, given my limited typing speed, and Grammarly for grammar, spelling, clarity, and rephrasing throughout the writing and revision process. During revision, I also used Claude (Anthropic), a generative AI assistant, to review drafts critically and to check the text against the source code and committed results. The design, implementation, experiments, analysis, and conclusions are my own, and I take full responsibility for this dissertation's content.

Finally, I want to thank my mother, whose endless love, encouragement, and faith in me have carried me through all my wild and wonderful adventures. I love you so much, Mom.

Thank you all.

Abstract

The content creator economy concentrates value in centralized intermediaries. No existing system, centralized or decentralized, provides subscription access, pay-per-view consumption, and permanent ownership through a single rights primitive that is portable across heterogeneous blockchain networks.

We ask how a shared-security, multi-chain framework built on Polkadot XCM and FRAME primitives can deliver a unified rights record supporting all three monetization models, and what transaction finality, cost efficiency, and creator revenue retention it can achieve. We built the Cross-Chain Content Rights Management Service (CCRMS), a Polkadot parachain prototype, and evaluated it through unit testing, performance benchmarks, a security audit, a centralized baseline comparison, and a Monte Carlo creator-revenue simulation.

On a local test network, the prototype sustains at least 26.2 transactions per second, the rate our benchmark client could offer, with approximately six-second inclusion latency, and cross-chain rights operations reach the destination chain in a median of five blocks (about 30 s). In simulation, mean creator revenue is 4.4% higher than on centralized platforms, but the median simulated creator earns less: CCRMS usually wins only for creators above roughly 6,000 followers, and the advantage disappears if its fee exceeds about 7% or centralized discovery is about 25% stronger than assumed. The prototype no longer limits the number of rights holders per content item, but it has not been tested at the audience sizes where the advantage appears.

We conclude that a unified rights record supporting all three access models can be built from standard FRAME primitives, and that the standard XCM `WithdrawAsset` → `BuyExecution` → `Transact` sequence, available since XCM v3, suffices for issuance-style cross-chain rights operations without custom protocol extensions. The revenue advantage of decentralized rights management is small and not universal; it depends on audience size and on the assumed value of centralized discovery.

Keywords: cross-chain content rights, Polkadot XCM, unified rights record, creator revenue retention, design science research

Contents

List of Abbreviations	xix
1 Introduction	1
1.1 The Problem	1
1.2 Background Context	2
1.3 Research Aim and Approach	3
1.4 Contributions	3
1.5 Scope, Claims, and Limitations	4
1.5.1 Claims	4
1.5.2 Non-Claims	5
1.5.3 Operational Limitations	5
2 Background and Related Work	7
2.1 Blockchain-Based Content Management	7
2.1.1 Market Context	7
2.1.2 Foundational Work on Blockchain-Based DRM	7
2.1.3 NFTs and Intellectual Property Rights	8
2.2 Cross-Chain Interoperability	9
2.2.1 Polkadot Architecture	9
2.2.2 Cross-Chain Technology Surveys	9
2.2.3 Bridges and Snowbridge	10
2.2.4 Polkadot 2.0 and JAM	11
2.2.5 Cross-Chain Copyright Management as a Research Target	11
2.3 Decentralized Monetization Models	12
2.3.1 Subscription, Pay-Per-View, and Purchase Models	12

2.3.2	Pay-Per-View and the Micropayment Problem	13
2.3.3	Smart Contracts as the Implementation Layer	13
2.4	Challenges and Gaps	13
2.5	Summary	17
3	Research Questions and Objectives	19
3.1	Primary Research Question	19
3.2	Sub-Questions	20
3.3	Research Objectives	22
3.4	Mapping Questions to Objectives	22
3.5	Summary	23
4	Methodology and System Design	25
4.1	Research Methodology	25
4.1.1	Design Science Research Framing	25
4.1.2	Iterative Prototyping	26
4.1.3	Data Collection Methods	26
4.1.4	Evaluation Philosophy: Measurement-Focused	26
4.1.5	Methodology Pivots	27
4.2	System Design	27
4.2.1	The Unified Rights Record Concept	27
4.2.2	The Three Monetization Models	27
4.2.3	NFT-Based Rights Tokenization	28
4.2.4	Cross-Chain Communication Model	30
4.2.5	Pallet-First Architecture	31
4.2.6	Layered Architecture	31
4.3	Summary	33
5	Implementation	35
5.1	Pallet Implementation and Smart Contract Layer	35
5.2	Cross-Chain Integration	36
5.2.1	XCM Message Flows	36
5.2.2	Snowbridge Integration with Ethereum	38
5.3	Challenges and Solutions	38
5.4	Summary	40
6	Evaluation and Testing	41
6.1	Evaluation Approach	41

6.2	Testing Layers	43
6.3	Performance Testing Framework	44
6.4	Test Scenarios	45
6.5	Metrics and Key Performance Indicators	47
6.6	Testing Infrastructure	49
6.7	Validity and Limitations	50
6.8	Summary	51
7	Results	53
7.1	Technical Results	53
7.1.1	Throughput	53
7.1.2	Sustained Throughput (Stress Test)	54
7.1.3	Latency	55
7.1.4	Block Weight Utilization	56
7.1.5	Storage Growth	56
7.1.6	Reliability	57
7.1.7	Snowbridge End-to-End Bridge	58
7.1.8	Comparative Analysis vs. Centralized Baseline	59
7.2	Economic Results	60
7.2.1	Monte Carlo Creator Revenue Simulation	60
7.2.2	Audience Size Dependency	62
7.2.3	Comparative Fee Analysis	63
7.2.4	Sensitivity Analysis	64
7.2.5	Tested versus Simulated Audience Sizes	64
7.3	Security and User-Centric Results	64
7.3.1	Security Audit Findings	65
7.3.2	Test Coverage	67
7.3.3	Decentralization Assessment (HHI)	67
7.3.4	Static Analysis and Dependency Audit	68
7.4	Summary	69
8	Discussion	71
8.1	Answering the Research Questions	71
8.1.1	SQ1: Cross-Chain Rights Operations and Finality	71
8.1.2	SQ2: Latency versus Decentralization	72
8.1.3	SQ3: Unified Rights Record	73
8.1.4	SQ4: Security and Authorization	73

8.1.5	SQ5: Economic Viability	73
8.2	Interpretation of Key Results	74
8.2.1	The 26.2 TPS Figure in Context	74
8.2.2	The Economic Crossover at ~6,000 Followers	75
8.3	Architectural Decisions Revisited	75
8.4	The Work in Context	75
8.5	Limitations	76
8.6	Implications	77
8.7	Critical Self-Reflection	78
8.8	Summary	78
9	Conclusion and Future Work	81
9.1	Summary of Contributions	81
9.2	Principal Findings	82
9.3	Future Work	83
9.4	Closing Reflection	85
	Appendix A Implementation Challenge Narratives	87
A.1	The Pivot from RMRK 2.0 to pallet-nfts	87
A.2	Contract-First to Pallet-First Pivot	88
A.3	The Snowbridge Fork-Version Configuration Issue	88
A.4	The Lodestar Preset Version Pinning	89
A.5	The XCM Sovereign Account Endianness Bug	89
A.6	The XCM Fee Scaling for Proof Size	90
A.7	Finding B: Authorization Gap	90
A.8	Auto-Renewal Child Cap	91
A.9	ink! Discontinuation	91
	Appendix B Research Objective Details	93
B.1	RO1: Design and Prototype XCM Extensions	93
B.2	RO2: Implement a Unified Rights Token	93
B.3	RO3: Merge Subscription, PPV, and Purchase Logic	93
B.4	RO4: Benchmark Scalability and Interoperability	94
B.5	RO5: Zero-Knowledge Proof Hooks (Not Met)	94
B.6	RO6: Quantify Creator Savings	94
	Appendix C Code Listings	95
C.1	Solidity Interface: ContentRightsPrecompile	95

C.2 Canonical XCM Message	96
C.3 Repository Directory Layout	96
C.4 Dispatchable Extrinsic Reference	97
C.5 Pallet Storage Layout Reference	98
Appendix D Security and Coverage Findings	101
D.1 Finding B: Authorization Gap	101
D.2 Finding E: View Pack Overwrite	102
D.3 Accepted Design Findings (A, C, D)	102
D.4 Finding I: Zero-View Denial of Service	103
D.5 Finding J: Child-NFT Divergence	103
D.6 Finding K: Auto-Renewal Starvation	104
D.7 Finding L: Unbenchmarked Weights	104
D.8 Finding M: Renewal-Queue Griefing	105
D.9 Error Variant Coverage	105
D.10 Static Analysis Advisory Breakdown	106
Appendix E Future Work Details	107
E.1 Production Deployment	107
E.2 Scaling Beyond the Prototype Configuration	107
E.3 Migration Paths for the Smart Contract Layer	108
E.4 Holder Scale and Creator Deposits	108
E.5 User Study and Consumer Application	108
E.6 The Bridged-Payment Final Hop	109
E.7 Resale Royalties	109
E.8 Origin Aliasing and Delegation	109
E.9 Monte Carlo Simulation Extensions	110
E.10 Formal Verification	110
E.11 JAM Transition	110
Appendix F Work Plan and Timeline	111
Appendix G Monte Carlo Simulation Parameters and Sensitivity	113
References	119

List of Tables

2.1	Literature Gaps and CCRMS Contributions	14
3.1	Research Sub-Questions	20
3.2	Research Objectives Summary	22
3.3	Mapping of Sub-Questions to Objectives, Methods, and Chapters	23
5.1	Implementation Challenges Summary	39
6.1	Mapping of Evaluation Activities to Research Sub-Questions	41
6.2	Evaluation Activities Classified by DSR Evaluation Type	42
6.3	Benchmark Scripts	44
6.4	Test Scenarios	46
6.5	KPI Definitions	47
6.6	Test Environment Specifications	49
7.1	Single-Chain Inclusion Capacity at Batch Size 100	54
7.2	XCM Cross-Chain Delivery ($n = 10$ per operation; finalized blocks)	55
7.3	Block Weight Utilization at Increasing Batch Sizes	56
7.4	Per-Item Storage Costs (SCALE-encoded)	57
7.5	Reliability Test Results (median and range over three SIGKILL runs; one SIGTERM run for comparison)	58
7.6	Snowbridge End-to-End Message Path (ETH Transfers to AssetHub; CCRMS Not Exercised)	59
7.7	Comparative Analysis: CCRMS Parachain vs. Centralized (Express.js + SQL.js)	59
7.8	Monte Carlo Creator Revenue Simulation (Annual Revenue Per Creator: Mean and Median)	61

7.9	CCRMS Win Rate Against Centralized Platform by Audience Tier	62
7.10	Per-Scenario Break-Even Rates	63
7.11	Central 95% Range of Simulated Total Revenue (2.5th–97.5th percentiles over 10,000 iterations; 1,000 creators)	63
7.12	Published fee ranges by platform category. Centralized literature values span 30–60%; the simulation uses 30–45% (Table G.2).	63
7.13	Security Audit Findings	65
7.14	Herfindahl-Hirschman Index (HHI) Comparison	67
7.15	KPI Results Summary	69
9.1	Future Work Summary	84
C.1	Dispatchable Extrinsic in pallet-content-rights	97
C.2	Storage Layout of pallet-content-rights	98
D.1	Error Variant Coverage	106
D.2	Static Analysis Advisory Breakdown	106
G.1	Monte Carlo Stochastic Parameters (per creator, per iteration)	114
G.2	Monte Carlo Platform Parameters (drawn uniformly once per iteration) . . .	114
G.3	Sensitivity of CCRMS Advantage to Individual Parameter Changes	115
G.4	Sensitivity of CCRMS Advantage to Fee Assumption	115
G.5	Sensitivity of CCRMS Advantage to Centralized Discovery Assumption . . .	116
G.6	Fine-Grained CCRMS Win Rate and Mean Advantage Near the Crossover Audience Size	116
G.7	CCRMS vs. Web3 Marketplace Under Varied Parameters	117

List of Figures

4.1	The three monetization models natively supported by the unified rights record (addresses SQ3). A single content registration may offer all three models concurrently; <code>check_access</code> resolves in priority order: Ownership > Subscription > PPV.	29
4.2	Parent-child NFT nesting model. Each content registration creates one NFT collection containing a parent Content NFT (the canonical on-chain identity of the content) and zero or more child rights NFTs, each owned by an individual user and tagged with their access mode.	29
4.3	XCM cross-chain rights operation (addresses SQ1 and SQ4). The three-instruction sequence dispatches an <code>xcm_*</code> extrinsic on the CCRMS parachain, decoupling the paying account (Para B sovereign account) from the rights beneficiary (the end user). The other <code>xcm_*</code> extrinsics use the same pattern; <code>xcm_transfer_ownership</code> succeeds remotely only when the owner signs the message on Para B and holds ownership through their derived account, not through the sovereign-account path shown here (Finding B, Section 7.3.1 and Appendix D.1).	30
4.4	CCRMS six-layer architecture (addresses SQ1 and SQ2). Each layer is independently replaceable: Layer 5 (smart contracts) may be removed, Layer 6 (XCM) may adopt later XCM versions, and Layer 1 (relay chain) may migrate to JAM, without modifying the business logic in Layer 3.	32

- 5.1 CCRMS implementation-level component diagram. Solid arrows indicate active extrinsic dispatch; dashed arrows indicate read-only storage access. pallet-content-rights is the sole authoritative state layer; the ink! contract and EVM precompile are stateless wrappers that forward calls or read storage, respectively. XCM delivers cross-chain calls via HRMP. pallet-rights-verifier checks Merkle storage proofs of rights state against a caller-supplied state root; binding that root to finalized CCRMS state is not implemented (Section 5.2.1). 36
- 6.1 Four-layer testing architecture (Layer 1 at base: fastest and most granular, run on every commit; Layer 4 at apex: broadest scope, run infrequently). Each layer tests what lower layers cannot: unit tests isolate individual extrinsic logic; the XCM simulator isolates message-construction failures; Zombienet provides definitive wall-clock performance evidence; Snowbridge validates the complete Ethereum bridge pathway. 44
- 7.1 Monte Carlo simulation structure: 1,000 creators $\times$ 10,000 iterations = 10 million revenue paths per platform. Platform fee drawn uniformly each iteration; full parameters in Appendix G. 61

List of Abbreviations

ABI	Application Binary Interface
AI	Artificial Intelligence
API	Application Programming Interface
ASCII	American Standard Code for Information Interchange
BLS	Boneh–Lynn–Shacham (signature scheme)
CAGR	Compound Annual Growth Rate
CCRMS	Cross-Chain Content Rights Management Service
CLI	Command-Line Interface
CPU	Central Processing Unit
DEX	Decentralized Exchange
DOT	Polkadot native token
DRM	Digital Rights Management
DSR	Design Science Research
ESMA	European Securities and Markets Authority
ETH	Ether (Ethereum’s native token)
EU	European Union
EVM	Ethereum Virtual Machine
FRAME	Framework for Runtime Aggregation of Modularized Entities
GB	Gigabyte
H160	160-bit hash (Ethereum-style address)
HHI	Herfindahl-Hirschman Index
HRMP	Horizontal Relay-routed Message Passing
HTTP	Hypertext Transfer Protocol
ID	Identifier

IPFS	InterPlanetary File System
JAM	Join-Accumulate Machine
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
MEV	Maximal Extractable Value
MiCA	Markets in Crypto-Assets (EU Regulation)
MTTR	Mean Time To Recovery
NFT	Non-Fungible Token
OTT	Over-The-Top (media service)
PIL	Programmable IP License (Story Protocol)
PolkaVM	Polkadot Virtual Machine
PPV	Pay-Per-View
RISC	Reduced Instruction Set Computer
RMRK	Remark (NFT protocol; derived from the <code>system.remark</code> extrinsic)
RPC	Remote Procedure Call
SCALE	Simple Concatenated Aggregate Little-Endian (Substrate codec)
SDK	Software Development Kit
SIGKILL	Signal Kill (POSIX signal)
SIGTERM	Signal Terminate (POSIX signal)
SQL	Structured Query Language
TLS	Transport Layer Security
TPS	Transactions Per Second
TVL	Total Value Locked
UI	User Interface
USD	United States Dollar
WASM	WebAssembly
XCM	Cross-Consensus Messaging
ZK	Zero-Knowledge (proof / cryptography)

Introduction

1.1 | The Problem

The prevailing content economy is unfavorable to creators. An independent musician earning USD 100,000 in streaming revenue typically retains between USD 55,000 and USD 70,000 after platform fees and distributor commissions; label-signed artists retain substantially less (Music Business Research, 2024). Centralized intermediaries retain between 30 and 60 percent of gross creator revenue, depending on the platform and contract structure (the per-platform breakdown is documented in Table 7.12). Fees are non-negotiable, audiences are non-portable, and content rights remain bound by the platform's terms of service.

Consumers face similar fragmentation. A film purchased on iTunes cannot be streamed on Amazon Prime; an Audible audiobook cannot be transferred to Google Play; a subscription started on one platform cannot be moved to another. Consumers do not own their purchases; they hold a platform-bound license that can be revoked, modified, or ignored. This fragmentation that constrains creators also restricts the consumers who support them.

Fragmentation extends to monetization models. Subscriptions typically live on Patreon; one-time sales on Gumroad; pay-per-view is scarcely supported because the overhead of conventional payment systems makes micro-transactions uneconomic. Each platform supports a single monetization model, has its own audience, charges its own fees, and produces its own accounting reports. A creator who wants to offer all three models is forced to run three or four platforms in parallel and reconcile them by hand.

Existing blockchain alternatives address only parts of the problem, not the whole. Non-fungible tokens (NFTs) provide transferable ownership but not recurring access or per-view consumption. Subscription-style decentralized platforms such as Audius offer

recurring revenue but are confined to a single chain and audience. Cross-chain bridges move assets between ecosystems but carry no rights metadata during those transfers. Academic work on blockchain-based digital rights management has produced tamper-evident provenance schemes (Garba et al., 2021; Patil et al., 2022), expert-derived design principles for royalty distribution in the music industry (Ciriello et al., 2023), and on-chain licensing frameworks (Mendoza-Tello et al., 2024), each addressing a single dimension of the problem on a single chain. No existing system, centralized or decentralized, combines subscription access, pay-per-view consumption, and permanent ownership in a single rights primitive that is portable across blockchains.

The technical primitives needed to address this gap already exist as standalone capabilities: cross-chain messaging, programmable smart contracts, on-chain royalty distribution, and NFT-based rights tokenization. Hanneke et al. (2024) characterize this class of systems as an emerging *Internet of Value*, in which blockchain tokenization enables asset ownership and revenue flows that bypass centralized intermediaries entirely. However, no prior academic or industry work has integrated these primitives into a system that unifies all three monetization models in a single, cross-chain-portable rights record, the specific gap that Chapter 2 maps across eight dimensions of prior literature (Table 2.1). The research questions that follow from this gap are formalized in Chapter 3.

1.2 | Background Context

This dissertation sits at the intersection of three active research and engineering areas: blockchain-based digital rights management, cross-chain interoperability, and decentralized content monetization. Chapter 2 surveys each area in depth; this section summarizes what has been proposed in those areas and explains why no existing proposal meets the combined requirements.

Story Protocol (now the DATA Foundation; The DATA Foundation, 2026), launched in February 2025, is the closest industry precedent: an EVM-compatible layer-1 blockchain that records IP ownership and licensing terms on-chain and supports programmable royalty flows (Story Foundation, 2025). Like the academic proposals in Section 1.1, it operates on a single chain and does not combine subscription, pay-per-view, and permanent-ownership access.

Cross-chain interoperability surveys (Deng et al., 2025; Li et al., 2025; Ren et al., 2023) examine mechanisms for moving assets between ecosystems but not the portability of rights metadata during those movements, which is the specific contribution of this dissertation's cross-chain design.

1.3 | Research Aim and Approach

The primary research question, which follows from gap G7 (Table 2.1), is:

How can a shared-security, multi-chain framework built on Polkadot’s Cross-Consensus Messaging (XCM) and FRAME pallet primitives deliver a unified rights record that natively supports recurring subscriptions, pay-per-view microtransactions, and permanent ownership transfers across heterogeneous blockchain networks, and what levels of transaction finality, cost efficiency, and creator revenue retention can it achieve?

The question has two mutually constraining components, design feasibility and measurement, which Chapter 3 decomposes.

To address this question, we design, implement, and evaluate a working prototype, the Cross-Chain Content Rights Management Service (CCRMS), built as a Polkadot parachain. The prototype supports all three monetization models through a single unified rights record, enabling automatic royalty distribution and cross-chain portability via XCM. Section 2.2.1 explains why Polkadot’s shared-security architecture suits this design.

Our research approach is grounded in design science research (Hevner et al., 2004) and applied to a multi-component blockchain prototype. We organize the methodology, detailed in Chapter 4, into iterative build-test-measure-refine cycles within a six-phase framework. Our evaluation strategy, as delineated in Chapter 6, combines quantitative measures (including technical benchmarks, security audits, comparative analyses, and Monte Carlo simulations of creator revenue) with qualitative insights from a literature synthesis. We frame the evaluation as measurement-focused rather than target-driven: our objective is to document and analyze the levels of performance, security, and economic viability achieved by the system, rather than to verify conformity with a predefined set of criteria.

1.4 | Contributions

We make three contributions, described in detail in Section 9.1:

1. **A feasibility result for a cross-chain unified rights record.** A single FRAME pallet with a priority-ordered access check across three storage maps encodes subscription, pay-per-view, and permanent ownership as native modes of a single

record that other parachains can acquire via XCM and, given a trusted CCRMS state root, verify by storage proof. This addresses gap G7 (Chapter 2).

2. **Quantified decentralization–latency and revenue trade-offs.** The latency cost of decentralization is measured directly: CCRMS incurs a $67,000\times$ overhead relative to a centralized baseline. The creator-revenue advantage is small and conditional: in simulation, CCRMS out-earns centralized platforms in most scenarios only for creators above approximately 6,000 followers (the 50% win-rate point); below 2,000 followers, its win rate falls below one in ten (8.7% at 1,500–2,000; Table G.6).
3. **A cross-chain rights artifact demonstrating XCM sufficiency.** The CCRMS parachain (two FRAME pallets, a smart contract layer, an EVM precompile, and benchmarking and simulation scripts) demonstrates that the standard three-instruction XCM sequence, available since v3, is sufficient for authenticated, issuance-style cross-chain rights operations without custom protocol extensions. The artifact is licensed under the MIT license.

1.5 | Scope, Claims, and Limitations

This section records what the dissertation demonstrates, what it deliberately does not claim, and the operational constraints that bound how the results should be read.

The research question refers to heterogeneous blockchain networks. The evaluation covers Polkadot parachains, which are distinct runtimes sharing one relay chain, and a Snowbridge path from Ethereum to AssetHub that did not reach the rights pallet. Claims about chains outside Polkadot are therefore limited to forward-compatibility.

1.5.1 | Claims

The following results are demonstrated only within the operational scope of Section 1.5.3:

1. **Unified access encoding.** A FRAME pallet can encode subscription, pay-per-view, and permanent ownership as three native access modes within a single content registration, resolved by a single priority `check_access` call. Demonstrated by the implementation and its unit test suite.
2. **Standard XCM instructions suffice for rights issuance.** The sequence `WithdrawAsset` $\rightarrow$ `BuyExecution` $\rightarrow$ `Transact` is sufficient for issuance-style cross-parachain rights operations with payer–beneficiary decoupling in a local

HRMP topology. No custom XCM extensions are required. The runtime executes XCM v5; the benchmark message was V3-encoded (Section 5.2.1). After the Finding B authorization fix, cross-chain transfer of existing ownership works when the owner signs on the sending chain and holds the right through their derived account, which we tested in the xcm-simulator only (Sections 7.3.1 and 8.1.4); we do not claim a complete cross-chain secondary market.

3. **Audience-conditional revenue advantage.** Under the assumptions documented in Appendix G, CCRMS yields higher creator revenue after fees than centralized platforms in most simulated scenarios once a creator’s audience exceeds approximately 6,000 followers. The mean advantage is small (4.4%) and disappears under modest changes to the fee or discovery assumptions. This is a design-level simulation result, not a measurement of the deployed prototype.
4. In the Zombienet topology described in Section 1.5.3, the prototype met eight of eleven KPI targets in Table 7.15 in every measurement; KPIs 2 and 6 are met at the median, and KPI 9 only as an author-audit interpretation (Section 7.4). Those figures characterize the test environment.

1.5.2 | Non-Claims

The dissertation does not claim, assert, or imply:

- **Cross-chain atomicity or auto-renewal.** XCM Transact does not guarantee atomicity across chains, and cross-chain recurring renewals are not implemented: XCM has no scheduling instruction, so a renewal paid from another chain would require that chain to schedule the messages (e.g., with pallet-scheduler).
- **The prototype has been tested at the audience sizes that win in the simulation.** The former fifty-holder cap was removed, but no test exceeded 60 holders per content item (Section 8.5).

1.5.3 | Operational Limitations

- **The prototype is a back-end service, not a consumer application.** We did not develop a web or mobile interface for CCRMS. Our assessment covers pallet- and runtime-level behavior, not user experience.

- **We measure performance on a local development network.** The prototype runs in a local Zombienet network (a tool for simulating multi-chain Polkadot environments) with two relay-chain validators and single-collator parachains. Production relay chains have hundreds of validators, real network latency, and competing parachains, so the throughput, latency, and reliability figures in Chapter 7 are **upper bounds for the test environment**, not deployment forecasts (Section 8.5).
- **Our Snowbridge integration is local and stops at AssetHub.** We transferred one ETH twice from a local Ethereum chain to a local AssetHub parachain; no public testnet or mainnet was used. The path from Ethereum to a CCRMS rights operation has not been tested end-to-end. The integration demonstrates architectural feasibility, not production validation.
- **We have not commissioned any third-party audit, security review, or independent benchmarking.** A single author conducted the implementation, test design, measurements, and interpretation (Section 8.5). Production deployment would require third-party validation first.

Background and Related Work

This chapter surveys four themes: blockchain-based content management (Section 2.1), cross-chain interoperability (Section 2.2), decentralized monetization models (Section 2.3), and the challenges and gaps that motivate the research (Section 2.4).

2.1 | Blockchain-Based Content Management

2.1.1 | Market Context

Three independent market research firms (Grand View Research, 2025; MarketsandMarkets, 2025; Mordor Intelligence, 2026) converge on estimates that the global DRM market is valued at approximately USD 6.7–6.9 billion in 2025–2026, with projections reaching USD 11–14 billion by 2030–2033, representing a Compound Annual Growth Rate (CAGR) of 10–11%. This growth is primarily driven by streaming services, AI-enabled piracy, and regulatory pressures. Commercial DRM (Widevine, FairPlay, PlayReady) focuses on content protection rather than monetization, as does most academic work (Section 2.1.2).

2.1.2 | Foundational Work on Blockchain-Based DRM

Blockchain-based Digital Rights Management (DRM) is an active research domain. Garba et al. (2021) incorporate digital watermarking into blockchain technology as a tamper-evident layer; Patil et al. (2022) build on this by introducing the DRMChain hybrid encryption architecture; and Yi et al. (2023) further extend the protection model with a redactable blockchain scheme that combines DRM with perceptual hashing, enabling selective content modification without invalidating ownership proofs. These studies prioritize protection mechanisms over the economics of content creators.

In the music industry, Ciriello et al. (2023) derive a set of design principles for blockchain-based DRM systems through expert interviews and prototype testing, identifying transparency of rights ownership, automated royalty distribution, and programmable licensing as the three foundational requirements. Cherdakov et al. (2023) and Mendoza-Tello et al. (2024) propose Web3 implementations that embed these principles in music-streaming smart contracts, demonstrating the feasibility of on-chain licensing but stopping short of cross-chain portability.

The most comprehensive recent work is Madapati and Pradhan (2025), whose Ethereum + IPFS framework achieves 409 TPS for registration; this figure is not directly comparable with ours, because their pipeline hashes off-chain. Wang et al. (2025) present a BLS threshold signature scheme achieving 40 ms aggregation. Two copyright studies (Xie and Liu, 2022; Xie and Tang, 2024) propose multi-chain frameworks; the later one, a relay-chain design with separate chains for registration, supervision and arbitration, names no implementation platform and is evaluated only qualitatively.

This literature establishes **blockchain-based DRM as a legitimate research domain**; its consistent limitation is that it emphasizes protection over monetization, a priority CCRMS reverses.

2.1.3 | NFTs and Intellectual Property Rights

Work on NFTs and intellectual property encompasses legal analysis (Davik, 2025; Eurojust, 2024; Murray, 2022; USPTO, 2024) and standardization proposals (Dreyer et al., 2021; Gatto et al., 2021; Reggianini, 2025). A preprint by Darshan et al. (2025) reports that most mainstream NFTs convey no copyright or usage rights to buyers, that only 12–30% of listings on three major marketplaces include any license information, and that 50–60% of secondary sales on platforms that do not enforce royalties bypass them; the paper does not describe how these figures were obtained, so we treat them as indicative. A bibliometric and systematic review of 190 NFT articles (Lahiri et al., 2026) lists ownership ambiguity and the enforceability of rights among the field’s legal challenges, while noting that legal research has so far concentrated on copyright and intellectual property. Cabot-Nadal et al. (2022) propose an ERC-721 extension enabling recipients to reject unwanted transfers, relevant to our permanent-ownership mode.

The most significant industry development is the February 2025 launch of **Story Protocol**, the first dedicated intellectual-property Layer-1 blockchain (Story Foundation, 2025). Built on the Cosmos SDK with EVM compatibility, it enables programmable IP registration, automated licensing, and on-chain royalty distribution. It is the most direct industry comparator to CCRMS: a single-chain L1 versus a cross-chain parachain.

2.2 | Cross-Chain Interoperability

2.2.1 | Polkadot Architecture

Polkadot (Burdges et al., 2020; Wood, 2016) is a heterogeneous multi-chain framework in which parachains share security through a central relay chain and communicate via XCM. The core primitives are parachains, the relay chain, validators, collators, and XCM. Habermeier et al. (2021) and Wood (2022) provide the primary technical documentation for the XCM message format and multi-hop features that underpin our cross-chain implementation. The XCM protocol itself has received dedicated academic attention: Morhác et al. (2023a) build ParaSpell, a developer SDK that abstracts XCM message construction across parachains, and Morhác et al. (2023b) demonstrate XCMP interoperability at the protocol level. Together with Morhác et al. (2025), which proposes the Unispell universal adapter, they form the most coherent body of peer-reviewed academic work on XCM in practice.

Bridge-based systems such as Wormhole and LayerZero accept a message when parties outside both chains' consensus attest to it, so compromising those parties allows forged ownership proofs or redirected royalty payments. Polkadot removes that external layer (Burdges et al., 2020): the same relay-chain validator set checks every parachain block, and HRMP carries messages through relay-chain storage, so a delivered message is as final as the relay chain itself. Shared security guarantees that each parachain followed its own rules; it does not guarantee that those rules are benign. A sibling parachain can still send well-formed but hostile XCM, so the receiving chain must decide what each origin may do. CCRMS makes that decision through its XCM barrier, origin conversion and pallet-level checks (Appendix D.1).

2.2.2 | Cross-Chain Technology Surveys

The cross-chain literature matured rapidly from 2022 to 2025, and its surveys classify the same mechanisms in different ways. Ren et al. (2023) sort solutions into five categories: sidechains, notary schemes, hashed time lock contracts, relays, and blockchain-agnostic protocols. Mao et al. (2023) propose five overlapping methods (hash locking, notary schemes, distributed private-key control, sidechains and relays, and tight coupling); Deng et al. (2025), a preprint drawing on more than 150 sources, group mechanisms instead by where verification takes place; and Li et al. (2025) distinguish **asset interoperability** (token transfers) from **data interoperability** (data exchange and contract calls across chains). Bigiotti et al. (2025) narrow the focus to blockchain-agnostic protocols that

carry generic messages, and are the only one of these surveys to examine XCM in depth. Three points recur. First, asset transfer is far better developed than message or data exchange; Mao et al. (2023) observe that most projects still lack practical work on data communication. Second, every mechanism retains some trust assumption, and Deng et al. (2025) note that cross-chain exchange cannot be made entirely free of trusted parties. Third, shared-security designs such as Polkadot are described but not evaluated: Deng et al. (2025) list relay-chain shared security as Polkadot's security mechanism without analyzing what a receiving chain must still check. None of the surveys considers rights, licensing, or content metadata crossing chains. CCRMS falls under data interoperability: it moves value through XCM payments and verifies state in part through pallet-rights-verifier Merkle proofs, which require a trusted CCRMS state root that the prototype does not yet supply.

2.2.3 | Bridges and Snowbridge

Bridge designs differ chiefly in whom the receiving chain must trust. Committee-based bridges replace a single notary with a threshold of signers: Bool Network (Yin et al., 2022) uses hidden, rotating committees that hold threshold keys inside trusted execution environments, and the Twin-Token Protocol of Kumar et al. (2025) relies on a threshold-signing Internet Computer subnet to mirror token transfers between EVM chains; both remain safe only while an honest threshold of signers (and, for Bool, the trusted hardware) holds. Chen et al. (2023) add reputation scoring to a relay-chain architecture, which filters misbehaving nodes but does not prove what happened on the source chain. MAP (Cao et al., 2025) verifies each source chain with on-chain light clients and zero-knowledge proofs, yet still assumes an honest two-thirds of its relay chain's stake and, by the authors' own analysis, is only as secure as the weakest connected chain. Maric et al. (2025) prove in Isabelle/HOL that a lock-and-mint bridge is fail-safe, meaning users can always recover their assets, under the explicit assumption that the validators who post state roots are honest. Two observations follow for CCRMS. Each design reduces trust rather than removing it, and each concerns token transfer rather than authorizing arbitrary calls on the receiving chain. That second question, which origin may invoke which operation, remains even under shared security (Section 2.2.1).

The most operationally significant development in blockchain infrastructure is **Snowbridge**, a trust-minimized bridge that enables interoperability between Polkadot and Ethereum. Snowbridge V2 was officially launched on the Polkadot mainnet on November 7, 2025, and includes support for arbitrary Ethereum contract calls from Polkadot, a 50% fee reduction, and unordered message execution (Snowfork, 2025a). The associated

Polkasassembly proposal (Snowfork, 2025b) documents more than a year of operational stability with no on-chain downtime. We developed and tested our integration against V1 and have not tested it against V2.

2.2.4 | Polkadot 2.0 and JAM

Two ecosystem developments shaped our implementation. **Polkadot 2.0** groups four changes that reached the production network during 2025 (Parity Technologies, 2025; Polkadot Network, 2025). Asynchronous backing lets collators build on parachain blocks that the relay chain has not yet included, which halves the parachain block time from 12 to 6 seconds and raises the execution time available per block from 0.5 to 2 seconds (Web3 Foundation, 2026). Agile Coretime replaces fixed-term slot auctions with the sale of core time, in bulk periods on a dedicated Coretime chain or block by block on demand (Wood, 2023). Elastic scaling lets one parachain use several cores at once (Parity Technologies, 2026). XCM v5 adds, among other changes, a `PayFees` instruction that succeeds `BuyExecution` (Aguirre, 2024), an `InitiateTransfer` instruction that combines several asset-transfer types in one message (Catangiu, 2024), and origin aliasing through `AliasOrigin`. Each bears on CCRMS: our testnet ran with asynchronous backing, elastic scaling is the main remaining route to higher throughput for a single chain (Section 8.2.1), and `AliasOrigin` would let remote users act as their local accounts (Appendix E.8). **JAM** (Join-Accumulate Machine, Wood (2024)) is expected to succeed the relay-chain model within the Polkadot ecosystem, with a public testnet since January 2026; as of September 2026 no mainnet date has been announced.

2.2.5 | Cross-Chain Copyright Management as a Research Target

Work on blockchain copyright management spans legal analysis (Buzu, 2021; Mareckova, 2024), qualitative study of rights holders (Orero et al., 2023), and conceptual multi-chain designs (Xie and Liu, 2022; Xie and Tang, 2024); little of it involves cross-chain execution. The one deployed multi-chain system we found, Eluvio’s Content Fabric (Munson et al., 2022), gates access to content on token ownership recorded on another chain, but by its own description it learns that ownership from a trusted oracle that signs the result, not through messages secured by a shared validator set. We found no work that combines subscription, pay-per-view, and ownership in a single record that users on other chains can acquire through such messages. This gap, together with the architectural analysis above, motivates selecting XCM and FRAME as the implementation substrate and Polkadot over single-chain or bridge-based alternatives.

XCM is preferred over Wormhole and LayerZero because it is native to the relay chain and adds no trusted party beyond the relay-chain validators and the sending chain's own runtime. Its structured instruction format also encodes rights operations as typed, on-chain-interpretable messages rather than opaque byte payloads, preserving rights metadata across chain boundaries (gap G3). FRAME is the implementation framework of choice over EVM and CosmWasm alternatives because its composable module system allows all three access modes to share a single pallet and access-check function, its weight benchmarking provides deterministic fee calculations, and its typed storage API exposes rights state as queryable on-chain objects via the RPC layer.

2.3 | Decentralized Monetization Models

Creator monetization is a growing commercial target for blockchain systems: one market report values blockchain-based content monetization at USD 499.52 million in 2024 and projects USD 2.74 billion by 2030 (360iResearch, 2025). The work reviewed here falls into three strands: subscription and purchase models, pay-per-view and the micropayment problem, and the smart-contract layer that implements them.

2.3.1 | Subscription, Pay-Per-View, and Purchase Models

Earlier work on blockchain subscription and pay-per-view models includes Banerjee et al. (2020); Munson (2019); Steem Team (2025). Most of these works examine blockchain incentives without addressing the micropayment problem. Two recent works more closely align with our aims. Osemwegie (2025) reviews blockchain-based media monetization, including micropayments, tokens, and NFTs, and identifies the problems CCRMS targets but does not propose a system. Shah (2025) outlines an Ethereum smart-contract framework in which oracles report usage from streaming platforms and a registry contract splits royalties among rights holders; it is the nearest precedent for our royalty splitting, but it is unimplemented, runs on a single chain, and does not propagate royalties through derivative works. Other work on royalty automation is scarce; Reggianini (2025) addresses part of the licensing standardization problem for European NFTs. CCRMS implements a comparable royalty split in a FRAME pallet, in which all split payments within one extrinsic succeed or fail together (Appendix D.6). Wamugo (2024) provides empirical data on NFT adoption by media organizations in Kenya, demonstrating that blockchain monetization is under active consideration in emerging markets.

The EVM ecosystem has standardized parts of this design space. ERC-5643 (cygaar, 2022) extends ERC-721 with subscription expiry and renewal; ERC-4907 (Anders and

Shrug, 2022) separates an NFT’s owner from a time-limited user; and Unlock Protocol (Unlock Protocol, 2026) deploys renewable, optionally transferable membership keys in production. Superfluid (Superfluid, 2026) addresses recurring payments through continuous token streams. Together, they show that time-bound and ownership-based access can be expressed as token state on a single chain. What they do not provide is (i) counter-based pay-per-view within the same record, or (ii) a way for a user on another chain to acquire or prove these rights without a bridge. CCRMS targets that intersection. On Polkadot, Unique Network (Unique Network, 2026) provides native NFT nesting at the runtime level, an alternative to the `pallet-nfts` reimplementation discussed in Appendix A.1.

2.3.2 | Pay-Per-View and the Micropayment Problem

Pay-per-view models have historically struggled on blockchain platforms because gas fees have often exceeded content costs. Goyal et al. (2019) introduce Gringotts, a secure micropayment system, but do not address the core fee issue. Chegenizadeh et al. (2024) directly address the micropayment verification layer, proposing a multi-agent, privacy-preserving approach for resource-constrained offline devices that reduces per-transaction overhead through smart-contract batching; their work supports our strategic decision to model PPV as a counter rather than a sequence of individual on-chain transactions.

2.3.3 | Smart Contracts as the Implementation Layer

Surveys of the smart contract literature (Ante, 2020; Lin et al., 2022; Quan et al., 2024; Singh et al., 2024) establish the field as mature; Parity’s ink! blog posts (Parity Technologies, 2022a,b) were the primary technical references during implementation. The most consequential development is the **discontinuation of ink! in January 2026** (ink! Alliance, 2026), discussed as gap G8 and in Appendix A.9.

2.4 | Challenges and Gaps

Table 2.1 maps eight literature gaps to CCRMS contributions.

Table 2.1. Literature Gaps and CCRMS Contributions

ID	Gap	Supporting literature	CCRMS contribution
G1	Existing systems are protection-focused, not monetization-focused	Garba et al. (2021); Patil et al. (2022); Ciriello et al. (2023); Madapati and Pradhan (2025); commercial DRM (Widevine, FairPlay, PlayReady)	Unified rights token with monetization as the primary design goal; protection becomes a secondary consequence of correct rights management.
G2	No native cross-chain recurring payments	Existing cross-chain protocols (XCM, Wormhole, LayerZero, Snowbridge) address one-time transfers; Xie and Liu (2022); Xie and Tang (2024) propose theoretical frameworks without recurring payments	On-chain <code>on_initialize</code> block hook scheduler for automatic renewal. XCM has no scheduling instruction; cross-chain renewal would require the paying chain to schedule XCM sends (e.g., <code>pallet-scheduler</code>), deferred to future work.
G3	Cross-chain protocols strip rights metadata on transfer	Reggianini (2025) addresses part of the standardization problem within European NFT licensing	<code>query_rights_metadata</code> extrinsic emitting a self-describing <code>RightsMetadata</code> event; metadata canonical on CCRMS, verifiable with <code>pallet-rights-verifier</code> Merkle proofs given a trusted CCRMS state root.

Continued on next page

Table 2.1 continued from previous page

ID	Gap	Supporting literature	CCRMS contribution
G4	Pay-per-view is economically unviable at micro-transaction scale on traditional blockchains	Goyal et al. (2019); Chegenizadeh et al. (2024); x402 (Coinbase and x402 Foundation, 2025)	PPV reconceptualized as a counter: one payment per pack, cheap storage updates per view, with additive purchases (Appendix C.4).
G5	The decentralization–latency trade-off is poorly characterized quantitatively	Most works endorse decentralization qualitatively or measure only centralized performance; Madapati and Pradhan (2025) is an exception but uses an off-chain pipeline; Rao et al. (2024) provide a scalability review but no cross-paradigm comparison	Direct comparative benchmark against an Express.js + SQL.js baseline (Section 7.1.8) plus HHI calculation (Section 7.3.3) make the trade-off numerical.
G6	Regulatory uncertainty for tokenized rights (EU AI Act Aug 2025; MiCA full enforcement Jul 2026; EP report A10-0019/2026)	European Securities and Markets Authority (2025); Guadamuz (2025)	Cryptographic provenance, immutable records, and automated royalty distribution align with emerging compliance infrastructure. MiCA classification of subscription tokens remains an open legal question.

Continued on next page

Table 2.1 continued from previous page

ID	Gap	Supporting literature	CCRMS contribution
G7	No existing system provides subscription, pay-per-view, and permanent ownership as native access modes for a single rights record that remains authoritative on one chain while being purchasable and verifiable on other chains under shared security. Single-chain standards approximate parts of this: ERC-5643 (cygaar, 2022) (expiry and renewal), ERC-4907 (Anders and Shrug, 2022) (owner/user separation), and Unlock Protocol (Unlock Protocol, 2026) (renewable membership keys). None provides counter-based access, and none is portable across chains without a bridge.	Patreon (subscription only); OpenSea (ownership only); pay-per-view absent from most blockchain systems; Story Protocol (flexible licensing primitive, not unified rights); Shah (2025) addresses royalty automation as a single feature	Unified rights token: one Content record, three maps, one ordered <code>check_access</code> , and renewal/expiry as first-class storage transitions, not as configurations of a generic licensing primitive. We do not claim that PIL cannot encode similar commercial outcomes.

Continued on next page

Table 2.1 continued from previous page

ID	Gap	Supporting literature	CCRMS contribution
G8	ink! discontinuation (January 2026) leaves Polkadot without a viable institutionally supported smart contract language; alternatives (revive, raw pallet-revive, wrevive) each carry trade-offs	Appendix A.9	Pallet-first architecture demonstrates that substantial functionality can be delivered without a smart contract language. The gap remains for systems whose business logic is better expressed in contracts.

The prototype addresses G1 and G7 directly, and G4 and G5 within its scope (a counter-based pay-per-view model and a local-testnet benchmark). Its contributions to G2 and G3 are partial: renewal is automated on CCRMS only, and metadata verification requires a trusted CCRMS state root. G6 and G8 depend on factors outside the prototype's control. The collective contribution is that a single working prototype demonstrates feasibility for a category of systems that the literature has addressed only partially.

2.5 | Summary

We surveyed the academic and industry literature on blockchain-based content management, cross-chain interoperability, and decentralized monetization. The foundational primitives are in place; commercial interest in decentralized content monetization is growing; and existing work addresses the problem only in parts, mostly through legal analysis, conceptual designs, or single-chain systems. CCRMS addresses two of the eight identified gaps directly and the others in part (Section 2.4) by deploying a unified rights record on a Polkadot parachain.

Research Questions and Objectives

This chapter presents the research questions and objectives that underpin our study. We formulated them during Phase 1 and refined them in Phase 2 following the gap analysis of Chapter 2.

3.1 | Primary Research Question

The primary research question of our work is:

How can a shared-security, multi-chain framework built on Polkadot's Cross-Consensus Messaging (XCM) and FRAME pallet primitives deliver a unified rights record that natively supports recurring subscriptions, pay-per-view micro-transactions, and permanent ownership transfers across heterogeneous blockchain networks, and what levels of transaction finality, cost efficiency, and creator revenue retention can such a system achieve?

The question has two components: **design feasibility** (whether such a system can be built using the available Polkadot SDK primitives) and **measurement** (what operational properties it exhibits once built). We treat them as mutually constraining; a design that is feasible to build yet performs poorly is not viable, nor is one that performs well in benchmarks but cannot be implemented in practice. The primary concern is framed as an **achievability** question rather than a hypothesis test, consistent with the measurement-focused evaluation philosophy established in Section 4.1.4.

3.2 | Sub-Questions

We decompose the primary research question into five sub-questions. Table 3.1 presents each sub-question, its motivation, and the literature gaps it addresses.

Table 3.1. Research Sub-Questions

ID	Question	Gaps	Motivation
SQ1	To what extent can XCM carry recurring subscription renewal messages and rich rights metadata across Polkadot parachains and external networks, and what atomic success rates and cross-chain finality times are achievable?	G2, G3	Existing cross-chain protocols lack native recurring payments and strip rights metadata. SQ1 asks whether standard XCM instructions (available since v3) can approximate subscription renewal behavior without custom extensions; XCM, including v5, has no scheduling instruction, so recurring behavior must come from runtime logic.
SQ2	What design patterns can minimize access verification latency while maintaining decentralization, and how do the resulting metrics compare across varying validator configurations?	G5	The decentralization-latency trade-off is poorly characterized quantitatively. SQ2 asks for a direct measurement on a working Polkadot parachain with explicit numerical comparisons against centralized alternatives.

Continued on next page

Table 3.1 continued from previous page

ID	Question	Gaps	Motivation
SQ3	Can a single on-chain rights object simultaneously enforce subscription auto-renewal, pay-per-view consumption limits, and permanent ownership with automated royalty distribution across chains, and what are the resulting creator cost savings?	G7	No existing system offers all three monetization models as first-class access modes of a single unified primitive. Story Protocol's PIL can be configured to approximate them but requires separate license-token types and does not natively model subscription renewal. SQ3 asks whether a unified primitive can be built and whether its economics justify the effort.
SQ4	What authorization model is appropriate for cross-chain rights operations that decouple the payer from the beneficiary, and how does it perform under adversarial conditions?	(added Phase 5)	Added in response to a finding from the security audit that cross-chain extrinsics (which decouple the transaction payer from the rights beneficiary) introduce authorization vulnerabilities absent in single-chain calls (Section 7.3.1, Finding B). SQ4 formalizes this concern and asks which authorization model eliminates it.
SQ5	Under realistic creator audience distributions, does the CCRMS model produce demonstrably better creator outcomes than centralized, bridge-based, or marketplace alternatives, and under what conditions?	(added Phase 5)	Added during the Monte Carlo simulation. SQ5 clarifies that the economic question is conditional ("yes, for creators with these characteristics") rather than a simple yes-or-no.

3.3 | Research Objectives

We pursue the five sub-questions through six research objectives. Table 3.2 summarizes each; Appendix B provides the full statements, deliverables, and revision notes.

Table 3.2. Research Objectives Summary

Objective	Focus	Sub-questions
RO1	Design and prototype XCM extensions for recurring and metadata-rich rights transfers	SQ1, SQ2
RO2	Implement a unified rights record (originally ink!, revised to FRAME pallet) with access-verification latency measurement	SQ2, SQ3
RO3	Merge subscription, PPV, and purchase logic into one composable rights pallet	SQ3, SQ4
RO4	Benchmark scalability, interoperability, and economic efficiency against centralized and bridge-based alternatives	SQ2, SQ5
RO5	Incorporate optional zero-knowledge proof hooks for selective regulatory disclosure (restated as future work in Section 9.3)	Broader feasibility
RO6	Quantify creator savings via Monte Carlo simulation and comparative cost analysis	SQ5

3.4 | Mapping Questions to Objectives, Methods, and Chapters

Table 3.3 maps each sub-question to its research objectives, evaluation methods, and the sections where results are reported.

Table 3.3. Mapping of Sub-Questions to Objectives, Methods, and Chapters

Sub-Question	Primary Objective	Supporting Objective(s)	Primary Method	Reported In
SQ1 (XCM rights operations)	RO1	RO3	XCM latency benchmark; end-to-end XCM test	Section 5.2; Section 7.1.3; Section 7.1.7
SQ2 (Latency vs decentralization)	RO2, RO4	–	Throughput and latency benchmark; HHI calculation	Section 7.1.3; Section 7.3.3
SQ3 (Unified rights token)	RO3	RO2, RO6	Unit tests; royalty propagation tests; Monte Carlo simulation	Section 5.1; Section 7.2
SQ4 (Security and authorization)	(added Phase 5)	RO3	Security audit; authorization unit tests	Section 5.3; Section 7.3.1
SQ5 (Economic viability)	RO6	RO4	Monte Carlo simulation; comparative benchmark	Section 7.2; Section 7.1.8

SQ4 and SQ5 were added in Phase 5 (no direct Phase 2 objective link); RO5 was not implemented and is deferred to future work in Chapter 9.

3.5 | Summary

The five sub-questions and six research objectives are grounded in the gaps identified in Chapter 2 and mapped to evaluation methods and reporting chapters in Table 3.1. SQ4 (security) and SQ5 (economic viability) were added in Phase 5, consistent with the DSR principle that the search process itself may modify the initial framing.

Methodology and System Design

This chapter outlines the research methodology we used to develop CCRMS and the resulting system design. Section 4.1 presents the methodology, including design science framing, iterative prototyping, data collection, and evaluation philosophy. Section 4.2 describes the system design at the conceptual level; Chapter 5 covers implementation details.

4.1 | Research Methodology

4.1.1 | Design Science Research Framing

CCRRMS is fundamentally a systems-building project with a primary artifact: a working software prototype. The most appropriate methodological framework is therefore **design science research (DSR)**, as defined by Hevner et al. (2004). Our dissertation aligns with Hevner’s seven DSR guidelines as follows: (1) *design as an artifact*: the CCRMS parachain and seven implementation specifications; (2) *problem relevance*: the commercial and academic gaps identified in Chapter 2; (3) *design evaluation*: eleven KPIs measured in artificial evaluation episodes, classified in Table 6.2; (4) *research contributions*: the artifact, the design knowledge captured in the specifications, and the empirical measurements; (5) *research rigor*: established Polkadot SDK primitives, standard Rust testing, and standard Monte Carlo methodology; (6) *design as a search process*: four design iterations (Section 4.1.5); (7) *communication of research*: this dissertation and an open-source code release. The work also follows the six activities of the DSR process model of Peffers et al. (2007): problem identification and motivation (Chapters 1 and 2), objectives of a solution (Chapter 3), design and development (Chapters 4 and 5), demonstration (the

Zombienet and xcm-simulator runs), evaluation (Chapters 6 and 7), and communication (this dissertation and the code release).

DSR is particularly appropriate here because it treats both the artifact and the design knowledge it produces as coequal contributions.

4.1.2 | Iterative Prototyping

Within the DSR framework, the development process follows an **iterative prototyping** model (build, test, measure, refine). We chose iteration deliberately: the Polkadot SDK monorepo releases stable versions approximately quarterly, making a waterfall approach prone to conclusions based on an outdated toolchain. The architectural intricacies of a cross-chain system mean that certain design decisions can be thoroughly assessed only once the system is operational. In practice, this iterative methodology led to four design iterations during Phase 4 (Section 4.1.5). We executed the work across six four-week phases (Appendix F); references to “Phase N” throughout the dissertation denote work produced in that phase.

4.1.3 | Data Collection Methods

Our evaluation uses a mixed-methods approach: quantitative measurements include unit tests, throughput and latency benchmarks, storage growth, cross-chain XCM latency, reliability tests, a centralized comparative benchmark, and a Monte Carlo simulation of creator revenue. Qualitative inputs are drawn from a literature synthesis. Chapter 6 describes each method. All quantitative outputs are systematically stored as machine-readable JSON files in the `scripts/perf/results/` directory to ensure reproducibility.

4.1.4 | Evaluation Philosophy: Measurement-Focused, Not Target-Driven

We adopt a **measurement-focused philosophy**: the goal is to document and analyze the system’s performance, efficiency, security, and economic viability rather than to verify that it meets predetermined targets. This measurement-centric approach preserves academic rigor regardless of whether specific results meet hypothetical benchmarks, treats unexpected results as findings, encourages truthful reporting, and establishes a baseline for future researchers.

Each of the eleven KPIs in Section 6.5 has a target derived from the Phase 2 methodology, but we treat those targets as reference points rather than pass/fail criteria. The outcome column in Table 7.15 is therefore headed “Target met?” rather than Pass/Fail.

4.1.5 | Methodology Pivots

Implementation produced four design iterations: `pallet-nfts` for RMRK 2.0 (Appendix A.1), `pallet-first` for contract-first (Appendix A.2), an `on_initialize` hook for scheduled XCM, and `Zombienet` for Kusama. These constitute the DSR search process. The remaining challenges were implementation defects (Appendices A.3–A.6), a security finding (Appendix A.7), a known limitation (Appendix A.8), and an external event (Appendix A.9).

4.2 | System Design

This section outlines the conceptual design. Section 5.1 provides the implementation details.

4.2.1 | The Unified Rights Record Concept

This subsection addresses SQ3: it demonstrates that subscription, pay-per-view, and permanent ownership can serve as first-class access modes for a single on-chain record, rather than as separate platform categories.

The principal intellectual contribution of CCRMS is the **unified rights record** and its cross-chain consumability: a single on-chain record that represents three modes of content access (an active subscription, a remaining pay-per-view balance, or permanent ownership) for a single content item, that other parachains can acquire via XCM and, given a trusted CCRMS state root, verify by storage proof.

This addresses gap G7 (Table 2.1): one content registration supports all three modes concurrently, with resolution via a single priority check.

4.2.2 | The Three Monetization Models

The academic design challenge is to encode three semantically distinct access modes (time-bounded, counter-based, and indefinite) in a single on-chain record without cross-contamination of state across modes. The literature identifies subscription, pay-per-view, and permanent ownership as the canonical creator-economy requirements (Ciriello et al.,

2023; Mendoza-Tello et al., 2024), but addresses each in isolation; CCRMS treats their unification into a single primitive as the central design problem.

The architecture supports three models, each motivated by a different gap in the existing landscape:

- **Subscription:** time-limited access valid for a configurable number of blocks per content item, with optional automatic renewal triggered by an on-chain block hook.
- **Pay-per-view (PPV):** prepaid access to a specified number of views of a content item is available. Each `consume_view` operation decrements the counter; additional purchases increase the count rather than overwrite it. Conceptualizing PPV as a counter rather than a sequence of individual view transactions avoids the micro-transaction problem that has historically impeded on-chain PPV models (Chegenizadeh et al., 2024).
- **Permanent ownership:** unrestricted and transferable access. Ownership is prioritized during the access verification process. Transfers burn the seller's child NFT and mint a new one for the buyer instead of moving the NFT. This keeps the Ownership map and the NFT holder in step, but the new NFT has a new item ID, so the token itself carries no ownership history; the chain of owners must be read from `OwnershipTransferred` events, which only archive nodes or indexers retain.

A single content item can offer all three models concurrently, each with independently configured pricing. The `check_access` function resolves rights in priority order (Ownership > Subscription > PPV).

4.2.3 | NFT-Based Rights Tokenization

NFT-based tokenization provides the technical foundation for SQ3: standard `pallet-nfts` primitives can encode RMRK 2.0 composability semantics for a unified token, without relying on unmaintained pallets.

CCRMS encodes content rights as non-fungible tokens using a parent-child nesting model built on `pallet-nfts`. The decision to use NFTs rather than a bespoke storage configuration was motivated by their widespread acceptance as a standard, compatibility with existing digital wallets, and the clear separation of the content item's identity (the parent) from user-specific rights derived from it (the children). The storage maps are authoritative; child NFTs are non-transferable receipts, and ownership moves only through `transfer_ownership`. The conceptual model: each content registration is an NFT collection comprising one parent Content NFT and zero or more child rights NFTs,

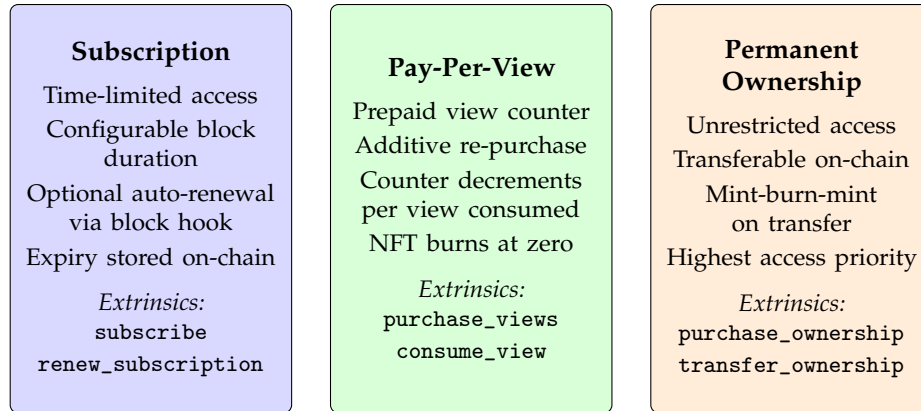


Figure 4.1. The three monetization models natively supported by the unified rights record (addresses SQ3). A single content registration may offer all three models concurrently; `check_access` resolves in priority order: Ownership > Subscription > PPV.

each tagged with a `rights_type` attribute (Subscription, PPV, or Ownership) and owned by the respective user. The parent-child structure serves three purposes: establishing identity, tokenizing individual rights, and enabling on-chain auditability of the rights-to-content relationship.

The nesting pattern is inspired by RMRK 2.0, the composable NFT specification for Polkadot; however, it is implemented directly on top of `pallet-nfts` rather than relying on the `rmrk-substrate` pallets, which are locked to SDK 0.9.36. Appendix A.1 details this transition. The outcome shows that RMRK-style composability can be achieved using only standard FRAME primitives.

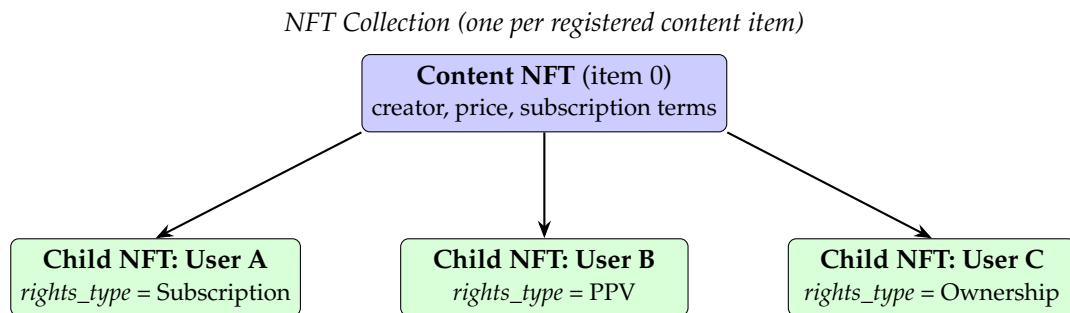


Figure 4.2. Parent-child NFT nesting model. Each content registration creates one NFT collection containing a parent Content NFT (the canonical on-chain identity of the content) and zero or more child rights NFTs, each owned by an individual user and tagged with their access mode.

The prototype originally kept a `Children` index as a `BoundedVec` (a FRAME collection type with a fixed upper bound), which limited each content item to fifty holders over its

lifetime. Because holders are already recorded in the three rights maps and nesting in the Parent map, commit 2865ed0 removed the index and the cap; Appendix A.8 describes how the bound was found and Appendix E.4 what remains.

4.2.4 | Cross-Chain Communication Model

This model addresses SQ1 (XCM rights operations across chains) and SQ4 (payer–beneficiary authorization): the two properties that make cross-chain rights management nontrivial are the decoupling of payment from rights receipt and the verification of remote rights state.

Cross-chain operation is the mechanism by which the unified rights record model enables portable digital ownership: a user who acquires rights on one blockchain should be able to use them on another without losing access or repurchasing. The model has two conceptual properties.

Payer–beneficiary decoupling. Each cross-chain rights operation distinguishes between the funding account and the rights recipient account. In a local call, the payer and the beneficiary are the same (the caller); in a cross-chain call, the dispatch origin is the sovereign account of the remote parachain, and the beneficiary is a separate user account. This configuration allows a remote parachain to acquire rights on behalf of its users without requiring those users to be present on CCRMS.

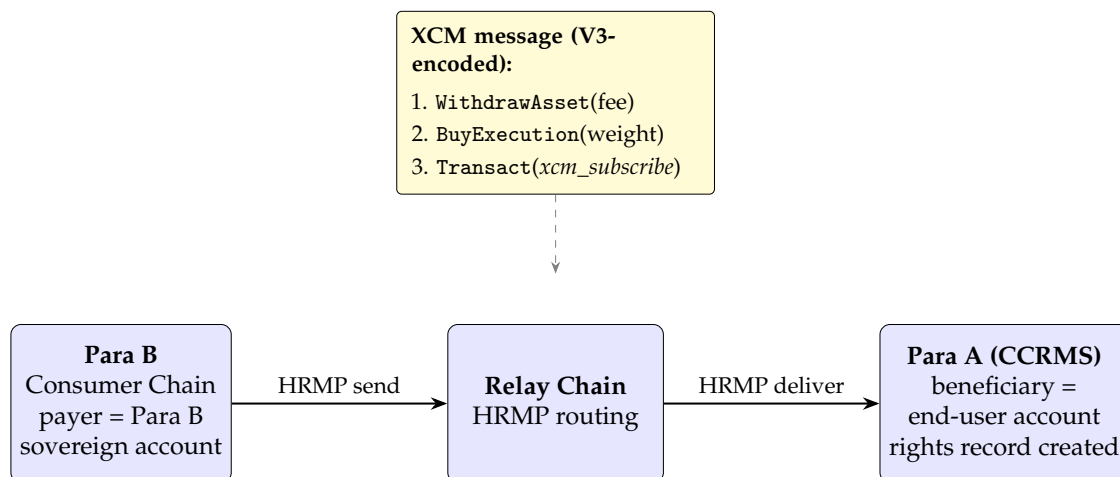


Figure 4.3. XCM cross-chain rights operation (addresses SQ1 and SQ4). The three-instruction sequence dispatches an `xcm_*` extrinsic on the CCRMS parachain, decoupling the paying account (Para B sovereign account) from the rights beneficiary (the end user). The other `xcm_*` extrinsics use the same pattern; `xcm_transfer_ownership` succeeds remotely only when the owner signs the message on Para B and holds ownership through their derived account, not through the sovereign-account path shown here (Finding B, Section 7.3.1 and Appendix D.1).

Verification of remote rights. A second pallet, `pallet-rights-verifier`, checks Merkle storage proofs of CCRMS rights state. Given a CCRMS state root it trusts, a consuming chain can verify that an account holds a subscription, view pack, or ownership record without querying CCRMS. The prototype does not supply that trusted root; the caller provides it (Section 5.2.1). The design principle is that CCRMS rights are **canonical on the CCRMS chain**: other chains check them against CCRMS state rather than maintaining replicas.

The model also extends beyond Polkadot via **Snowbridge**, a trust-minimized bridge connecting Polkadot and Ethereum. We configure the runtime to accept messages from Ethereum using the standard `LocationToAccountId` converters. At this stage, our architectural choice was to ensure **forward compatibility with Ethereum bridging without relying on it**. Section 5.2 provides a comprehensive account of the cross-chain integration.

4.2.5 | Pallet-First Architecture

The pallet-first decision directly shapes the answers to SQ2 (latency versus decentralization) and SQ3 (unified primitive): placing business logic in a FRAME pallet rather than in smart contracts reduces execution overhead and eliminates the chain-extension boundary that would otherwise constrain write operations.

The most significant architectural decision was to centralize the business logic within a **FRAME pallet** rather than in smart contracts. This decision was made during Phase 4 for three reasons: the operational maturity gap between FRAME pallets and `pallet-revive`; the unavailability of the chain extension mechanism, which the original contract-first approach relied on for write operations; and a pallet-first design that uses standard Polkadot wallets without EVM-style adaptation. Appendix A.2 documents this decision; the January 2026 ink! discontinuation announcement (Appendix A.9) confirmed the engineering rationale.

4.2.6 | Layered Architecture

The layered architecture determines the answers to SQ1 (cross-chain reach) and SQ2 (the latency–decentralization trade-off): each layer can evolve independently, so XCM protocol upgrades and relay-chain migration to JAM do not require changes to the Layer 3 business logic.

The system is organized into six distinct, independently replaceable layers: (1) the **Polkadot relay chain** provides shared security and enables HRMP message passing;

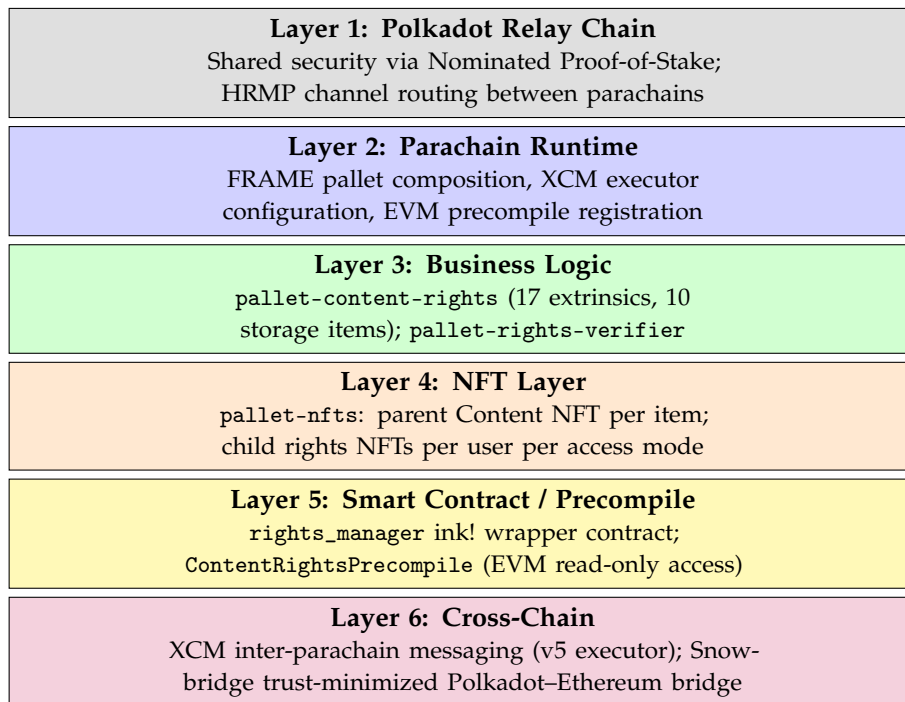


Figure 4.4. CCRMS six-layer architecture (addresses SQ1 and SQ2). Each layer is independently replaceable: Layer 5 (smart contracts) may be removed, Layer 6 (XCM) may adopt later XCM versions, and Layer 1 (relay chain) may migrate to JAM, without modifying the business logic in Layer 3.

(2) the **parachain runtime** composes the CCRMS FRAME pallets, configures the XCM executor, and includes a custom EVM precompile; (3) the **business logic** layer comprises pallet-content-rights (17 extrinsics, 10 storage items) and pallet-rights-verifier; (4) the **NFT layer** uses the standard pallet-nfts, with parent-child nesting enabled by pallet-content-rights; (5) the **smart contract / precompile layer** provides EVM- and ink!-compatible access through the rights_manager contract and the ContentRightsPrecompile, both of which are non-load-bearing wrappers; (6) the **cross-chain layer** manages XCM messaging (the runtime executes XCM v5) and the Snowbridge integration. Independence among layers is a fundamental design objective: the smart contract layer may be removed or replaced, the XCM protocol may evolve beyond v5, and the relay chain may transition to JAM, all without affecting the pallet code.

4.3 | Summary

This chapter framed CCRMS within the design science research paradigm and presented the six conceptual design decisions that govern implementation: the unified rights record (addressing gap G7), the three monetization models, NFT-based tokenization via pallet-nfts, the cross-chain payer-beneficiary model, the pallet-first architecture, and the six-layer system structure. Chapter 5 covers implementation details.

Implementation

This chapter describes what was built and how key decisions played out in practice. The development environment, toolchain, and workspace layout are documented in Appendix C. Section 5.1 summarizes the pallet implementation; Section 5.2 covers cross-chain integration; Section 5.3 documents the nine challenges encountered and their resolutions.

5.1 | Pallet Implementation and Smart Contract Layer

Figure 5.1 shows the six implementation components and their relationships.

The core of CCRMS is `pallet-content-rights`, a FRAME pallet that implements all business logic: content registration, the three monetization models (subscription, pay-per-view, and permanent ownership), on-chain royalty distribution to up to ten collaborators, automated subscription renewal via an `on_initialize` hook that processes a `RenewalQueue` keyed by expiry block, and cross-chain variants of every financial operation. It exposes 17 dispatchable extrinsics, 10 storage items, and 22 event variants; the full reference tables are in Appendix C. `check_access` is a signed extrinsic, so each call is recorded on-chain for auditability; a free, read-only view function, `access_rights_of`, returns the same answer without a transaction (commit 5cb09cd). Neither enforces access by itself: a content server must query CCRMS and serve content only on a positive answer, and the prototype does not include such a server. Neither returns anything to a caller on another chain; given a trusted CCRMS state root, other chains could check rights status with `pallet-rights-verifier` (Section 5.2.1). Content rights are represented as NFT collections using a parent-child nesting model built on `pallet-nfts`: a parent Content NFT is the canonical on-chain identity for each piece of content, and each user's access rights are a child NFT tagged with a `rights_type` attribute (Subscription, PPV,

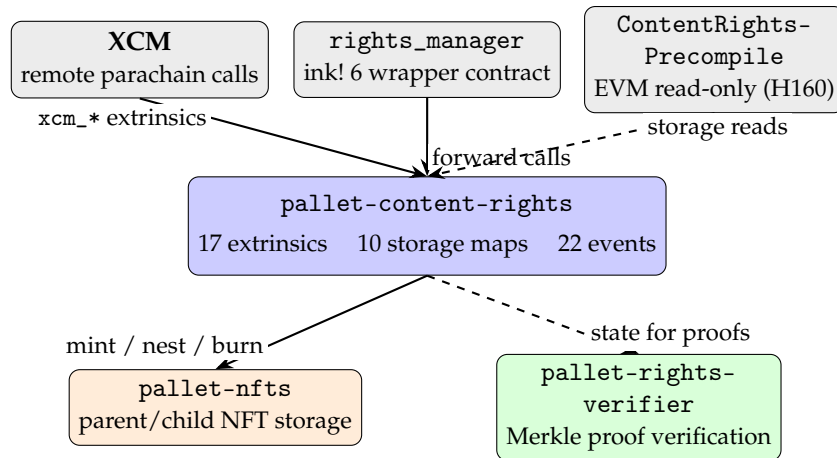


Figure 5.1. CCRMS implementation-level component diagram. Solid arrows indicate active extrinsic dispatch; dashed arrows indicate read-only storage access. `pallet-content-rights` is the sole authoritative state layer; the ink! contract and EVM precompile are stateless wrappers that forward calls or read storage, respectively. XCM delivers cross-chain calls via HRMP. `pallet-rights-verifier` checks Merkle storage proofs of rights state against a caller-supplied state root; binding that root to finalized CCRMS state is not implemented (Section 5.2.1).

or Ownership). The smart contract and precompile layers are stateless wrappers; all authoritative state resides in the pallet (Appendices A.2 and A.9).

5.2 | Cross-Chain Integration

The core challenge in cross-chain integration is enabling remote parachains to acquire and verify content rights without a shared trust anchor or custodian. Two sub-problems follow: (1) how to dispatch authenticated, fee-paying rights operations across HRMP as a single XCM message; and (2) how to extend that reach to Ethereum without custodial bridges. The key lesson from both is that the CCRMS runtime’s XCM configuration and payer-beneficiary decoupling are sufficient; no custom XCM extensions were required.

5.2.1 | XCM Message Flows

Cross-chain operations use XCM to enable remote parachains to acquire content rights. The runtime’s XCM executor implements XCM v5, the current version in the SDK used (the configuration imports `xcm::latest` and advertises `CurrentXcmVersion`). The benchmark sends a V3-encoded message, which `pallet-xcm` converts to the current version before delivery (Appendix C.2). The three instructions it uses, `WithdrawAsset`,

BuyExecution and Transact, exist in every version since v3, so the sufficiency result concerns this standard instruction subset rather than an older protocol version. The pallet exposes five XCM extrinsics: `xcm_subscribe`, `xcm_renew_subscription`, `xcm_purchase_views`, `xcm_purchase_ownership`, and `xcm_transfer_ownership` (callable locally only after Finding B remediation; see Appendix D). Each separates the dispatch origin (**payer**) from the **beneficiary** of the rights record: when a remote parachain sends a Transact message, the origin is the remote parachain's sovereign account, but the rights record is issued to a distinct beneficiary account representing the end user on the remote chain.

The runtime's XCM configuration follows the standard Cumulus pattern, with one customization: the `LocationToAccountId` chain adds the `HashedDescription` converter (with `DescribeFamily<DescribeAllTerminal>>`) and the `GlobalConsensusParachainConvertsFor` converter, so the runtime can interpret Ethereum locations of the form `GlobalConsensus(Ethereum) → AccountKey20(0x...)`. This is required for Snowbridge (Section 5.2.2); the full Ethereum-to-CCRMS flow is configured but has not been end-to-end tested.

Each XCM rights operation uses the three-instruction sequence `WithdrawAsset`, `BuyExecution`, and `Transact`. This is sufficient because rights operations are stateful side effects, not asset transfers (Appendix C shows the full message structure). The endianness and fee-sizing findings (A.5, A.6 in Table 5.1) are the two most operationally significant surprises in the XCM integration.

All 30 test messages (10 per operation) were delivered. The median destination-chain delay was five CCRMS blocks (about 30 s; range 18–42 s), measured from on-chain timestamps of finalized blocks (Section 7.1.3, Table 7.2).

A secondary pallet, `pallet-rights-verifier`, checks Merkle storage proofs of CCRMS rights state. Three extrinsics (`verify_ownership`, `verify_subscription`, `verify_view_pack`) take a state root and a proof from the caller, construct the storage key, read the value from the proof, and emit a result event. The check is only as trustworthy as the state root, and the pallet accepts whatever root the caller supplies: anyone can build a trie containing a chosen value and submit its root. A consuming chain would first need a finalized CCRMS state root from a trusted source, for example the CCRMS head data in the relay-chain state, which a parachain can read through its relay-chain state proof. We did not implement that binding. The pallet is exercised only by seven unit tests that build their own tries, not across two chains, and `verify_subscription` reports whether a subscription entry exists, not whether it has expired; the caller must compare the expiry block with the CCRMS block height. Proof validation costs roughly 100 million `ref_time` per call.

5.2.2 | Snowbridge Integration with Ethereum

We extended the cross-chain architecture to Ethereum via Snowbridge, a trust-minimized bridge that uses light-client verification on both sides, eliminating custodians and relay trust. During Phase 4 we bridged two 1-ETH transactions from a local Ethereum chain to a local AssetHub parachain using the Snowbridge pipeline, demonstrating that the CCRMS runtime can accept bridged Ethereum messages with the same XCM configuration as sibling-parachain messages (via `HashedDescription` and `GlobalConsensusParachainConvertsFor` in `LocationToAccountId`).

The topology spans four parachains and a local Ethereum network: CCRMS (parachain 100); Bridge Hub (parachain 1013), which hosts the inbound queue and the Ethereum beacon light client; AssetHub (parachain 1000), which holds the foreign-asset reserve; and the Rococo relay chain. Four HRMP channels link Bridge Hub ↔ AssetHub and AssetHub ↔ CCRMS, carrying bridge messages and enabling future Polkadot–Ethereum interoperability.

The Ethereum side runs Geth v1.17.1 (chain ID 11155111) as the execution layer, Lodestar v1.35.0 as the beacon node, and the 16-contract Snowbridge Gateway suite was deployed via Forge (a Solidity contract deployment tool from the Foundry framework). After initializing the beacon light client, two relay processes run: a beacon relay that forwards Lodestar finality events to the `EthereumBeaconClient` pallet, and an execution relay that forwards Gateway events from Geth to the `EthereumInboundQueue` pallet.

The integration revealed three persistent issues, documented in Appendix A: a Lodestar version-pinning requirement that forces source compilation; a configuration bug in the fork version that produced invalid Merkle proofs; and a beacon state cache timing issue that required reordering the setup sequence.

The final step, using bridged Ether on AssetHub to pay for a CCRMS rights operation, is conceptually straightforward but was not implemented in the prototype. The CCRMS XCM side is already prepared; the remaining task is to configure `pallet-content-rights::PaymentCurrency` to accept both foreign Ether and the native token. This is listed as future work in Chapter 9.

5.3 | Challenges and Solutions

The CCRMS implementation encountered nine technical issues that shaped the final architecture and security posture. Table 5.1 summarizes each issue, our response, and the key lesson; Appendix A provides full details.

Table 5.1. Implementation Challenges Summary

#	Challenge	Response	Key Lesson
A.1	RMRK 2.0 pallets frozen on SDK 0.9.36, incompatible with current monorepo	Replicated RMRK nesting semantics on pallet-nfts directly	Standards reimplementation is preferable when reference implementation is unmaintained
A.2	Contract-first design blocked: chain extensions non-functional for writes, debugging complexity	Inverted to pallet-first; retained ink! contract as thin wrapper	Pallet-first proved resilient to ink! discontinuation (A.9)
A.3	Snowbridge beacon relay generated proofs at wrong gindex (54 vs. 86)	forkVersions field means activation epoch, not version hex; changed to plain epoch numbers (Appendix A.3)	Cryptographic systems fail silently on configuration errors
A.4	Lodestar v1.41.0 ignored LODESTAR_PRESET=mainnet in dev mode (32 vs. 512 validators)	Pinned Lodestar v1.35.0, built from source	Light clients impose strict counterpart requirements; bridge toolchain compatibility is fragile
A.5	XCM tests: funds debited on source chain but InsufficientPayment on CCRMS	polkadot.js toHex() used big-endian; CCRMS expects little-endian; fixed with toHex(true)	Endianness bugs produce valid-looking but wrong addresses

Continued on next page

Table 5.1 continued from previous page

#	Challenge	Response	Key Lesson
A.6	XCM BuyExecution failed at 1B tokens; required $\approx 75\text{B}$ (100B used)	Placeholder XCM weights (64 KiB proof size per instruction) make proof size the fee-setting dimension under BlockRatioFee	Fees follow the dominant weight dimension; unbenchmarked weights distort them
A.7	xcm_transfer_ownership had no check that caller == from (High severity)	Added ensure! (authorizer == from); unit test confirms block; cross-chain transfer disabled pending E.8	Cross-chain extrinsics accepting identity parameters need explicit authorization
A.8	MaxChildrenReached at 50 subscribers per content item	Documented as known limitation; benchmark scripts rotated content items; cap removed in 2865ed0	Bounded FRAME collections trade benchmarkability for capacity limits
A.9	ink! development discontinued Jan 2026	95% of logic in pallet; contract layer is non-load-bearing; migration deferred	Minimize dependency on non-essential components for ecosystem resilience

5.4 | Summary

Cross-chain rights operations are delivered via standard XCM instructions with payer–beneficiary decoupling, and the Snowbridge integration configured the path for Ethereum bridging, though the Ethereum-to-CCRMS flow has not been end-to-end tested. Nine challenges shaped the final architecture; the two most consequential (the RMRK-to-pallet-nfts pivot and the contract-first-to-pallet-first pivot) are examined in greater detail in Chapter 8. Full implementation details are provided in Appendix C.

Evaluation and Testing

This chapter outlines the evaluation methodology used to assess CCRMS against the research questions in Chapter 3. It focuses on **how** the system was evaluated; the results are presented in Chapter 7. This chapter covers the evaluation approach, the four-layer testing architecture, the eight test scenarios, the 11 KPIs, the testing infrastructure, and the validity considerations that inform our interpretation of the findings.

6.1 | Evaluation Approach

Our evaluation methodology spans unit testing, integration testing, performance benchmarking, security assessment, comparative analysis against a centralized baseline, and Monte Carlo simulation, covering the technical, economic, and security domains of the research questions. Table 6.1 maps each evaluation activity to the specific research sub-questions it addresses.

Table 6.1. Mapping of Evaluation Activities to Research Sub-Questions

Research Question	Primary evaluation activity	Supporting activities
SQ1 (XCM cross-chain rights operations and finality)	XCM latency benchmark, end-to-end XCM test	Unit tests for xcm_* extrinsics
SQ2 (Latency vs decentralization trade-offs)	Throughput and latency benchmark, HHI calculation	Comparative analysis vs centralized

Continued on next page

Table 6.1 continued from previous page

Research Question	Primary evaluation activity	Supporting activities
SQ3 (Unified rights token with royalty distribution)	Unit tests, royalty propagation tests, comparative benchmark	Monte Carlo creator revenue simulation
SQ4 (Security and authorization)	Security audit, authorization unit tests	Reliability test
SQ5 (Economic viability)	Monte Carlo creator revenue simulation	Literature comparison

Table 6.2 classifies the same activities using the Framework for Evaluation in Design Science (FEDS) (Venable et al., 2016) and the evaluation methods of Hevner et al. (2004). Every evaluation episode is artificial: it runs on a local testnet, in a simulator, or in a model, with no real creators or consumers. This places the evaluation in the FEDS Technical Risk and Efficacy strategy, which suits an artifact whose main design risks are technical and for which evaluation with real users and production systems would be prohibitively expensive. Formative episodes, chiefly the security audit and the tests that guided the design iterations, shaped the artifact; summative episodes report on it as built, with the code versions given in Section 6.7. Because no evaluation was naturalistic, the results say nothing about adoption or user behavior.

Table 6.2. Evaluation Activities Classified by DSR Evaluation Type

Activity	Question	FEDS purpose	Paradigm	Method (Hevner et al., 2004)
Pallet unit tests	SQ3, SQ4	Formative and summative	Artificial	Testing (structural)
xcm-simulator integration tests	SQ1, SQ4	Formative and summative	Artificial	Testing (functional)

Continued on next page

Table 6.2 continued from previous page

Activity	Question	FEDS purpose	Paradigm	Method (Hevner et al., 2004)
Throughput, block weight, storage, and stress (Scenarios 1–3, 5)	SQ2	Summative	Artificial	Experimental (controlled experiment); Analytical (dynamic analysis)
XCM latency on Zombienet (Scenario 4)	SQ1	Summative	Artificial	Experimental (controlled experiment)
Security audit (Scenario 6)	SQ4	Formative	Artificial	Analytical (static analysis); Testing
Reliability under collator failure (Scenario 7)	SQ4	Summative	Artificial	Experimental (controlled experiment)
Centralized baseline comparison (Scenario 8)	SQ2	Summative	Artificial	Experimental (controlled experiment)
HHI calculation	SQ2	Summative	Artificial	Analytical (architecture analysis)
Monte Carlo creator revenue simulation	SQ5	Summative	Artificial	Experimental (simulation)
Literature and gap analysis (Chapter 2)	All	Formative	Artificial	Descriptive (informed argument)

6.2 | Testing Layers

We organize testing into four layers, each executed at the lowest applicable level of abstraction. Figure 6.1 summarizes the stack; each layer is described below.

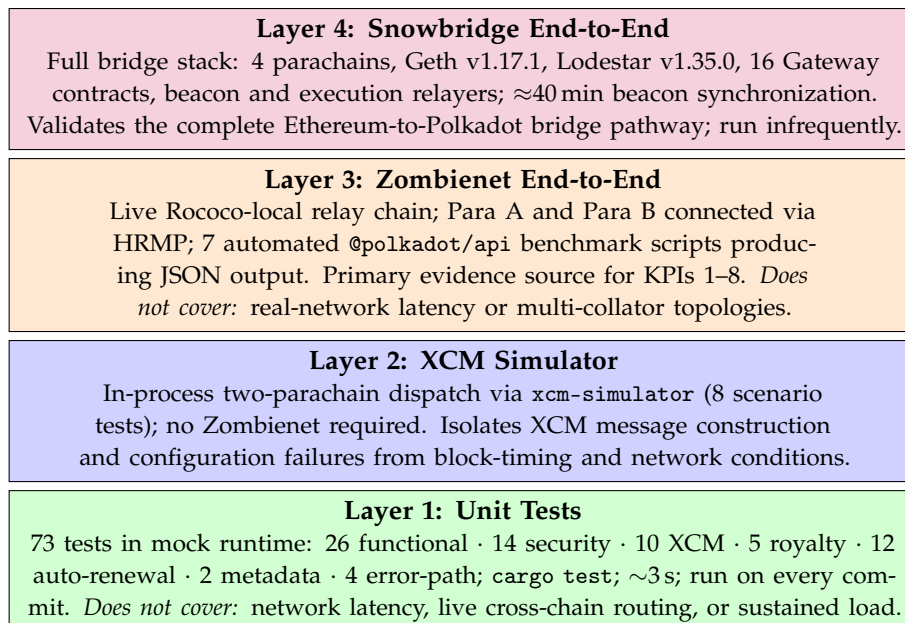


Figure 6.1. Four-layer testing architecture (Layer 1 at base: fastest and most granular, run on every commit; Layer 4 at apex: broadest scope, run infrequently). Each layer tests what lower layers cannot: unit tests isolate individual extrinsic logic; the XCM simulator isolates message-construction failures; Zombienet provides definitive wall-clock performance evidence; Snowbridge validates the complete Ethereum bridge pathway.

6.3 | Performance Testing Framework

Seven automated scripts in `scripts/perf/` cover various aspects of system behavior and each produce machine-readable JSON output. Table 6.3 summarizes the benchmark categories.

Table 6.3. Benchmark Scripts

Category	Script	What it measures
Throughput	<code>local-throughput.mjs</code>	TPS across 11 extrinsic types at batch sizes 1, 10, 20, 50, 100
Stress	<code>stress-test.mjs</code>	Sustained TPS from 200 concurrent accounts over 180 seconds
XCM latency	<code>xcm-latency.mjs</code>	Block delta between send (Para B) and event arrival (Para A)

Continued on next page

Table 6.3 continued from previous page

Category	Script	What it measures
Block weight	<code>block-utilization.mjs</code>	Percentage of <code>ref_time</code> and <code>proof_size</code> consumed at batch sizes 50–300
Storage	<code>storage-growth.mjs</code>	SCALE-encoded byte size per content item, subscriber, and view pack
Resource monitoring	<code>resource-monitor.mjs</code>	CPU, memory, and disk usage via Prometheus scraping during load
Reliability	<code>reliability-test.mjs</code>	Net chain growth, recovery time after the collator is killed (SIGKILL; SIGTERM for comparison), state persistence
Comparative	<code>centralized-drm-benchmark/</code>	Same six operations on Express.js + SQL.js under identical conditions

A separate Monte Carlo simulation (`monte-carlo-revenue.mjs`) models 1,000 creators over 10,000 iterations across four platform configurations (centralized 30–45% fees, Web3 marketplace 0–2.5%, CCRMS 1–5%, and an illustrative bridge-based configuration with an assumed 8–15%, reported only in Appendix G), incorporating algorithmic discovery and monthly churn. This simulation provides the primary evidence for research question SQ5. We document the methodology in `scripts/perf/results/monte-carlo-findings.md`.

All scripts are reproducible: each is an independent Node.js program with explicit parameters and deterministic output paths, and each is archived as timestamped JSON in `scripts/perf/results/`.

6.4 | Test Scenarios

We designed eight test scenarios to cover the full range of evaluation activities. Table 6.4 summarizes each scenario, and the corresponding scripts are located in `scripts/perf/`.

Table 6.4. Test Scenarios

#	Scenario	Description	Target
1	Throughput and latency	Submit concurrent batches (1, 10, 20, 50, 100) for each of the 11 local extrinsics; measure submission-to-inclusion time.	~6 s inclusion latency
2	Block weight saturation	Submit progressively larger <code>register_content</code> batches (50–300); query <code>system.blockWeight</code> at inclusion.	Block weight <75% of budget at max batch
3	Storage growth	Register content, subscribers, and view packs in incremental quantities; measure SCALE-encoded byte size per map.	<500 bytes per content item; linear growth
4	XCM latency	Send <code>xcm_subscribe</code> , <code>xcm_purchase_views</code> , <code>xcm_purchase_ownership</code> from Para B to Para A via the canonical <code>WithdrawAsset</code> → <code>BuyExecution</code> → <code>Transact</code> sequence; measure block delta.	<10 parachain blocks per operation
5	Stress test	200 funded accounts continuously submit <code>register_content</code> for 180 seconds.	>10 TPS sustained (KPI 1)
6	Security audit	Static analysis (cargo <code>clippy</code> , cargo <code>audit</code>), manual review of all 17 extrinsics, targeted unit tests for authorization gaps and edge cases.	No Critical/High findings unremediated at submission (KPI 9)
7	Reliability	Monitor block production for 120 s (baseline); kill the collator (<code>SIGKILL</code> , three runs; <code>SIGTERM</code> , one run), restart it and verify state; monitor for 120 s (recovery).	Recovery <60 s; 100% state persistence

Continued on next page

Table 6.4 continued from previous page

#	Scenario	Description	Target
8	Comparative analysis	Re-implement the same six operations in Express.js + SQL.js (in-memory SQLite); run identical benchmarks against both.	Document the performance cost of decentralization

The comparative benchmark is intentionally minimal, using Express.js for HTTP and SQL.js for in-process SQLite. A production centralized service adds network hops, replication and caching. Its per-request latency would be higher than this in-process baseline, while its aggregate throughput could be much higher through horizontal scaling. The baseline is therefore a best case for centralized latency and a conservative figure for centralized throughput; in both cases the gap to CCRMS is large enough that the conclusions do not depend on the exact values.

6.5 | Metrics and Key Performance Indicators

Our evaluation uses 11 key performance indicators (KPIs) across four domains: technical, economic, security, and reliability. Each KPI includes a description, measurement methodology, originating source (the test scenario or activity that generates it), and a target value. These targets were set as stated design objectives during Phase 2 planning, informed by commercial comparators rather than by published academic benchmarks; they are treated throughout as reference points rather than strict pass/fail criteria. The inclusion latency target of two block intervals reflects the 6-second parachain block time of the test environment; the chain-growth target of 85% is sized for a single-collator Zombienet topology.

Table 6.5. KPI Definitions

#	KPI	Definition	Target	Source
1	Throughput (TPS)	Successful transactions per second under sustained load	>10 TPS	Stress test

Continued on next page

Table 6.5 continued from previous page

#	KPI	Definition	Target	Source
2	Inclusion latency	Time from submission to block inclusion	<12 seconds (2 blocks)	Throughput test
3	XCM latency	Block delta for cross-chain operations	<10 blocks	XCM latency test
4	Block utilization	Percentage of block weight consumed at capacity	<75%	Block saturation test
5	Storage efficiency	Bytes per content item and per subscriber	<500 bytes	Storage growth test
6	Net chain growth	Blocks added to the canonical chain as a percentage of 6 s slots	>85%	Reliability test
7	MTTR	Time from collator kill to the first block that remains in the canonical chain	<60 s	Reliability test
8	State persistence	Data integrity after kill and restart	100%	Reliability test
9	Security findings	Critical/High vulnerabilities in dissertation code	No Critical/High findings unremediated at submission (originally 0 Critical, 0 High; Section 7.4)	Security audit
10	Test coverage	Error variants exercised by unit tests	>80%	Test suite
11	Decentralization ratio	Performance cost vs centralized baseline	Documented	Comparative analysis

The target value and measurement source for each KPI are defined in Table 6.5. Measured values and outcomes for all 11 KPIs are reported in Chapter 7 (Table 7.15). The security audit and a subsequent code review produced eleven findings (Table 7.13, Appendix D): five vulnerabilities remediated (B, E, I, J, K), three accepted design properties (A, C, D), and three reliability observations from the kill-restart test (F, G, H). Chapter 7 presents the full KPI outcomes and discusses each in relation to the research questions; Chapter 8 interprets the findings.

6.6 | Testing Infrastructure

Table 6.6. Test Environment Specifications

Component	Specification
Machine	macOS Tahoe 26.5.2, Apple Silicon
Relay chain	Rococo-local, 2 validators (alice, bob)
Parachain A	Content Rights (parachain 100), 1 collator
Parachain B	Consumer chain (parachain 200), 1 collator
Bridge Hub	Parachain 1013 (Snowbridge tests only)
AssetHub	Parachain 1000 (Snowbridge tests only)
Ethereum	Geth v1.17.1 + Lodestar v1.35.0 (Snowbridge tests only)
Zombienet provider	Native
Block time	~6 seconds (parachain)
Node binary	parachain-template-node (release build)
Polkadot SDK version	polkadot-sdk master, commit a20ac9bd (10 February 2026), for the parachain runtime and node and for the relay chain (polkadot 1.21.1)
Zombienet	v1.3.138 (local build, commit bc7877e)
Test scripts	scripts/perf/*.mjs (Node.js, @polkadot/api library)
Centralized benchmark	centralized-drm-benchmark/ (Express.js + SQL.js)

The single-machine topology simplifies setup and eliminates reliance on external infrastructure, with implications for interpreting the results discussed in Section 6.7. Benchmark scripts and raw JSON outputs are version-controlled, allowing any reader with access to the Polkadot SDK toolchain to independently re-execute them.

6.7 | Validity and Limitations

Internal validity. We kept the test environment constant across runs, applied the same methodology to CCRMS and the centralized baseline, and ran the deterministic unit-test suite on every commit. The XCM latency benchmark was run ten times per operation and is reported as medians; the throughput, block-utilization, storage-growth and stress benchmarks were each run once, so their figures carry no measure of run-to-run variation.

Not every benchmark was re-run after every fix. The throughput, storage-growth and stress measurements (31 March–1 April 2026) predate the remediation of Findings B and E (2 April) and of Findings I, J and K (September 2026), and the XCM latency re-run (24 September) predates commit a20ad8c. The fixes add checks and change collection settings but do not change the per-item storage layout (Table 7.4); we expect their effect on the reported figures to be small but did not re-measure it. We redesigned and re-ran the reliability test on 28 September 2026 using the final pallet code (commit d88ec4e); it is the only measurement taken after all fixes.

External validity. This is the dimension in which our evaluation is most limited. Four constraints apply: (1) we measure in a local Zombienet environment with two relay-chain validators and no real network latency, so the figures serve as upper bounds within the test environment rather than predictive metrics for production; (2) the Snowbridge integration uses a local Ethereum stack rather than a public testnet, avoiding variability in block times, MEV, contention, and beacon chain reorganizations; (3) the figures come from a local testnet with one core per parachain; elastic scaling, which could increase throughput, was not tested; (4) our scale is modest, comprising up to 200 concurrent accounts and a few hundred content items, and empirical verification of linear storage growth at production scale has not yet been conducted.

Construct validity. The figure of 26.2 TPS is the inclusion rate our closed-loop client could sustain, a lower bound on capacity rather than a capacity measurement (Section 7.1.2), and it is insufficient to assess user-perceived performance, which also depends on wallet user experience, mempool latency, and RPC responsiveness. The 90% net chain growth describes a single collator on one machine and says nothing about availability with several collators. The Monte Carlo simulation evaluates expected revenue using a probabilistic model; the input parameter sources and assumptions are documented in Appendix G.

Threats to validity. (a) **Selection bias:** the eight test scenarios may not capture production conditions such as high subscriber concurrency on a single content item, large multi-collator topologies, or extended multi-month uptime. (b) **Measurement instrument bias:** any bug in `@polkadot/api` or `system.blockWeight` would propagate

into our results; although the library is widely used, it is not bug-free. (c) **Confirmation bias:** the author was responsible for designing, implementing, and evaluating the system; this risk is mitigated by using externally referenced KPI targets and by disclosing all scripts and raw results. (d) **Single-author validation:** no external code review, third-party security audit, or benchmark validation has been conducted; we recognize this as a significant limitation and intend to address it in future work (Appendix E, E.1).

6.8 | Summary

The evaluation uses four testing layers, eight test scenarios, seven automated benchmarks, a Monte Carlo simulation, and 11 KPIs grounded in a measurement-focused philosophy. Section 6.7 outlines the primary validity constraints. Chapter 7 presents the results.

Results

This chapter presents the measured results. Section 7.1 covers technical results, Section 7.2 covers economic results, and Section 7.3 covers security and user-centric results. We defer interpretation to Chapter 8.

7.1 | Technical Results

The technical results address SQ1 (Section 7.1.3), SQ2 (Section 7.1.8), and SQ3 (Sections 7.1.1–7.1.5).

7.1.1 | Throughput

Table 7.1 reports 100 concurrent extrinsics per type against a single collator (6-second blocks). All four non-minting extrinsics achieved 100/100 inclusion in one block (≥ 100 extrinsics per block, ≈ 16.6 TPS). No failure arose from block weight, space, or transaction-pool capacity.

All failures were dispatch errors from the fifty-child bound then in place (removed in 2865ed0; Section 4.2.3): `MaxChildrenReached` directly, or cascading `SubscriptionNotFound/ViewPackNotFound` for dependent extrinsics. The benchmark characterizes storage-design capacity, not parachain throughput. Sustained throughput is in Section 7.1.2.

Per-extrinsic TPS is not reported: elapsed time reflects how many blocks the harness waited before observing inclusion (1–4 blocks), not execution cost. Relative costs are in the FRAME weights (Appendix C; hand-estimated, not benchmarked).

Table 7.1. Single-Chain Inclusion Capacity at Batch Size 100

Extrinsic	Succ.	Failed	Blocks	Failure cause
register_content	100	0	1	None
check_access	100	0	1	None
set_royalty_splits	100	0	1	None
query_rights_metadata	100	0	1	None
subscribe	50	50	1	MaxChildrenReached
purchase_ownership	50	50	1	MaxChildrenReached
purchase_views	0	100	0	MaxChildrenReached
renew_subscription	50	50	1	SubscriptionNotFound [†]
enable_auto_renew	50	50	1	SubscriptionNotFound [†]
disable_auto_renew	50	50	1	SubscriptionNotFound [†]
consume_view	22	78	1	ViewPackNotFound [†]

[†] Subscription, ownership and auto-renewal batches each used a freshly registered content item, so their 100-call batches exceeded the fifty-child bound on that item: 50 calls succeeded and 50 failed (renewal and auto-renewal calls failed for the 50 accounts whose subscriptions could not be created). Pay-per-view batches reused one item; this run predates the Finding E fix, so each repeat purchase minted a new child, and the item reached the bound during the 50-call batch. `consume_view` then succeeded only for the 22 accounts holding view packs. The bound was later removed (commit 2865ed0).

7.1.2 | Sustained Throughput (Stress Test)

Under sustained load (200 accounts, 180 s), the harness recorded 4,713 included `register_content` extrinsics in 27 blocks: 26.2 TPS, or 174.6 per block. This is the rate the benchmark client offered, not the chain's capacity. The client is closed-loop: it submits one transaction per account (200 in total), waits until all are included, and only then submits the next batch, so it cannot offer more than about 200 transactions per block. The chain was not saturated: 300 of the same extrinsics used 12.9% of declared `ref_time`, 2.2% of measured proof size and about 62 ms of a 6 s block (Section 7.1.4). The figure is therefore a lower bound on single-collator capacity, which we did not measure. The harness counters are also approximate: at several sampling points, recorded successes plus failures exceed submissions, so some transactions were counted more than once. From about 142 s the client exhausted nonces and issued ~17,000 failed submissions

in six seconds, a client limit rather than chain capacity. Only `register_content` was measured, and the run predates the fixes for Findings B, E, I, J and K (Section 6.7).

7.1.3 | Latency

Single-chain inclusion latency, measured from submission to inclusion in a block, was $\sim 6,000$ ms (one parachain block) for 42 of the 53 successful batches in the throughput benchmark; within each batch, all transactions were included in the same block (spread < 15 ms). Nine batches took longer: seven ~ 18 s, one ~ 24 s, and one ~ 12 s. Two subscribe batches took ~ 264 s and ~ 606 s (about 44 and 101 block intervals). The ~ 264 s batch is the 100-call subscribe batch in which 50 calls failed with `MaxChildrenReached` (Table 7.1). For the ~ 606 s batch, the node logs needed to determine whether block production paused or whether the transactions waited in the pool were not retained, so it is reported as an unexplained outlier. The median batch latency was ~ 6.0 s.

In the XCM benchmark, Para B sent each message with `sudo(polkadotXcm.send)`, so every message originated from Para B itself and CCRMS charged execution fees to Para B's sovereign account. Para B runs the CCRMS binary, so an ordinary user there can also send a signed `polkadotXcm.send`; CCRMS then charges fees to, and dispatches under, that user's `HashedDescription`-derived account (Appendix D.1). The latency benchmark did not exercise this path.

XCM results are in Table 7.2.

Table 7.2. XCM Cross-Chain Delivery ($n = 10$ per operation; finalized blocks)

Operation	Delivered	CCRMS blocks, median (range)	Latency (s), median (mean $\pm$ s.d.)
<code>xcm_subscribe</code>	10/10	5 (3–6)	30 (28.8 $\pm$ 6.2)
<code>xcm_purchase_views</code>	10/10	5 (3–7)	30 (30.6 $\pm$ 8.7)
<code>xcm_purchase_ownership</code>	10/10	5 (3–6)	30 (28.8 $\pm$ 6.2)
All	30/30	5 (3–7)	30 (29.4 $\pm$ 6.9); range 18–42

Blocks are counted from the CCRMS head observed when the send was confirmed on Para B; seconds are measured from the timestamp of the Para B block that included the send. Because the start points differ, blocks $\times$ 6 s need not equal the latency (a run can show 24 s with 6 blocks).

Each run sent an `xcm_*` extrinsic from Para B with a fresh beneficiary, and the harness read finalized CCRMS blocks until that beneficiary's `CrossChain*` event appeared. All

30 messages were delivered. Latency is measured on-chain, from the timestamp of the Para B block that included the send to that of the CCRMS block containing the event, at 6 s resolution. The median was 30 s (5 CCRMS blocks), consistent across operations; client-observed time was longer (median 68 s) because the harness waits for finality. An earlier harness ($n = 3$) matched the first pallet event in a fixed 15-block window and could match unrelated events; a first run of the corrected harness read unfinalized blocks and misreported delivered messages. The figures reported here read finalized blocks only (xcm-latency.mjs v2.2). Asynchronous backing was already active (Section 8.2.1), so these figures include its effect; we did not break the delay down by stage.

7.1.4 | Block Weight Utilization

Table 7.3. Block Weight Utilization at Increasing Batch Sizes

Batch Size	ref_time Consumed	proof_size Consumed	Block Construction Time
50	2.2%	0.4%	~10 ms
100	4.3%	0.7%	~21 ms
150	6.5%	1.1%	~31 ms
200	8.6%	1.5%	~42 ms
250	10.7%	1.8%	~52 ms
300	12.9%	2.2%	~62 ms

At 300 transactions, block weight accounts for 12.9% of ref_time and 2.2% of proof_size, with a construction time of approximately 62 ms (~1% of the 6-second block). The declared proof size is zero in every hand-estimated weight (Appendix C), but the runtime's StorageWeightReclaim extension measures the proof size each transaction actually consumes and records that figure in the block weight, so the 2.2% proof_size figure is measured. The ref_time figure, by contrast, is the sum of declared weights, not measured execution time.

7.1.5 | Storage Growth

Table 7.4. Per-Item Storage Costs (SCALE-encoded)

Item Type	Storage Map	Bytes per Item
Content registration	Contents	191
Active subscription	Subscriptions	112
View pack	ViewPacks	111
Ownership record	Ownership	36
Royalty splits (per collaborator)	RoyaltySplits	34
Child NFT nesting record	Children (removed in 2865ed0; Parent also stores 8 bytes per child)	8 (per child)

Growth was linear: the 50th item costs the same number of bytes as the first, so the cost per record does not depend on how many records exist. This measures storage cost, not lookup time. At one million content items and ten million subscribers, the CCRMS values alone come to about 1.3 GB, within the 500-byte-per-record Phase 2 target. This is a lower bound on state size: each child NFT also creates `pallet-nfts` entries (the item, the owner’s account index and the `rights_type` attribute), and every entry adds a storage key and trie nodes. None of these were measured.

7.1.6 | Reliability

The reliability test (`scripts/perf/reliability-test.mjs`, commit `d88ec4e`) killed the collator with `SIGKILL`, which gives the node no chance to shut down cleanly, restarted it with the same command line, and repeated this three times; one further run used `SIGTERM` for comparison. Each run monitored the chain for 120 s before the kill and for 120 s after recovery. Table 7.5 reports medians and ranges.

After a `SIGKILL`, the node answered RPC requests within 3.8 s and authored a block within 6 s, but that block and the next one or two were later discarded: the first block to become part of the canonical chain was authored 17.9 s after the kill in all three runs, and the first post-kill block was finalized after about 47 s. After a `SIGTERM`, the first block authored (5.9 s) was kept, and finality followed at 40 s. Both are within the 60 s target (KPI 7). In every run, the last block finalized before the kill remained canonical, the test content and subscription records were byte-for-byte unchanged, and a new transaction was included after restart (KPI 8).

Table 7.5. Reliability Test Results (median and range over three SIGKILL runs; one SIGTERM run for comparison)

Phase	Metric	SIGKILL (3 runs)	SIGTERM (1 run)
Baseline	Net chain growth	18 of 20 blocks (90%)	18 of 20 (90%)
Baseline	Median block interval	6.0 s	6.0 s
Failure	Process exit after signal	66 ms (63–67)	505 ms
Failure	RPC ready	3.8 s (3.78–3.82)	4.8 s
Failure	First block authored (any branch)	6.0 s	5.9 s
Failure	First block kept in canonical chain	17.9 s (all runs)	5.9 s
Failure	First post-kill block finalized	47.0 s (44.7–47.9)	40.3 s
Failure	Post-kill blocks discarded	6 (5–7)	4
Recovery	State checks passed	3 of 3 runs	1 of 1
Recovery	Net chain growth	90% (85–90%)	90%

Net chain growth was 18 of 20 expected blocks (90%) before and after the kill in every run except one (17 of 20 after the kill in the second SIGKILL run). The median interval between consecutive blocks was 6.0 s; the shortfall comes from blocks that the collator authored and then discarded and re-authored, which happened throughout the test, not only after the kill (Finding F). The 90% figure therefore measures net chain growth on this single-collator testnet, not time during which the collator was unavailable.

Two practical points follow. Blocks that are not yet finalized can be discarded after a hard failure, so an application should treat a purchase as settled only at finality, about 35–40 s after a block is authored in this environment, not at inclusion. And these figures come from a single collator on one machine; with several collators, block production could continue while one restarts, but this was not tested.

7.1.7 | Snowbridge End-to-End Bridge

We bridged two 1 ETH transactions from the local Ethereum network to AssetHub via Snowbridge (four parachains, Geth, Lodestar, 16 Gateway contracts, two relayers). Both succeeded; the destination account held 22 ETH on AssetHub (20 pre-minted, 2 bridged). Table 7.6 provides the message pathway.

Table 7.6. Snowbridge End-to-End Message Path (ETH Transfers to AssetHub; CCRMS Not Exercised)

Step	Component	Action
1	Geth	Transaction Gateway.sendToken(1 ETH, Alice) included in block 5157
2	Lodestar	Block finalized at epoch ≥ 162
3	Beacon Relay	Generated inclusion proof, submitted beacon header update
4	Bridge Hub	ethereumBeaconClient.submit(...) verified BLS signatures, advanced latestFinalizedBlockRoot
5	Execution Relay	Generated receipt + execution + ancestry proof, submitted message bundle
6	Bridge Hub	ethereumInboundQueue.submit(...) verified the proof against the verified beacon state root
7	Bridge Hub → AssetHub	XCM message dispatched via HRMP
8	AssetHub	foreignAssets.mint(Ether, Alice, 1 ETH) succeeded

CCRMS was not on the path; the result confirms the local pipeline and the CCRMS runtime's location-converter configuration. The final hop to a CCRMS operation was not tested end-to-end (Section 8.5); integration challenges are in Appendix A.

7.1.8 | Comparative Analysis vs. Centralized Baseline

We benchmarked CCRMS against Express.js with SQL.js for the same six operations, using the same methodology. Table 7.7 presents the comparison.

Table 7.7. Comparative Analysis: CCRMS Parachain vs. Centralized (Express.js + SQL.js)

Metric	CCRMS Parachain	Centralized	Ratio
Sustained TPS	26.2 (harness-limited lower bound)	7,305	$\leq 270\times$
Operation latency (server-side)	$\sim 6,000$ ms (median)	0.09 ms	$67,000\times$

Continued on next page

Table 7.7 continued from previous page

Metric	CCRMS Parachain	Centralized	Ratio
Stress test success rate	22% (RPC nonce conflicts)	100%	–
Storage per content	191 bytes	~100–150 bytes	1.5×
MTTR	~18 s (first block kept)	2–3 s*	~7×
Cross-chain equivalent	~30 s (median)	5–50 ms (HTTP request)	600–6,000×

* Assumed restart time for a single Node.js process; not measured.

The 22% stress-test figure in Table 7.7 is the client-visible success rate over the full saturation window, including the nonce-exhaustion burst described in Section 7.1.2. It is not a consensus failure rate.

The centralized implementation outperforms CCRMS across all quantitative metrics: throughput by up to 270× (the CCRMS figure is harness-limited, so the true ratio is lower) and latency by 67,000×. CCRMS’s qualitative properties (no single point of failure, cross-chain support, provable rights state, and censorship resistance) are not reflected in quantitative data but are central to the use case (gaps G1 and G7, Chapter 2). Even at the harness-limited 26.2 TPS (≈ 2.3 million operations per day), CCRMS would suffice for creator platforms where rights transactions are low-frequency events.

7.2 | Economic Results

The economic results address SQ5 using Monte Carlo simulation, the audience-size crossover finding, and a comparative fee analysis.

7.2.1 | Monte Carlo Creator Revenue Simulation

The simulation models 1,000 creators across 10,000 iterations under four platform configurations, of which this chapter reports three. The fourth, a bridge-based configuration, uses an assumed fee range (8–15%) with no published source, so it is reported only in Appendix G as an illustrative comparator. The simulation projects one-year revenue using platform-specific fees, algorithmic discovery, and monthly churn. It uses a seeded pseudo-random number generator (xoshiro128**, seed 42), so every figure is reproducible

to every reported digit (Node.js 22 or later). Full parameters are in Appendix G; sensitivity analysis in Section 7.2.4.

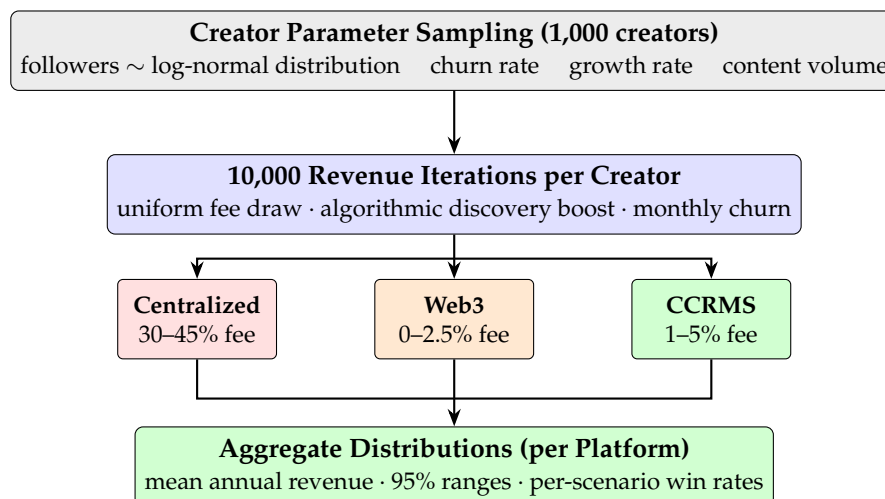


Figure 7.1. Monte Carlo simulation structure: 1,000 creators $\times$ 10,000 iterations = 10 million revenue paths per platform. Platform fee drawn uniformly each iteration; full parameters in Appendix G.

Table 7.8. Monte Carlo Creator Revenue Simulation (Annual Revenue Per Creator: Mean and Median)

Platform Model	Fee Range	Mean Revenue	Median Revenue	Mean vs. CCRMS
Centralized (YouTube/Spotify-style)	30–45%	\$20,546	\$5,903	CCRMS +4.4%
Web3 marketplace (OpenSea-style)	0–2.5%	\$20,545	\$3,268	CCRMS +4.4%
CCRMS	1–5%	\$21,443	\$2,377	Baseline

The means in Table 7.8 are dominated by creators with large audiences. Across all ten million simulated creator-scenarios, median annual revenue is \$5,903 on the centralized platform and \$2,377 under CCRMS: the median creator earns less under CCRMS, because the median simulated audience (about 1,000 followers) lies well below the crossover. CCRMS earns more in 20.6% of scenarios. Its 4.4% higher mean comes from creators with

large audiences, where both revenue and the fee saving are concentrated. The equal 4.4% margin over Web3 marketplaces rests on unsourced churn and discovery assumptions, and its sign changes within their plausible range (Table G.7).

The model credits every platform with a secondary-sale stream, which the prototype does not pay to creators (Appendix E.7). Without that stream, the configuration closest to the prototype as built, the CCRMS mean advantage over the centralized configuration is 0.6% and its win rate 18.1%. An earlier version of the simulation overstated resale activity; Appendix G documents the correction and both configurations.

7.2.2 | Audience Size Dependency

CCRMS's mean advantage (Table 7.8) is not uniform: it is heavily dependent on **audience size**. Table 7.9 breaks results down by tier.

Table 7.9. CCRMS Win Rate Against Centralized Platform by Audience Tier

Tier	Followers	Win rate vs centralized	Notes
Tiny	<100	0.0%	Centralized algorithmic discovery dominates
Small	100–500	0.0%	Fee savings insufficient to offset discovery
Medium	500–2,000	3.5%	Centralized usually preferred
Large	2,000–10,000	37.5%	Crossover; the 50% point falls near 6,000 followers
Huge	10,000–100,000	84.5%	CCRMS usually preferred
Mega	100,000+	93.0%	CCRMS usually preferred

Win rate is the share of simulated scenarios in which a creator earns more under CCRMS than under a centralized platform. Below ~6,000 followers (the 50% point), centralized discovery usually outweighs fee savings; the win rate falls to 8.7% at 1,500–2,000 followers (Table G.6). At 10,000 followers or more, the fee differential usually dominates, and CCRMS wins in 76–92% of scenarios, rising with audience size. The crossover follows from the assumed additive discovery bonus: with no bonus on either platform, CCRMS wins in 97% of scenarios (Table G.5). It therefore characterizes the model, not observed creator behavior.

These results describe the CCRMS design; the prototype no longer caps holders per content item but has not been tested at these audience sizes (Section 8.5).

Table 7.10 reports the per-scenario break-even rates.

Table 7.10. Per-Scenario Break-Even Rates

Comparison	CCRMS Per-Scenario Win Rate
CCRMS vs Centralized	20.6%
CCRMS vs Web3 marketplace	28.6%

Per-scenario win rates are lower than the mean comparison in Table 7.8 suggests, because CCRMS loses in small-audience scenarios.

Table 7.11 reports the central 95% range of simulated total revenue.

Table 7.11. Central 95% Range of Simulated Total Revenue (2.5th–97.5th percentiles over 10,000 iterations; 1,000 creators)

Platform	Total Revenue Range
Centralized	\$13.5M–\$31.9M
Web3 marketplace	\$13.4M–\$33.2M
CCRMS	\$14.0M–\$34.5M

These ranges overlap across all platforms; the per-scenario win rates in Table 7.9 provide the stronger comparison.

7.2.3 | Comparative Fee Analysis

Table 7.12. Published fee ranges by platform category. Centralized literature values span 30–60%; the simulation uses 30–45% (Table G.2).

Category	Fee Range	Representative Platforms
Centralized	30–60%	YouTube (Google LLC, 2025), Spotify (Spotify AB, 2025), Apple Music (Apple Inc., 2025), Audible (Amazon.com, Inc., 2025)

Continued on next page

Table 7.12 continued from previous page

Category	Fee Range	Representative Platforms
Web3 marketplace	0–2.5%	Chua (2023); OpenSea (2.5% (OpenSea, 2025)); Magic Eden (2% (Magic Eden, 2025)); Blur 0.5% proposal (Watson, 2024)
CCRMS	1–5%	Modeling assumption: protocol or treasury allowance, not a measured fee (Appendix G)

7.2.4 | Sensitivity Analysis

A sensitivity analysis (Appendix G) finds the advantage insensitive to content price (expected by construction: price multiplies every platform’s gross revenue equally), moderately sensitive to adoption and conversion rates, and strongly sensitive to the audience distribution, the subscription period, the CCRMS fee, and the centralized discovery bonus. Each of the following eliminates the advantage on its own: a median audience of 500 followers, a 24-month subscription period, a CCRMS fee above about 7%, or a discovery bonus about 25% larger than assumed.

7.2.5 | Tested versus Simulated Audience Sizes

The win-rate results in Tables 7.9 and 7.10 characterize the CCRMS design. They do not characterize the artifact as benchmarked.

The prototype originally capped each content item at fifty holders; commit 2865ed0 removed the cap (Section 8.5, item 4). CCRMS usually wins only in the Huge and Mega tiers (Table 7.9), which imply far more holders per item than any test exercised (at most 60). The prototype can now represent these audiences, but its behavior at that scale is untested.

What the current artifact supports does not depend on audience size: the three rights maps and the priority check_access (Claim 1, Section 1.5.1); the fee assumptions used in the simulation; and the issuance-style XCM on local HRMP (Claim 2). The economic figures remain a design-level result under Appendix G (Claim 3, Section 1.5.1).

7.3 | Security and User-Centric Results

7.3.1 | Security Audit Findings

The security audit (Scenario 6, Section 6.4) produced Findings A–I; later code reviews added J–M. Table 7.13 summarizes all thirteen.

Table 7.13. Security Audit Findings

ID	Severity	Description	Status
A	Low	XCM extrinsics callable locally as gift transactions (intentional)	Accepted design
B	High	Authorization gap in <code>xcm_transfer_ownership</code> : any funded account could transfer any user’s ownership token	Fixed (commit 9ceca57)
C	Low	Permissive XCM filter configuration (<code>SafeCallFilter</code> , <code>XcmExecuteFilter</code> , <code>XcmTeleportFilter</code> , <code>XcmReserveTransferFilter</code> all <code>Everything</code>)	Accepted (prototype); Medium if deployed unchanged
D	Low	Zero-price content registration allowed	Accepted feature
E	Low	View pack overwrite on re-purchase	Fixed (commits 9ceca57, a20ad8c)
F	Informational	After a hard kill, the collator’s first 1–3 post-restart blocks were discarded; first block kept at 17.9 s. Blocks are also re-authored periodically in steady state	Test-environment observation
G	Positive	Finalized chain and state intact after three <code>SIGKILL</code> runs and one <code>SIGTERM</code> run	Confirmed
H	Positive	Net chain growth unchanged after recovery (90% median)	Confirmed
I	Medium	Zero-view denial of service: <code>purchase_views(num_views=0)</code> charged \$0 but minted a child NFT, permanently exhausting a child slot	Fixed (commit a20ad8c)

Continued on next page

Table 7.13 continued from previous page

ID	Severity	Description	Status
J	High	Child-NFT divergence: collections created with default settings allowed child NFTs to be moved via pallet-nfts while Ownership still listed the original account; a subsequent transfer_ownership then burned the buyer's token	Fixed (commit 221e5fa)
K	Medium	Auto-renewal starvation and non-atomic payment: the on_initialize hook read only the first 20 AutoRenewIndex entries per block (FIFO starvation); multi-split payments were not rolled back on partial failure	Fixed (commit 221e5fa)
L	Medium if deployed	Unbenchmarked weights: every extrinsic weight is hand-set with zero proof size, so fees and block admission do not reflect proof-size cost (Appendix D.7)	Open (benchmarking needed before deployment)
M	Medium	Renewal-queue griefing: about 1,000 short-period auto-renewing subscriptions could keep the shared RenewalQueue full, disabling auto-renewal for all users (Appendix D.8)	Fixed (commit facc3dc)

Six findings (B, E, I, J, K, M) have been remediated: B and E in commit 9ceca57; J and K in commit 221e5fa; I, together with E's cross-chain path, in commit a20ad8c; and M in commit facc3dc. Regression tests are listed in Appendix D. The Finding B remediation still allows a cross-chain transfer when the owner signs the message on the sending chain and holds ownership through their derived account; two xcm-simulator tests (commit e258f1e) confirm both the permitted transfer and the rejection of a different user. Transfers from a sovereign account, or between a user's local and remote identities, remain future work (Appendix E.8). Three findings (A, C, D) are accepted design properties, three (F, G, H) are reliability observations, and L (unbenchmarked weights) remains open.

7.3.2 | Test Coverage

Our unit test suite covers **18 of the 20 distinct error variants** defined in pallet-content-rights (90 percent), exceeding the Phase 2 objective of 80 percent. One of these, `NoViewsRemaining`, is no longer reachable through the public API after the Finding I fix; it is tested by writing an empty view pack directly to storage. Two variants remain untested: `ItemIdOverflow` requires exhausting a u32 counter under normal extrinsic dispatch, and `NftOperationFailed` is triggered only by internal pallet-nfts state corruption that the mock runtime does not replicate. The comprehensive error variant table is provided in Appendix D.

7.3.3 | Decentralization Assessment (HHI)

The Herfindahl-Hirschman Index (HHI) was calculated from the Polkadot mainnet validator distribution (parachains inherit relay-chain security). Comparison values for Ethereum PoS and Solana are our estimates from publicly reported stake distributions (Rated Labs, 2026; Solana Compass, 2026); the U.S. DOJ Horizontal Merger Guidelines define HHI below 1,500 as unconcentrated (U.S. Department of Justice and Federal Trade Commission, 2010). Table 7.14 presents the comparison.

Table 7.14. Herfindahl-Hirschman Index (HHI) Comparison

System	Validators / Entities	HHI (approx.)	Nakamoto Coefficient
Polkadot mainnet (NPoS, ~600 validators, as of September 2026; Polkadot OpenGov, 2025)	~600 (near-equal stake)	~16.7	~160–200
Ethereum PoS	~900K validators (concentrated in ~5 staking providers)	~1,200– 2,000	~5–7
Solana	~1,900 validators (top-heavy stake distribution)	~200–500	~19–25

Continued on next page

Table 7.14 continued from previous page

System	Validators / Entities	HHI (approx.)	Nakamoto Coefficient
Centralized DRM (e.g., Spotify)	1 entity	10,000	1
US DOJ “unconcentrated” threshold	–	<1,500	–

Polkadot’s HHI of ~ 16.7 (as of September 2026, ~ 600 validators) is well below the U.S. DOJ “unconcentrated” threshold of 1,500 and orders of magnitude lower than a centralized monopoly at 10,000. The Nakamoto coefficient (~ 160 – 200 for Polkadot vs. ~ 5 – 7 for Ethereum PoS and 1 for centralized DRM) points the same way: counted per validator, Polkadot’s HHI is about 600-fold lower than that of a single-entity centralized system, and nominally 70–120-fold lower than Ethereum PoS and 12–30-fold lower than Solana.

The HHI in Table 7.14 counts validators, not the entities that control them. Several operators run many Polkadot validators, so the entity-level HHI is higher than 16.7 and the Nakamoto coefficient lower; we did not compute them. The Ethereum row, by contrast, reflects concentration among staking providers, so the rows are not strictly comparable. The conclusion is nonetheless robust: HHI for N equal-sized entities is $10,000/N$, so Polkadot’s validators would have to be controlled by fewer than seven equal entities to reach the 1,500 threshold.

Table 7.14 should be read as a statement about safety under a production parachain, not about the test network. The relay-chain HHI indicates who secures CCRMS state once a block is included. The evaluated prototype’s liveness is limited to a single collator (Nakamoto coefficient 1 for block production). A deployed CCRMS would inherit Polkadot’s validator set for safety and would still need a collator set (or equivalent coretime) for liveness. The validator-level 600-fold HHI contrast with centralized DRM is therefore a property of the intended hosting environment, not of the Zombienet used in Chapter 7.

7.3.4 | Static Analysis and Dependency Audit

cargo clippy reported zero warnings. cargo audit reported 8 advisories in transitive SDK dependencies; none affect pallet code and all are mitigated by the sandboxed WASM

runtime architecture (full list in Appendix D).

7.4 | Summary

Table 7.15 consolidates the measured outcomes for all 11 KPIs.

Table 7.15. KPI Results Summary

#	KPI	Target	Measured	Target met?
1	Throughput	>10 TPS	26.2 TPS sustained (harness-limited lower bound)	Yes
2	Inclusion latency	<12 s	Median ~6.0 s; 42 of 53 batches in one block; 43 of 53 within 12 s (Section 7.1.3)	Median
3	XCM inclusion delay	<10 blocks	3–7 blocks (median 5, ~30 s); 30/30 delivered	Yes
4	Block utilization	<75%	12.9% ref_time (declared) and 2.2% proof_size (measured) at 300 txs; construction ~62 ms	Yes
5	Storage efficiency	<500 bytes	191 bytes/content, 112 bytes/subscriber	Yes
6	Net chain growth	>85%	90% median; 7 of 8 windows above 85%, one at 85%	Median
7	MTTR	<60 s	17.9 s to first block kept (SIGKILL); 47 s to finality	Yes
8	State persistence	100%	4 of 4 runs intact	Yes

Continued on next page

Table 7.15 continued from previous page

#	KPI	Target	Measured	Target met?
9	Security findings	No Critical/High unremediated at submission	2 High (Findings B, J) identified and patched by the author; 0 unremediated Critical/High findings in that audit. No third-party review (Section 1.5.3).	Met as interpreted: no unremediated Critical/High findings in the author's audit. Not independently verified.
10	Test coverage	>80%	90% (18/20 error variants)	Yes
11	Decentralization ratio	Documented	Up to 270× lower throughput; qualitatively superior decentralization	Yes

Eight of eleven KPIs met their Table 6.5 targets in every measurement. KPIs 2 and 6 are met at the median (KPI 2: ten of 53 batches exceeded 12 s; KPI 6: one of eight 120 s windows reached exactly 85%), and KPI 9 is recorded as met only under the interpretation “no unremediated Critical/High in the author's audit”; there was no third-party review (Section 1.5.3). The KPI 9 target was originally “0 Critical / 0 High findings”; it was reworded to “no unremediated Critical/High findings at submission” in September 2026, after Findings B and J were identified, because the audit's purpose was to find and fix such issues rather than to certify their absence.

Discussion

This chapter analyzes the findings of Chapter 7 by identifying the questions addressed, the decisions substantiated, and the limitations that shape the interpretation of the contribution.

8.1 | Answering the Research Questions

The primary question asks what levels of transaction finality, cost efficiency, and creator revenue retention the framework achieves. On **finality**, a cross-chain rights operation reached a finalized CCRMS block a median of about 68 s after submission on the sending chain (range 52–84 s, $n = 30$; Section 7.1.3), and a local operation becomes final about 35–40 s after its block is authored (Section 7.1.6). On **cost efficiency**, we did not measure per-operation fees: the pallet’s weights are hand-estimated rather than benchmarked, so fees derived from them would not be meaningful, and the XCM fee was set by placeholder instruction weights (Appendix A.6). The simulation’s 1–5% CCRMS fee is an assumption (Table 7.12), so this part of the question remains unanswered. On **creator revenue retention**, the answer is the conditional one in Section 8.1.5.

8.1.1 | SQ1: Cross-Chain Rights Operations and Finality

The key finding is that the payer–beneficiary decoupling pattern, combined with XCM’s Transact instruction, is sufficient to delegate rights issuance to the authoritative chain without requiring two-phase commit. The remote parachain pays; CCRMS issues the rights record; no distributed rollback is needed because rights state is always authoritative on one chain.

Affirmative for issuance, with caveats. Standard XCM instructions, available since v3 and executed here by a v5 executor, are sufficient for executing cross-chain rights operations: the canonical sequence `WithdrawAsset` → `BuyExecution` → `Transact` carries the four `xcm_*` operations exercised in the xcm-simulator tests (subscribe, renew, purchase views, purchase ownership); `xcm_transfer_ownership` uses the same sequence and, after the Finding B fix, succeeds from another chain only when the owner signs the message there and holds ownership through their derived account; we tested that path in the xcm-simulator, not on Zombienet (Appendix D.1). Each operation is atomic within CCRMS's execution, but XCM `Transact` does not provide atomicity across chains (Section 1.5.2). Cross-chain operations were delivered in 3 to 7 blocks (median 5, about 30 s; 30 of 30 runs; Section 7.1.3) and were final on CCRMS a median of about 68 s after submission. Atomic success rates, which SQ1 also asks about, reduce here to delivery: all 30 messages executed, but without error reporting the sending chain cannot observe a failure (Section 8.5). No custom XCM extensions are required. A further caveat is recurring renewal. XCM has no scheduling instruction, so auto-renewal runs on the CCRMS chain as an expiry-keyed queue in `on_initialize`; renewals paid from another chain would need that chain to schedule XCM sends (e.g., with `pallet-scheduler`). This demonstration is inter-parachain (local HRMP). It is not an end-to-end Ethereum → CCRMS rights path (Section 1.5.2).

8.1.2 | SQ2: Latency versus Decentralization Trade-Offs

Partially. SQ2 asked for design patterns that minimize access-verification latency and for comparisons across validator configurations. We measured one configuration (two relay-chain validators, one collator) and did not compare alternative access-verification designs, so the answer below holds for that configuration only. The trade-off is real but highly asymmetric: CCRMS operational latency is approximately $67,000\times$ higher than the centralized baseline (Table 7.7: $\sim 6,000$ ms vs. 0.09 ms), while a production parachain would inherit an HHI of ~ 16.7 compared with 10,000 for a centralized monopoly (Table 7.14), a decentralization gain, counted per validator, of about 600-fold (nearly three orders of magnitude) versus centralized DRM on that index, for safety, not for test-network liveness (Section 7.3.3). For discrete, low-frequency rights operations (the CCRMS use case), the trade-off is advantageous; for high-frequency interactive applications it is not.

8.1.3 | SQ3: Unified Rights Record Enforcing All Three Monetization Models

Yes, within scope. Three maps and one priority-ordered check enforce all three access semantics within a single record (Section 5.1), comparable to single-chain standards (Section 2.3.1). The distinguishing property is that the record supports cross-chain issuance and proof-based verification against a CCRMS state root, addressing G7; binding that root to finalized CCRMS state is not implemented (Section 5.2.1).

8.1.4 | SQ4: Security and Authorization

Partially. The author's audit and code review found two High-severity issues (B, J) and four Medium (I, K, L, M); all except L (unbenchmarked weights) are remediated, but the review was not independent, and the remediation of B authorizes cross-chain ownership transfer only by the owner's own derived account, not by a delegate. `xcm_transfer_ownership` accepted a `from` identity parameter without verifying that the caller held it; a one-line `ensure!` check (Finding B, Section 7.3.1) remediated the issue. Because a signed message from another chain arrives with a per-user origin (`DescendOrigin`, mapped to a local account through `HashedDescription`), the same check accepts the genuine owner and rejects any other user of that chain (commit `e258f1e`); CCRMS must still trust the sending chain to identify its signer correctly. The lesson generalizes: **cross-chain extrinsics that accept identity parameters require explicit authorization checks.**

8.1.5 | SQ5: Economic Viability under Realistic Creator Audience Distributions

Conditionally, and by a small margin. The Monte Carlo simulation (Section 7.2) indicates, at the level of the design, that CCRMS yields 4.4% higher mean creator revenue than the centralized configuration and the same margin over Web3 marketplaces, whose published fees (0–2.5%) are as low as CCRMS's; the Web3 margin depends on unsourced churn and discovery assumptions and changes sign within their plausible range. The comparison with bridge-based distribution uses an assumed fee range, so it is reported only as an illustrative comparator (Appendix G). Against Web3 marketplaces, CCRMS's case rests on capabilities the simulation does not model: native subscription and pay-per-view in one record, and cross-chain issuance. The advantage is fragile: it disappears if the CCRMS fee exceeds about 7% or the centralized discovery bonus is about 25% larger than assumed, and without the secondary-sale stream that the prototype does

not pay to creators it falls to 0.6% (Appendix G). The median simulated creator earns less under CCRMS (Section 7.2.1), and the advantage depends heavily on audience size (Section 8.2.2). Our contribution is to make this conditionality **quantitative and reproducible**, together with the assumptions on which it depends.

8.2 | Interpretation of Key Results

8.2.1 | The 26.2 TPS Figure in Context

The 26.2 TPS figure comes from a single-collator local Zombienet. Asynchronous backing was active: the runtime uses the parachain template's configuration (lookahead collator, unincluded segment of three, 6 s slots), and the relay chain's scheduling lookahead was 5,¹ which is why blocks arrive every ~ 6 s. Elastic scaling was not used (one core per parachain), and Agile Coretime does not apply to a local relay chain with genesis-registered parachains. The figure is not the chain's capacity either: the closed-loop benchmark client limited it (Section 7.1.2). Block construction for 300 transactions took about 62 ms of a 6 s slot, so a single collator could probably include considerably more, but we did not measure the saturation point. Adding parachains raises the network's aggregate throughput but not that of a single application. A busy CCRMS would instead need elastic scaling, which assigns several cores to one parachain at the cost of the additional coretime (Appendix E.2). The upper bound of $270\times$ relative to the centralized baseline therefore needs context. CCRMS is designed to process low-frequency events; it records discrete rights transactions with cryptographic finality, automatic royalty propagation, and cross-chain portability, none of which the centralized baseline provides at any throughput level. Purchases, subscriptions and ownership transfers occur far less often than content views, and even at the measured 26.2 TPS a single chain can record about 2.3 million such events per day. Pay-per-view is the exception: `consume_view` is one transaction per view, so a view-heavy workload scales with viewing rather than purchasing, and the same rate supports about 2.3 million metered views per day. Beyond that, a platform would need more capacity per chain (elastic scaling, above) or off-chain view metering with periodic on-chain settlement.

¹The relay chain's `asyncBackingParams` field reads zero in this configuration, but in the SDK version used, it is deprecated and ignored by validators; the permitted relay-parent ancestry is derived from the scheduling lookahead instead.

8.2.2 | The Economic Crossover at ~6,000 Followers

Where the economic advantage of CCRMS exists, it benefits creators with established audiences. Below ~6,000 followers, centralized platforms usually generate more revenue (CCRMS wins in 8.7% of scenarios at 1,500–2,000 followers; Table G.6) because the model gives centralized platforms a flat discovery bonus that matters most to small creators. That bonus has no published source, and the crossover moves with it (Table G.5). Within the model, CCRMS is best understood as a tool for creators who have outgrown the discovery benefits of centralized platforms; whether real creators cross over near 6,000 followers is an empirical question the simulation cannot answer.

8.3 | Architectural Decisions Revisited

The pallet-first decision avoided a dependency that was later discontinued. The pivot was driven by three engineering constraints (Appendix A.2), and it was taken against the backdrop of the January 2026 announcement that ink! development would be discontinued, which removed any expectation that the contract layer would mature. The episode illustrates that architectural decisions reducing dependence on any single non-essential component become more valuable as the ecosystem evolves (Bass et al., 2021; Parnas, 1972). This principle is also reflected in the choice of pallet-nfts over the frozen RMRK pallets and in HRMP rather than custom inter-parachain protocols. The same pattern holds for the Snowbridge integration: the configuration required for Ethereum bridging was completed at minimal additional cost, though the Ethereum-to-CCRMS path itself remains untested.

The unified rights record and the payer–beneficiary pattern also held: the former shows linear storage growth, and the latter is sound subject to Finding B’s authorization requirement.

8.4 | The Work in Context

The Polkadot ecosystem evolved during implementation. **Polkadot 2.0** features reached production by late 2025. The prototype already used asynchronous backing; elastic scaling, the main remaining throughput lever, was not exercised, so our figures are local-testnet measurements rather than forecasts of production performance. **Snowbridge V2** launched after our V1-based integration, which we have not retested against it. **DATA Foundation** (formerly Story Protocol, rebranded June 2026 to focus on AI training-data provenance) strengthens the case that the cross-chain creator-rights niche remains un-

served; comparisons with the Programmable IP License remain relevant. **EU regulation** (AI Act, MiCA) drives demand for cryptographic provenance; MiCA's treatment of subscription tokens remains legally unclear. Recent cross-chain surveys (Deng et al., 2025; Li et al., 2025) and formal bridge verification (Maric et al., 2025) provide context but do not change the conclusions.

8.5 | Limitations

The CCRMS prototype has eight principal limitations:

1. **No production deployment.** All measurements are from a local Zombienet topology with two relay-chain validators, single-collator parachains, and no real-world latency. Chapter 7 results are upper bounds in this test environment and do not predict production behavior.
2. **Single-author validation.** A single author performed the implementation, test design, measurement collection, and interpretation. No independent code review, third-party security audit, or external benchmark validation has been conducted. Production deployment would require all three before any non-trivial value passes through the system.
3. **Local-only Snowbridge integration.** Geth and Lodestar ran on the same machine as the Polkadot side, avoiding many realities of public Ethereum (variable block times, MEV, network contention, beacon chain reorgs). The integration serves as a proof of architectural feasibility but provides no data on production bridge behavior.
4. **Holder scale untested.** A bounded Children index originally limited each content item to fifty subscribers, view-pack holders and owners over its lifetime. The index duplicated the Parent map and the rights maps, so commit 2865ed0 removed it; a unit test now admits 60 holders. No benchmark has exercised the audience sizes at which CCRMS wins (Section 7.2.5). Each child NFT also reserves `pallet-nfts` item and attribute deposits from the creator, about two existential deposits per holder, so a large audience requires a matching reserved balance (Appendix E.4). The Monte Carlo result therefore remains a design-level result (Claim 3, Section 1.5.1).
5. **No empirical proof of the bridged-payment final hop.** The bridged-ETH-pays-for-CCRMS-operation flow is conceptually straightforward but has not been tested

end-to-end. We claim forward-compatibility with the flow, which is a weaker claim than an empirical demonstration would provide.

6. **Simulation input sensitivity.** Content price scales revenue on every platform by the same factor, so the relative result is insensitive to it by construction, not by empirical test. The result depends strongly on the assumed audience distribution, subscription period, CCRMS fee, and value of algorithmic discovery, none validated against real data, and consumer onboarding friction is unmodeled.
7. **Auto-renewal capacity.** The `RenewalQueue` processes at most ten renewals per block and searches at most 100 blocks ahead, so about 1,000 renewals can fall due in any window and about 4.3 million per 30-day month across the whole chain. If the ten million subscribers in the storage projection (Section 7.1.5) all renewed monthly, the chain would need about 23 renewals per block. Renewals that spill into later blocks leave access lapsed for up to 100 blocks (about 10 minutes) and shift the next expiry by the same amount. When no slot is free, `enable_auto_renew` fails with `RenewalQueueFull`, and a renewal that cannot be re-queued turns auto-renew off, with only an event to record it.
8. **Incomplete XCM message.** The benchmark message (Appendix C.2) buys execution with `weightLimit: Unlimited` but contains no `RefundSurplus` or `DepositAsset`, so fees withdrawn and not spent are left in the asset trap on CCRMS instead of returning to Para B's sovereign account. It also contains no `SetErrorHandler`, `ReportError` or `ReportTransactStatus`, so Para B is never told whether the `Transact` succeeded; a failed purchase is visible only as an event on CCRMS. A production message would return surplus fees to the payer and report the outcome to the sender, which in turn needs a pallet on the sending chain to act on the report.

8.6 | Implications

The findings extend beyond the immediate prototype in four directions.

For platform designers, the overarching lesson is that **architectural choices that minimize reliance on any single non-essential component pay dividends during ecosystem-level changes**. Two corollaries: **cross-chain extrinsics that accept identity parameters require explicit authorization audits** (Finding B), and **reimplementing an unmaintained standard is preferable to depending on it** (the RMRK 2.0 pivot).

For creators, the Monte Carlo simulation suggests that the question “Is a decentralized platform actually better for me?” has a conditional answer: **yes, depending on audience size**. Creators with over 10,000 followers usually benefit from lower fees (in about three quarters or more of simulated scenarios), whereas those with fewer than ~6,000 are usually better served by centralized discovery. The choice should reflect audience size, and the threshold depends on how much discovery a centralized platform actually provides.

For regulators, on-chain rights management systems should be treated as **compliance infrastructure** for emerging copyright and AI training regimes rather than as alternative payment platforms. The open question about MiCA’s classification of subscription tokens with fungible characteristics would benefit from regulatory clarity, as the current ambiguity poses risks for would-be EU deployers.

For the research community, the prototype, implementation specifications, and Monte Carlo simulation are intended to support follow-up research. The methodological contribution is a worked DSR example for multi-component blockchain prototyping. The empirical contribution is the audience-dependent finding, which future research could refine using real-world migration data or models that incorporate creator psychology and switching costs.

8.7 | Critical Self-Reflection

In retrospect, we would have **initiated the Snowbridge integration earlier**; the setup complexity (Appendix A) required several days of unexpected debugging. We would also have **prioritized FRAME benchmarking earlier** to report fee-adjusted throughput. We would **not** change the pallet-first decision, the unified rights record model, or the cross-chain operation pattern.

8.8 | Summary

SQ3 is answered affirmatively, and SQ5 conditionally, by a small margin that depends on audience size and on unsourced assumptions. SQ1, SQ2 and SQ4 are answered in part: cross-chain issuance works, but ownership transfer from another chain works only for rights held through the owner’s derived account and was tested only in the xcm-simulator; latency was measured in one validator configuration only; and the security evaluation was the author’s own. Of the primary question’s three measures, finality and revenue retention are quantified and cost efficiency is not. The most economically

significant finding is the audience-dependent CCRMS advantage. The major architectural decisions held up, the pallet-first decision most clearly. The eight principal limitations constrain how the contribution should be read but do not invalidate it. Chapter 9 presents the conclusion and future work.

Conclusion and Future Work

This closing chapter restates the contributions, summarizes the key findings, identifies the most important threads for future work, and offers a brief closing reflection.

9.1 | Summary of Contributions

Section 1.4 stated three contributions. We expand each here.

1. **A feasibility result for a cross-chain unified rights record.** Subscription, pay-per-view, and ownership are native modes of a single content record: three storage maps, one priority-ordered check, and shared royalty logic. Single-chain standards (ERC-5643 (cygaar, 2022), ERC-4907 (Anders and Shrug, 2022), Unlock Protocol (Unlock Protocol, 2026)) express similar access; the contribution makes the record acquirable via XCM and verifiable by storage proof against a trusted CCRMS state root, which the prototype does not yet supply. Issuance was demonstrated on local HRMP; owner-signed cross-chain transfer was demonstrated in the xcm-simulator only. This addresses G7.
2. **Quantified decentralization–latency and revenue trade-offs.** Prior work endorsed decentralization qualitatively or measured only centralized performance. This dissertation quantifies the trade-offs directly. The latency overhead is $67,000\times$ and the throughput penalty is at most $270\times$ against a centralized Express.js baseline (the CCRMS figure is harness-limited); the Herfindahl-Hirschman Index, counted per validator, is approximately 600-fold lower than for centralized DRM (Polkadot HHI ~ 16.7 , based on approximately 600 active validators as of September 2026, vs. centralized DRM at 10,000). The Monte Carlo simulation shows CCRMS yields a

4.4% higher mean revenue than the centralized baseline overall (Table 7.8), with the advantage concentrated among creators with 10,000 or more followers; the median simulated creator, however, earns less under CCRMS, and the advantage disappears under modest changes to the fee or discovery assumptions. None of the content-rights studies reviewed in Chapter 2 quantifies these trade-offs for an on-chain system (gap G5, Table 2.1).

3. **A cross-chain rights artifact demonstrating XCM sufficiency.** The CCRMS parachain (pallet-content-rights, pallet-rights-verifier, an ink! 6 wrapper contract, an EVM read-only precompile, and benchmark and simulation scripts) demonstrates that standard XCM instructions with payer-beneficiary decoupling are sufficient for cross-chain rights issuance without custom protocol extensions (all five `xcm_*` operations in the `xcm-simulator`, with ownership transfer limited to the owner's own derived account). Seven implementation specifications (pallet design, XCM message flows, security threat model, royalty algorithm, precompile interface, NFT nesting model, and Snowbridge integration, the last covering the integration and its debugging findings but not end-to-end Ethereum-to-CCRMS compatibility, which remains untested) provide design knowledge intended to support future replication and extension. The artifact and specifications are released open source under the MIT license.

The dissertation also documents a DSR cycle applied to a multi-component blockchain prototype, which may serve as a reference for similar projects.

9.2 | Principal Findings

Six principal findings emerge from the work:

1. **The primary research question is answered in the affirmative for Polkadot parachains:** a Polkadot parachain with standard XCM instructions and FRAME can deliver the unified rights record. Issuance and access work; the Finding B fix limits cross-chain ownership transfer to the owner's derived account (`xcm-simulator` only), and the prototype has not been tested at large audience sizes. Of the question's three measures, finality (about 68 s to a finalized cross-chain delivery) and revenue retention were quantified; cost efficiency was not measured. Eight of eleven KPIs in Table 7.15 are met outright; KPIs 2 and 6 at the median and KPI 9 only as an author-audit interpretation (Section 7.4).

2. **The major architectural decisions held up.** The pallet-first decision avoided dependence on ink!, which was discontinued in January 2026; the RMRK 2.0 to pallet-nfts pivot, the unified rights record model, and the cross-chain operation pattern also held up in testing. In retrospect, we would not change these decisions.
3. **The economic advantage depends on audience size.** CCRMS usually benefits creators with 10,000+ followers but usually loses to centralized platforms below ~6,000 followers, where the assumed algorithmic discovery outweighs fee savings. The mean advantage is small (4.4%) and depends on unsourced discovery and churn assumptions. While CCRMS earns slightly more on average, the conditional framing better reflects individual creator decisions. This advantage lies in audience sizes the prototype can now represent but has not been tested at (Section 8.5), so the Monte Carlo result characterizes the design; it is not an empirical property of the prototype.
4. **Cross-chain operations are operationally feasible, but latency-bound.** Standard XCM instructions are sufficient for issuance-style rights operations on local HRMP. The destination-chain inclusion delay was measured on a local single-collator test-net with asynchronous backing active and one core per parachain; it describes that environment, not a fundamental limit. Ethereum interoperability is a configured runtime path plus a local ETH → AssetHub transfer; the rights pallet was not on that path.
5. **Standards reimplementations are sometimes preferable to standards adoption** when the reference implementation is unmaintained; the RMRK 2.0 pivot exemplifies this.
6. **Forward compatibility is a defensive engineering goal.** Configuring the runtime to accept Ethereum-originated XCM messages at minimal additional cost in Phase 4 completed the Snowbridge integration configuration; the `gindex` and `forkVersions` debugging findings (Appendix A) are a useful engineering result, though the Ethereum-to-CCRMS path itself remains untested.

9.3 | Future Work

11 threads of future work emerged from the research. Appendix E provides full descriptions. A cross-chain secondary market is out of scope in the current prototype; Appendix E.8 describes origin aliasing and delegation, which would extend `xcm_transfer_ownership` beyond owner-signed transfers.

Table 9.1. Future Work Summary

#	Thread	Priority	Key action
E.1	Production deployment	Near-term	Deploy to public testnet/mainnet; obtain third-party security audit
E.2	Scaling beyond the prototype	Near-term	Acquire coretime; enable elastic scaling (runtime velocity, unincluded segment and collator changes; no pallet changes); measure the gain.
E.3	Smart contract layer migration	Near-term	Choose among Solidity rewrite, wrevive adoption, or contract removal (ink! discontinued Jan 2026)
E.4	Holder-scale testing	Near-term	Benchmark content items with thousands of holders and review per-holder creator deposits; prerequisite for the economic findings to apply at scale
E.5	User study	Near-term	Build minimal wallet/CLI; test with Web3 creators and consumers (highest-priority research extension)
E.6	Bridged-payment final hop	Near-term	Configure PaymentCurrency for foreign Ether or add swap step
E.7	Resale royalties	Medium-term	Add optional <code>resale_royalty_basis_points</code> ; requires on-chain marketplace for price verification
E.8	Origin aliasing and delegation	Medium-term	Let remote users act as their local account (<code>AliasOrigin</code>) and authorize delegates
E.9	Monte Carlo extensions	Medium-term	Add creator psychology, network effects, longitudinal and regulatory modeling

Continued on next page

Table 9.1 continued from previous page

#	Thread	Priority	Key action
E.10	Formal verification	Long-term	Apply Isabelle/HOL techniques to authorization model, royalty algorithm, cross-chain pattern
E.11	JAM transition	Long-term	Migrate to JAM (no mainnet date announced as of mid-2026; multi-client implementations completing early milestones); no rebuild required

9.4 | Closing Reflection

CCRMS started with the idea that creators deserve more value, and blockchain offers a way to achieve this. We built a prototype, tested its operations, and simulated its economic effects. The premise held only in part: the Monte Carlo simulation shows a small advantage for creators who have outgrown centralized platforms' discovery value, not a universal one, and that advantage rests on assumptions we could not validate. An honest, conditional claim is more valuable than a universal one.

The Polkadot ecosystem evolved significantly during implementation, with major updates like Polkadot 2.0, Snowbridge V2, and others. None invalidated our work because we chose conservatively: we favored stable primitives, minimized reliance on non-essential parts, and preferred proven deployment patterns. This highlights a study in **defensive engineering** and **conservative, well-documented architecture** as the most durable approach.

CCRMS is a prototype in a single blockchain ecosystem, addressing a specific industry's structural issues. We do not claim that it is the future of content rights management. However, it shows that content rights management can be built on open, decentralized, cryptographically verifiable infrastructure, serving as a starting point for others to build upon.

Implementation Challenge Narratives

The nine challenges summarized in Table 5.1 are presented here in full detail with Challenge, Response, and Lesson for each.

A.1 | The Pivot from RMRK 2.0 to `pallet-nfts`

Challenge. The original Phase 3 design envisioned using the RMRK 2.0 nested-NFT specification, implemented atop the `rmrk-substrate` pallet suite. RMRK 2.0 precisely offers the parent-child nesting semantics that the CCRMS unified rights record model requires. Reusing an established standard is intended to reduce implementation risk. However, during initial investigations in Phase 4, we discovered that the `rmrk-substrate` pallets were frozen on Polkadot SDK version 0.9.36 and relied on the now-deprecated `pallet-uniques` instead of the modern `pallet-nfts`. Our efforts to compile `rmrk-substrate` against the current Polkadot SDK monorepo were unsuccessful because of incompatible storage schemas and removed traits.

Response. We decided to replicate RMRK 2.0's nesting semantics directly on top of `pallet-nfts` without depending on any RMRK pallets. This involved designing a parent-child storage layout (a Parent index, originally paired with a bounded Children index that was later removed), implementing the lifecycle helpers (`mint_and_nest_child`, the burn and re-mint patterns in `transfer_ownership`), and tagging child NFTs with a `rights_type` attribute via `pallet_nfts::set_attribute`. The implementation took longer than reusing `rmrk-substrate` would have, but the result is fully integrated with the modern Polkadot SDK and depends only on actively maintained pallets.

Lesson. Established libraries pose latent compatibility risks when they are not kept up to date with the current ecosystem. Reimplementing essential primitives from scratch

was justified by the maintainability gain and by the opportunity to demonstrate that RMRK-style composability does not require RMRK itself.

A.2 | The Pivot from Contract-First to Pallet-First Architecture

Challenge. The Phase 3 design designated ink! smart contracts as the primary interface for users, featuring three contracts (each corresponding to a different monetization model) to manage user operations. A chain extension mechanism was incorporated to facilitate access to a lightweight pallet dedicated to state coordination. During Phase 4 implementation, two issues arose concurrently: first, the chain extension mechanism within the current pallet-revive release was effectively non-operational for write activities; second, the operational complexity of debugging cross-pallet contract calls was significantly greater than that of direct FRAME pallet interactions.

Response. We inverted the architecture. The pallet (pallet-content-rights) became the definitive source of truth for all business logic, whereas the ink! contract (rights_manager) was retained as a streamlined user-facing interface that mirrors the pallet's API but holds no independent storage. We developed a custom EVM precompile (ContentRightsPrecompile) to provide read-only access to pallet state for Solidity-based clients, replacing the original chain extension design.

Lesson. The pallet-first decision avoided a dependency that was later discontinued. The pivot was driven by engineering constraints, and it was taken against the backdrop of the January 2026 announcement that ink! development would be discontinued, which removed any expectation that the contract layer would mature. The episode illustrates that architectural decisions reducing dependence on any single non-essential component become more valuable as the ecosystem evolves.

A.3 | The Snowbridge Fork-Version Configuration Issue

Challenge. During the integration of Snowbridge, the beacon relay generated Merkle inclusion proofs at incorrect tree positions (gindex 54 instead of 86 for the Electra fork), leading to the rejection of all submitted updates by the Bridge Hub's EthereumBeaconClient pallet.

Response. The relayer's forkVersions field holds fork activation epochs, not 4-byte fork version identifiers. Our configuration used version identifiers, so Electra's

0x05000000 was interpreted as epoch 83,886,080. The relayer therefore treated every slot as pre-Electra and built `current_sync_committee` proofs at gindex 54, while the Bridge Hub light client, with Electra active from epoch 0, expected gindex 86. We replaced the identifiers with epoch numbers (`{ "deneb": 0, "electra": 0, "fulu": 5000000 }`, Fulu being a far-future epoch the local chain never reaches). After the change, both the relayer and the pallet computed gindex 86, and the checkpoint was accepted.

Lesson. Cryptographic verification systems are sensitive to silent configuration discrepancies: the failure mode is “verification fails” with no informative error pointing to the bad value.

A.4 | The Lodestar Preset Version Pinning

Challenge. The `EthereumBeaconClient` pallet within Bridge Hub is compiled with `pubkeys: [PublicKey; 512]`. Any beacon header update originating from a smaller sync committee (e.g., the 32-validator minimal preset used by Lodestar’s development mode) is rejected during deserialization. Configuring Lodestar v1.41.0 with `LODESTAR_PRESET=mainnet` proved ineffective; the development mode silently disregarded the environment variable.

Response. Lodestar v1.35.0, when built from source, adheres to the `LODESTAR_PRESET=mainnet` variable. We modified the setup scripts to clone, build, and pin v1.35.0.

Lesson. Cryptographic light clients impose strict requirements that cannot be circumvented through configuration alone. Toolchain compatibility for a research-stage bridge is fragile and must be reconfirmed with each component upgrade.

A.5 | The XCM Sovereign Account Endianness Bug

Challenge. Our initial cross-chain XCM tests revealed a perplexing failure pattern: the source parachain reported successful XCM dispatch and balance deduction, yet the target parachain indicated `InsufficientPayment`.

Response. The source chain constructed the sovereign account using `polkadot.js`’s `toHex()` in default big-endian mode, whereas the CCRMS pallet expected little-endian byte order (the `SiblingParachainConvertsVia` convention). This produced two separate accounts: one funded but unused, one used but unfunded. We resolved the issue by changing to `toHex(true)`.

Lesson. Endianness bugs are particularly insidious because both orderings produce valid 32-byte addresses and the symptom (insufficient balance) misdirects the

investigation.

A.6 | The XCM Fee Scaling for Proof Size

Challenge. Initial XCM tests set the `BuyExecution` fee at 1B tokens, and every test failed on CCRMS.

Response. The runtime converts weight to fee using `BlockRatioFee<1,1>`, which prices each weight dimension by its share of the block limit and charges the higher value. Our XCM configuration uses `FixedWeightBounds`, assigning every instruction a flat 1B `ref_time` and a 64 KiB proof size (Finding C). Three instructions therefore declare 196,608 bytes of proof size: 3.75% of the 5 MiB PoV limit, against 0.15% of the `ref_time` limit. The proof-size component, about 75B tokens, sets the fee. We set test fees to 100B tokens to leave a margin for the `Transact` call's own weight.

Lesson. Under `BlockRatioFee`, the fee is determined by whichever weight dimension accounts for the larger share of the block. Placeholder XCM weights can make proof size the fee-setting dimension even for trivial messages; benchmarked XCM weights would likely reduce the fee considerably.

A.7 | Finding B: The Authorization Gap in

`xcm_transfer_ownership`

Challenge. During the Phase 5 security audit, we identified an authorization flaw in `xcm_transfer_ownership`: the initial implementation performed only `ensure_signed(origin)` before burning/minting NFTs based on a `from` parameter, allowing any signed account to steal ownership.

Response. We added `ensure!(authorizer == from, Error::::Unauthorized)` and a unit test (`security_xcm_transfer_ownership_rejects_non_owner`) that verifies the exploit is blocked.

Lesson. Any cross-chain extrinsic accepting a “from” identity parameter requires explicit authorization. Payer-beneficiary separation introduces impersonation risk absent from local-only calls.

A.8 | The Auto-Renewal `MaxChildrenReached` Boundary

Challenge. Batches of 100 subscribe calls to the same content item failed midway with `MaxChildrenReached`. The `Children` storage entry is limited to 50 child NFTs by `BoundedVec<_, ConstU32<50>>`.

Response. We documented the constraint as a known limitation and updated benchmark scripts to rotate content items. Subsequent analysis showed the bound to be stricter than a concurrency limit. Children are minted per buyer and released only when a view pack is exhausted, so expired subscriptions permanently occupy their slots. The bound therefore limits a content item to fifty subscribers, view-pack holders and owners over its lifetime. Commit 2865ed0 later removed the `Children` index and with it the cap; Section 8.5 discusses what remains untested.

Lesson. Bounded FRAME collections trade benchmarkability for capacity limits, and a bound that suits a benchmark may conflict with the target workload. Bounds should be checked against the audience sizes the system is meant to serve, and redundant indexes should be removed rather than bounded.

A.9 | The ink! Discontinuation and Its Implications

Challenge. On 27 January 2026, the ink! Alliance announced the cessation of ink! development following three failed OpenGov treasury funding proposals. Both the Web3 Foundation and Parity Technologies declined continued funding. The announcement outlined three migration options: Solidity via `revive`, low-level Rust against `pallet-revive`, or the community-maintained `wrevive` SDK.

Response. Because our pallet-first architecture (Appendix A.2) centers business logic in a FRAME pallet rather than in ink! contracts, the discontinuation has minimal practical impact. The ink! code in `contracts/rights_manager` serves as a non-load-bearing wrapper. We deferred migration.

Lesson. Architectural decisions that minimize dependence on any single non-essential component provide advantages during ecosystem-wide changes.

Research Objective Details

B.1 | RO1: Design and Prototype XCM Extensions

Statement. Design and prototype the use of XCM messages to transmit recurring rights operations and comprehensive rights metadata across Polkadot parachains. Evaluate atomic success rates and cross-chain finality durations.

Sub-questions: SQ1 primarily; SQ2 indirectly.

Deliverable: Functioning prototype with documented XCM latency and success rate metrics.

B.2 | RO2: Implement a Unified Rights Token

Statement. Implement a unified rights record (originally ink!, revised to FRAME pallet in Phase 4) with off-chain indexing support and access-verification latency measurement.

Sub-questions: SQ2, SQ3.

Deliverable: pallet-content-rights enforcing all three monetization models with documented latency.

Note: Off-chain indexer (SubQuery) was not implemented; pallet events suffice for on-chain queries via polkadot-js API. Acknowledged as a limitation.

B.3 | RO3: Merge Subscription, PPV, and Purchase Logic

Statement. Develop and integrate subscription, pay-per-view, and ownership transfer mechanisms into a unified, composable rights framework.

Sub-questions: SQ3, SQ4.

Deliverable: Single pallet implementing all three models through a unified API, with documented trade-offs.

B.4 | RO4: Benchmark Scalability, Interoperability, and Economic Efficiency

Statement. Benchmark against centralized DRM, bridge-based solutions, and (originally) Ethereum L2s.

Sub-questions: SQ2, SQ5.

Deliverable: Comparative TPS, latency, and creator revenue retention analysis.

Note: Ethereum L2 target was substituted with Express.js + SQL.js local benchmark in Phase 5. Core analysis is intact.

B.5 | RO5: Zero-Knowledge Proof Hooks (Not Met)

Statement. Integrate optional ZK proof hooks for selective regulatory disclosure.

Status: Not achieved. Integrating a ZK proof system into a Substrate runtime (selecting a proof system, building a verifier pallet, designing a disclosure protocol, producing test circuits) was substantially more complex than anticipated. Restated as future work in Section 9.3.

B.6 | RO6: Quantify Creator Savings via Monte Carlo Simulation

Statement. Quantify creator savings through Monte Carlo simulation comparing CCRMS with centralized (30–45% fees), Web3 marketplace, and bridge-based (8–15% fees) alternatives.

Sub-questions: SQ5.

Deliverable: Monte Carlo simulation with documented input distributions, model assumptions, and comparative output statistics.

Note: Phase 2 listed this as potentially replaceable by centralized benchmarking. Phase 5 included both; RO6 is achieved. The bridge-based fee range proved to have no published source, so that comparison is reported in Appendix G as illustrative only.

Code Listings

C.1 | Solidity Interface: ContentRightsPrecompile

The EVM precompile (Section 5.1) exposes the following Solidity interface. Solidity contracts deployed on pallet-revive can call these methods to read CCRMS rights state without crossing the EVM/Substrate boundary at write time.

```
1 interface IContentRights {
2     function checkAccess(uint32 contentId, address who)
3         external view returns (uint8 rightsType);
4
5     function getContent(uint32 contentId) external view returns (
6         address creator,
7         bytes32 metadataHash,
8         uint128 subscriptionPrice,
9         uint128 ppvPrice,
10        uint128 ownershipPrice,
11        uint32 periodLength
12    );
13
14    function isOwner(uint32 contentId, address who)
15        external view returns (bool owned);
16
17    function getSubscription(uint32 contentId, address subscriber)
18        external view returns (bool exists, uint32 expiryBlock);
19
20    function getViewPack(uint32 contentId, address viewer)
21        external view returns (bool exists, uint32 viewsRemaining);
22 }
```

C.2 | Canonical XCM Message for Cross-Chain Rights Operations

The canonical XCM message follows the three-instruction pattern `WithdrawAsset` → `BuyExecution` → `Transact`. It is constructed on the sending parachain (Para B) and sent to CCRMS via `polkadotXcm.send`; the `Transact` instruction carries the encoded call to one of the `xcm_*` extrinsics (Section 5.2.1), which CCRMS dispatches from Para B's sovereign account. The JavaScript form below shows the message as submitted via `@polkadot/api` in the latency benchmark. It is V3-encoded; `pallet-xcm` converts it to XCM v5, the version the runtime executes, before delivery. In v5, the `requireWeightAtMost` value becomes `fallback_max_weight`, which is kept only for conversion to older versions; the v5 executor weighs the call from its own dispatch information.

```

1  const xcmMessage = {
2    V3: [
3      { WithdrawAsset: [{
4        id: { Concrete: { parents: 1, interior: 'Here' } },
5        fun: { Fungible: 100_000_000_000 }
6      } ] },
7      { BuyExecution: {
8        fees: {
9          id: { Concrete: { parents: 1, interior: 'Here' } },
10         fun: { Fungible: 100_000_000_000 }
11        },
12        weightLimit: 'Unlimited'
13      } },
14      { Transact: {
15        originKind: 'SovereignAccount',
16        requireWeightAtMost: { refTime: 1_000_000_000, proofSize: 100_000
17        },
18        call: { encoded: encodedCall }
19      } }
20    ]
21  };

```

C.3 | Repository Directory Layout

The CCRMS parachain repository follows the standard Polkadot SDK template structure:

```
content-rights-parachain/
```

```

|-- pallets/
| |-- content-rights/           # Core business logic and tests
| |-- rights-verifier/         # Cross-chain Merkle proof verification
|-- runtime/
| |-- src/
| | |-- lib.rs                 # Runtime composition
| | |-- configs/               # Pallet configurations and XCM
| | |-- precompiles.rs         # Custom EVM precompile
|-- contracts/
| |-- rights_manager/          # ink! 6 contract
|-- node/                      # Collator binary
|-- scripts/                   # Benchmarking, testing, deployment
|-- docs/                      # Setup and integration documentation

```

C.4 | Dispatchable Extrinsic Reference

Table C.1 lists all 17 dispatchable extrinsics of pallet-content-rights.

Table C.1. Dispatchable Extrinsic Reference in pallet-content-rights

Index	Extrinsic	Purpose
0	register_content	Create a new content item, instantiate the underlying NFT collection, mint the parent Content NFT
1	subscribe	Purchase a new subscription
2	renew_subscription	Renew an expired subscription
3	purchase_views	Purchase a pay-per-view pack of N views
4	consume_view	Decrement a view from an existing pack
5	purchase_ownership	Purchase permanent ownership
6	check_access	Verify whether a user holds any active rights
7	xcm_subscribe	Cross-chain subscription with payer-beneficiary decoupling
8	xcm_renew_subscription	Cross-chain renewal
9	xcm_purchase_views	Cross-chain pay-per-view purchase

Continued on next page

Table C.1 continued from previous page

Index	Extrinsic	Purpose
10	<code>xcm_purchase_ownership</code>	Cross-chain ownership purchase
11	<code>transfer_ownership</code>	Peer-to-peer secondary-market ownership transfer
12	<code>xcm_transfer_ownership</code>	Cross-chain ownership transfer (caller-must-be-owner)
13	<code>set_royalty_splits</code>	Configure the royalty distribution table (creator-only)
14	<code>enable_auto_renew</code>	Enable automatic renewal for an existing subscription
15	<code>disable_auto_renew</code>	Disable automatic renewal
16	<code>query_rights_metadata</code>	Emit a self-describing metadata event for cross-chain consumption

In addition to the extrinsics, the pallet declares one FRAME view function, `access_rights_of(content_id, who)`, which returns the holder's highest-priority right (Ownership, Subscription or PayPerView) or none. It is served through the `RuntimeViewFunction` runtime API, pays no fee and records nothing on-chain (commit 5cb09cd).

C.5 | Pallet Storage Layout Reference

Table C.2 lists all 10 storage items of `pallet-content-rights`; the former `Children` index was removed in commit 2865ed0.

Table C.2. Storage Layout of `pallet-content-rights`

Storage	Key → Value	Purpose
<code>NextContentId</code>	→ u32	Global content ID counter
<code>NextItemId</code>	<code>collection_id</code> → u32	Per-collection child item counter

Continued on next page

Table C.2 continued from previous page

Storage	Key → Value	Purpose
Contents	content_id → ContentMetadata	Master record per content
Subscriptions	(content_id, user) → SubscriptionInfo	Active subscription state
ViewPacks	(content_id, user) → ViewPackInfo	Pay-per-view balance
Ownership	(content_id, user) → OwnershipInfo	Permanent ownership records
Parent	(collection_id, child) → (u32, u32)	NFT nesting index (child to parent)
AutoRenewIndex	(content_id, user) → bool	Auto-renew-enabled subscriptions
RoyaltySplits	content_id → BoundedVec<10>	Collaborator payout shares
RenewalQueue	block → BoundedVec<(content_id, user), 10>	Renewals due at each block

Security and Coverage Findings

D.1 | Finding B: Authorization Gap in `xcm_transfer_ownership`

The original implementation performed only `ensure_signed(origin)` and then looked up `Ownership[content_id, from]`, burned the seller's NFT, and minted a new one for `to`. No check confirmed that the dispatch caller had any relationship to the `from` account, so any signed account could call the extrinsic with arbitrary `from` and `to` arguments and transfer ownership from any user to itself.

We rate this finding **High**. Exploitation requires only a signed account with sufficient balance for transaction fees, no relationship to the victim, and no prior state; the impact is unauthorized transfer of a permanent, transferable ownership right. We initially recorded the finding as Medium on the grounds that the affected extrinsic is reachable only through the cross-chain path in normal operation, but Finding A establishes that the `xcm_*` extrinsics are also callable locally by any signed account, which removes that mitigation.

Our remediation was `ensure!(authorizer == from, Error::::Unauthorized)`, committed alongside unit test `security_xcm_transfer_ownership_rejects_non_owner`.

This remediation still permits genuine cross-chain transfers, but only by the owner's own chain-derived account. When a sibling parachain sends an XCM Transact from its own root, as in our benchmark, the call dispatches under that parachain's sovereign account, which can never equal `from`. When an ordinary user on the sibling chain signs `polkadotXcm.send`, `pallet-xcm` prepends `DescendOrigin(AccountId32)`, so CCRMS receives the origin `(1, [Parachain(n), AccountId32])`. The runtime's

`LocationToAccountId` converts that location through `HashedDescription` into an account derived from the user's chain and key, and `Transact` with `SovereignAccount` kind dispatches under that derived account. If the derived account holds the ownership, the equality check holds remotely. Two xcm-simulator tests (commit e258f1e) confirm this: a signed Para B user transfers ownership held by their derived account, and a second Para B user attempting the same transfer is rejected with `Unauthorized` after the message executes under that user's own derived account. Three qualifications apply. First, the derived account is distinct from the same person's local CCRMS account, so ownership bought locally cannot be moved this way, and the derived account needs a CCRMS balance to pay execution fees. Second, CCRMS trusts the sending chain to prepend the correct `DescendOrigin`; a malicious or compromised sibling runtime could act as any of its own users, though not as users of another chain or of CCRMS itself. Third, the tests run in the xcm-simulator against a mock runtime aligned with the production XCM configuration; we did not repeat them on Zombienet. Appendix E.8 describes how XCM v5 origin aliasing would let a remote user act as their local account.

D.2 | Finding E: View Pack Overwrite

The original `purchase_views` always overwrote `ViewPacks[content_id, buyer]` with a new `ViewPackInfo`, so a user holding 10 unused views who purchased 5 more would end up with 5, not 15. Our remediation made purchases additive: if a view pack already exists, new views are added to the existing balance.

This remediation initially covered the local `purchase_views` path only; the XCM path (`xcm_purchase_views`) still overwrote `ViewPacks` and minted a fresh child NFT on each call, orphaning the previous one. Commit a20ad8c moved both paths onto a single shared purchase function, so cross-chain purchases are now additive as well. Regression tests: `security_purchase_views_adds_to_existing_pack`, `xcm_purchase_views_adds_to_existing_pack` and `local_then_xcm_purchase_views_share_one_pack`.

D.3 | Accepted Design Findings (A, C, D)

Finding A: `xcm_*` extrinsics are callable locally as gift transactions (economically neutral; documented for transparency). **Finding C:** Runtime XCM `SafeCallFilter` is set to `Everything`; the `XcmExecuteFilter`, `XcmTeleportFilter`, and `XcmReserveTransferFilter` are similarly set to `Everything`, and `FixedWeightBounds` is used in place of measured XCM weights; production should restrict all four to explicit allowlists

(this also inflates XCM fees; Appendix A.6). We rate this Low for the prototype because the reach of Transact is bounded by the origin converter, not by the filter: the runtime has no superuser conversion, so a remote Transact dispatches only as a signed sovereign or derived account, or as a relay-chain or sibling native origin, and, except from the relay chain, must pay for execution (`AllowTopLevelPaidExecutionFrom`). With a signed origin, a remote chain can do nothing a funded local account cannot. The remaining exposure is calls whose weight cannot be bounded before dispatch, which is what `SafeCallFilter` exists to exclude. If deployed unchanged, we would rate it Medium.

Finding D: Content can be registered with all prices at zero (legitimate free-content feature, not a vulnerability).

D.4 | Finding I: Zero-View Denial of Service

Severity: Medium. **Status:** Fixed (commit a20ad8c).

Description. `purchase_views` accepted `num_views = 0`, charged `ppv_price × 0 = 0`, and still minted a child NFT. `consume_view` then failed with `NoViewsRemaining`, so the slot was never released. Fifty throwaway accounts could permanently block all future sales of any content item for the cost of transaction fees alone.

Remediation. Both purchase paths now call a shared function that rejects `num_views = 0` with a new `ZeroViews` error before any payment or minting. Regression tests: `security_zero_views_purchase_rejected` and `security_xcm_zero_views_purchase_rejected`. The fix removed the free exploit; commit 2865ed0 later removed the bound itself (Appendix E.4).

D.5 | Finding J: Child-NFT Divergence

Severity: High. **Status:** Fixed (commit 221e5fa).

Collections were created with default `pallet-nfts` settings, which allow child NFTs to be transferred directly via `pallet-nfts` extrinsics. Because `pallet-content-rights` did not disable this capability, the holder could sell or transfer their child NFT through `pallet-nfts` while the `Ownership` storage map still named them; a later `transfer_ownership` then burned the NFT from the buyer's wallet.

The fix creates all collections with `TransferableItems` disabled (only the runtime's `ForceOrigin` (root) can re-enable it) so child NFTs cannot be moved except through `transfer_ownership`. Burn failures now propagate as errors rather than being silently ignored.

Regression tests: `child_nft_cannot_be_transferred_via_pallet_nfts` and `transfer_ownership_works_with_non_transferable_children`.

D.6 | Finding K: Auto-Renewal Starvation and Non-Atomic Payment

Severity: Medium. **Status:** Fixed (commit 221e5fa).

The `on_initialize` hook reads the first 20 entries of `AutoRenewIndex` (in storage-hash order) every block. It skips unexpired entries at the front, and expired entries beyond them are never reached (starvation). Separately, multi-recipient royalty payments were not wrapped in a storage transaction: if one payment failed, earlier payments in the same renewal were not rolled back, leaving the pallet in a partially-paid state. The fix introduces a `RenewalQueue` keyed by expiry block: each block processes only its own bucket (at most ten renewals), and overflow spills to the following blocks, searching up to 100 blocks ahead (about 1,000 renewals per expiry window).

Regression tests: the four existing auto-renewal group tests plus `enable_auto_renew_fails_when_queue_full`.

D.7 | Finding L: Unbenchmarked Extrinsic Weights

Severity: Medium if deployed; Informational on the local testnet. **Status:** Open.

Description. Every weight in `pallet-content-rights` (13 functions in `weights.rs`) and in `pallet-rights-verifier` (three functions) is hand-set with `Weight::from_parts(..., 0)`: an estimated `ref_time` plus database reads and writes, and a declared proof size of zero. The `on_initialize` renewal hook also accounts only for database operations. The runtime's `StorageWeightReclaim` extension records the proof size each transaction actually consumed, which is why the proof-size figures in Section 7.1.4 are measured, but it does so after dispatch. Fees and block admission use the declared weight, so on a live parachain the proof-size cost of a transaction is not charged and cannot be bounded before it executes, and the `ref_time` figures are estimates rather than measurements.

Remediation. Benchmark both pallets with FRAME benchmarking, including worst cases (the maximum number of royalty splits, a full `RenewalQueue` bucket, the deepest Merkle proof), and replace the hand-set values with the generated weights. Bench-

marking was outside the scope of the prototype; it is a prerequisite for deployment (Appendix E.1).

D.8 | Finding M: Renewal-Queue Griefing

Severity: Medium. **Status:** Fixed (commit facc3dc).

Description. The `RenewalQueue` introduced for Finding K (Section D.6) is shared by all content: each block holds at most ten renewals, and `enqueue_renewal` searches at most 100 blocks ahead. `enable_auto_renew` accepted content with any period. Keeping every bucket full needs about $10 \times \max(p, 100)$ queued entries for a period of p blocks, so about 1,000 auto-renewing subscriptions with periods of 100 blocks or less would suffice; since the holder cap was removed (commit 2865ed0) they can all target a single content item. The attacker pays the subscription price to themselves, and renewals in `on_initialize` pay no transaction fee, so after the initial transactions the queue stays full at no further cost; a minimum price would not prevent this. While the queue is full, `enable_auto_renew` fails for every user with `RenewalQueueFull`, and existing subscribers lose auto-renewal when their next renewal cannot be queued. A period of zero also re-queued the entry into the bucket just processed, where it never ran again. We found this by code review; the attack was not executed.

Remediation. `enable_auto_renew` now rejects content whose period is below a new configuration constant, `MinAutoRenewPeriod` (14,400 blocks in the runtime, about one day at 6-second blocks), with a new `AutoRenewPeriodTooShort` error. Keeping the whole queue full then needs about 144,000 subscriptions. Content with a shorter period can still be registered, subscribed to and renewed manually. Regression tests: `enable_auto_renew_rejects_period_below_minimum` and `enable_auto_renew_rejects_zero_period_and_accepts_minimum`. A targeted variant remains: an attacker can still fill the 100-block window around one predictable expiry block, at a one-time cost of about 1,000 subscriptions per window. A refundable deposit per queued entry would raise that cost further.

D.9 | Error Variant Coverage

Table D.1. Error Variant Coverage

Error Variant	Tested	Error Variant	Tested
ContentNotFound	Yes	AutoRenewPeriodTooShort	Yes
NotContentCreator	Yes	ContentIdOverflow	Yes
SubscriptionAlreadyExists	Yes	ItemIdOverflow	No
SubscriptionNotFound	Yes	OwnershipNotFound	Yes
SubscriptionNotExpired	Yes	InvalidRoyaltySplits	Yes
ViewPackNotFound	Yes	AutoRenewAlreadyEnabled	Yes
NoViewsRemaining	Yes [†]	AutoRenewNotEnabled	Yes
AlreadyOwned	Yes	Unauthorized	Yes
InsufficientPayment	Yes	NftOperationFailed	No
RenewalQueueFull	Yes		
ZeroViews	Yes		

[†] Defensive; tested via direct storage write.

D.10 | Static Analysis Advisory Breakdown

Table D.2. Static Analysis Advisory Breakdown

Category	Package	Risk	Mitigation
Networking (2)	quinn-proto, ring	DoS, panic	Relay/p2p layer only; no runtime impact
WASM executor (4)	wasmtime	Resource exhaustion, panics, f64 segfault	Sandboxed WASM execution model
TLS (1)	rustls-webpki	CRL bypass	Node-side only
Logging (1)	tracing-subscriber	Log injection	Observer layer only

All 8 advisories are mitigated by the architectural separation between the WASM-sandboxed runtime and the node binary. None are in code we authored.

Future Work Details

E.1 | Production Deployment

The most significant future work is deployment to a public testnet or mainnet. This would yield realistic latency and throughput measurements under network conditions that include geographic distribution, real-world contention, and validator set diversity; multi-collator reliability data replacing the single-collator recovery figure of about 18 seconds; genuine cross-chain operation against other parachains; and the opportunity for third-party security audit that would address the single-author validation limitation.

E.2 | Scaling Beyond the Prototype Configuration

The prototype already runs with asynchronous backing (Section 8.2.1). Elastic scaling was not exercised: the runtime uses one core per parachain (block processing velocity 1), and Agile Coretime does not apply to the local test network, where parachains are registered at genesis. A production deployment would acquire coretime on the Polkadot relay chain and could assign additional cores to CCRMS through elastic scaling. This requires no pallet-level changes, but it does require raising the runtime's block processing velocity and unincluded-segment capacity, and switching to a collator that supports multiple cores; the resulting throughput gain would need to be measured rather than assumed. The runtime already executes XCM v5; sending V5-encoded messages (for example with PayFees in place of BuyExecution) would change only how Para B constructs messages, not the pallet.

E.3 | Migration Paths for the Smart Contract Layer

The January 2026 ink! discontinuation leaves `rights_manager` without upstream toolchain support. Three migration paths are available: (a) Solidity rewrite using the revive Solidity-to-PolkaVM compiler (broadest tooling support, requires Solidity reimplementation); (b) adoption of `wrevive`, the community-maintained ink! successor (preserves the Rust experience, introduces a community-project dependency); (c) removal of the contract layer entirely (zero maintenance cost, loses ink!-style API convenience). The right choice depends on the intended client base and available maintenance resources.

E.4 | Holder Scale and Creator Deposits

The bounded `Children` vector duplicated information already held in `Subscriptions`, `ViewPacks`, and `Ownership` (holders) and in `Parent` (nesting), and capped each content item at fifty holders. Commit 2865ed0 removed it; content items no longer have a holder cap, and the regression test `security_no_holder_cap` admits 60 holders.

Two tasks remain. First, benchmark content items with thousands of holders, including purchases, burns and `check_access`, to confirm that per-holder costs stay constant. Second, review per-holder deposits: each child NFT reserves a `pallet-nfts` item deposit and an attribute deposit from the creator (about two existential deposits per holder; regression test `child_mint_reserves_creator_deposit`), so a creator with a large audience needs a matching reserved balance, and for low-priced content the deposit can exceed the price. A production runtime could charge these deposits to the buyer, or waive them for non-transferable receipts.

Subscription expiry should also release or burn the associated child NFT, so that state and deposits reflect current rather than historical holders.

E.5 | User Study and Consumer-Facing Application

A future iteration would build a minimal web wallet integration or CLI tool and conduct user testing with creators and consumers drawn from Web3 communities. Specific questions: Can a typical Web3 user successfully purchase a subscription, consume a view, and transfer ownership without prompting? Do users understand the three monetization models from the wallet UI alone? What error messages are most confusing? Are creators willing to migrate from existing platforms? This was originally planned for Phase 2

but was not executed because we built no front end; it is the highest-priority near-term research extension.

E.6 | The Bridged-Payment Final Hop

Closing the gap between bridged ETH on AssetHub and a paid CCRMS rights operation requires either configuring `PaymentCurrency` to accept the foreign Ether asset directly or inserting an explicit swap step to convert ETH to the native parachain token. Either approach is conceptually straightforward.

E.7 | Resale Royalties

The current `transfer_ownership` extrinsic has no creator share on secondary-market transfers. CCRMS could add an optional per-content `resale_royalty_basis_points` field; the implementation challenge is verifying the declared sale price, which suggests resale royalties would work best in conjunction with an on-chain marketplace pallet that records sale prices as part of the transfer transaction.

E.8 | Origin Aliasing and Delegation for Cross-Chain Transfers

A signed user on a sibling parachain can already transfer ownership held by their `HashedDescription`-derived account (Appendix D.1). Two gaps remain. First, the derived account differs from the user's local CCRMS account, so ownership cannot follow a user between their local and remote identities. XCM v5 addresses this with `AliasOrigin`: if the runtime set `Aliasers` to `pallet_xcm::AuthorizedAliasers` (with a deposit configured through `AuthorizedAliasConsideration`, currently `Disabled`), a user could call `add_authorized_alias` on CCRMS to authorize their Para B location, and a message from that location could then replace its origin with the user's local account before `Transact`. Both the type and the extrinsic exist in the SDK version we used, but our runtime sets `Aliasers = Nothing`, so this route is untested. Second, a transfer by a third party on the owner's behalf, such as a marketplace, still requires delegation: the owner pre-authorizes a specific delegate for a specific operation and time window. The implementation challenge is the storage layout for delegation records and the revocation lifecycle.

E.9 | Monte Carlo Simulation Extensions

Future extensions could incorporate creator psychology (risk aversion, platform-switching costs, brand attachment), model network-effect dynamics at critical creator mass, add longitudinal modeling over multiple years, and model regulatory effects from the EU AI Act or MiCA enforcement.

E.10 | Formal Verification

Following Maric et al. (2025) on Isabelle/HOL verification of cross-chain bridges, similar techniques could be applied to the CCRMS authorization model, the royalty distribution algorithm, and the cross-chain operation pattern. Formal verification is expensive but offers guarantees that empirical testing cannot match, and is worthwhile for any production deployment handling significant value.

E.11 | JAM Transition

JAM (the Join-Accumulate Machine) represents Polkadot's architectural successor to the existing relay chain. The testnet was launched in January 2026 with the participation of multiple implementation teams; no mainnet date has been announced as of mid-2026, with multi-client implementations completing early milestones across several language runtimes. CCRMS does not incorporate any JAM-specific features, and our implementation specifications provide sufficiently detailed architectural decisions to facilitate potential future migration without necessitating a complete rebuild.

Work Plan and Timeline

We structured the work into six phases, originally planned across 24 weeks:

- **Phase 1 (Weeks 1–6): Research and Literature Review.** Establishes the academic and commercial context. Produces the initial 78-source literature review (later expanded to about 100).
- **Phase 2 (Weeks 7–10): Design Conceptualization.** Translates high-level concepts into the system design and methodology.
- **Phase 3 (Weeks 11–12): Architecture Finalization.** Bridges the conceptual design to a concrete implementation plan.
- **Phase 4 (Weeks 13–16): Core Implementation.** The primary build phase: pallet-content-rights, runtime configuration, contract layer, XCM, Snowbridge integration.
- **Phase 5 (Weeks 17–20): Testing and Evaluation.** Unit tests, benchmarks, security audit, reliability test, comparative benchmark, Monte Carlo simulation, and literature review version 2.
- **Phase 6 (Weeks 21–24): Finalization.** Dissertation writing, implementation specifications, and LaTeX submission preparation.

References to “Phase N” throughout the dissertation denote work produced during that phase.

Monte Carlo Simulation Parameters and Sensitivity

This appendix documents the full parameter set and sensitivity results for the Monte Carlo revenue simulation described in Section 7.2. The simulation is seeded (xoshiro128**, seed 42) and reproducible to every reported digit from the published repository (Node.js 22 or later).

The headline run used $1,000 \text{ creators} \times 10,000 \text{ iterations}$. Creator attributes are redrawn in every iteration, so the simulation samples ten million independent creator-scenarios per platform rather than following 1,000 fixed creators. The sensitivity and sweep results below used $200 \text{ creators} \times 2,000 \text{ iterations}$ with the same seed; on this smaller grid the baseline advantage reads 4.7% against the 4.4% of the headline run.

Correction. An earlier version of the simulation set the secondary-sale rate to $\lambda = 0.1$. Because the sampler draws $-\ln(u)/\lambda$, that value gives a mean of 10, so about 90% of draws reached the cap of 1.0 and the model assumed that roughly 95% of each creator’s audience made a secondary purchase every year. We corrected the rate to $\lambda = 10$ (mean about 0.1) in commit 52454ea. Every figure in this dissertation uses the corrected value. The correction reduced the mean advantage of CCRMS over the centralized configuration from 16.8% to 4.4%.

G.1 Stochastic and Platform Parameters

Table G.1 documents the per-creator stochastic parameters drawn each iteration.

Table G.2 documents the per-platform parameters drawn uniformly once per iteration.

^a Chua (2023); Watson (2024). ^{*} Modeling assumption; no published source. ^b Illustrative comparator: all parameters are assumptions; the fee range stands for bridge and swap fees plus slippage on a multi-hop

Table G.1. Monte Carlo Stochastic Parameters (per creator, per iteration)

Parameter	Distribution
Base audience	Log-normal (median 1,000, $\sigma = 2$)
Content price	Uniform (\$1–\$50)
Subscription adoption	Beta ($\alpha = 2$, $\beta = 5$), mean $\approx 28.6\%$
PPV conversion	Beta ($\alpha = 1$, $\beta = 10$), mean $\approx 9.1\%$
Secondary-sale probability	Exponential (rate $\lambda = 10$, mean ≈ 0.1), capped at 1.0
Stream weights	Subscription 50%, PPV 30%, secondary 20%
Subscription period	12 months

Table G.2. Monte Carlo Platform Parameters (drawn uniformly once per iteration)

Platform	Fee	Discovery added	Monthly churn	Cost/tx
Centralized	30–45%	+500–3,000*	2–5%*	\$0.000*
Web3 market-place	0–2.5% ^a	+100–800*	5–10%*	\$0.005*
Bridge-based ^b	8–15%	+50–400	6–12%	\$0.020
CCRMS	1–5%*	+0–200*	4–8%*	\$0.001*

route and is not drawn from a published fee schedule.

The centralized fee range is drawn from the platform sources in Table 7.12. The discovery, churn and per-transaction parameters for every platform are modeling assumptions. The CCRMS fee of 1–5% is an allowance for protocol or treasury fees rather than a measured cost; Polkadot fees are not levied as a share of revenue, and the model charges CCRMS per-transaction costs separately.

The bridge-based configuration represents cross-chain distribution through third-party bridges such as Wormhole and LayerZero. Because its parameters are assumptions, its results are reported here rather than in Chapter 7, and the comparison with it is not an empirical result. With the headline settings, mean annual revenue per creator is \$16,183 (median \$2,058) and the central 95% range of simulated total revenue is \$10.4M–\$26.8M. The CCRMS mean is 32.5% higher, and CCRMS earns more in 74.3% of scenarios, a margin that follows from the assumed fee ranges.

Discovery is additive rather than multiplicative: a flat bonus of 500–3,000 followers matters substantially at 200 followers and marginally at 50,000. This assumption produces the audience-size crossover. With no discovery bonus on either platform, CCRMS earns more in 97.1% of scenarios (Table G.5); the crossover is therefore a consequence of the assumed discovery bonus, which has no published source, rather than an independent

finding.

G.2 Eight-Parameter Sensitivity Grid

Table G.3 varies each input parameter independently. Price has no effect because it multiplies every platform's revenue equally. The result is sensitive to the audience distribution and to the subscription period, which sets how long the higher assumed CCRMS churn compounds.

Table G.3. Sensitivity of CCRMS Advantage to Individual Parameter Changes

Parameter varied	Range tested	Advantage over centralized
Audience median	500 → 5,000	−10.8% to +21.9%
Audience σ	1.0 → 3.0	−34.1% to +24.4%
Content price floor	\$0.50 → \$5	+4.7% to +4.6%
Content price ceiling	\$20 → \$200	+4.6% to +4.7%
Subscription adoption α	1 → 5	+9.3% to +0.5%
PPV conversion α	0.5 → 3	+2.5% to +10.0%
Resale rate λ (mean 0.2 → 0.02)	5 → 50	+7.4% to +1.7%
Subscription period	1 → 24 months	+24.9% to −5.7%

G.3 CCRMS Fee Sweep

Table G.4 fixes the CCRMS fee at single values. The mean-revenue advantage disappears at a CCRMS fee of about 7%, two percentage points above the top of the assumed 1–5% range.

Table G.4. Sensitivity of CCRMS Advantage to Fee Assumption

CCRMS fee	Advantage over centralized	Win rate
1–5% (baseline)	+4.7%	20.6%
5%	+2.5%	19.2%
6%	+1.4%	18.5%
7%	+0.3%	17.8%
8%	−0.8%	17.2%
10%	−2.9%	15.7%
15%	−8.3%	12.1%

G.4 Discovery Sweep

Table G.5 scales the centralized discovery bonus. The advantage reverses at about 1.25 times the assumed value, so discovery is the most influential assumption in the model.

Table G.5. Sensitivity of CCRMS Advantage to Centralized Discovery Assumption

Centralized discovery	Advantage over centralized	Win rate
0× (no bonus)	+28.9%	97.1%
0.25× (+125–750)	+21.9%	49.3%
0.5× (+250–1,500)	+15.5%	32.5%
0.75× (+375–2,250)	+9.8%	25.0%
1× (+500–3,000, baseline)	+4.7%	20.6%
1.5× (+750–4,500)	–4.4%	15.4%
2× (+1,000–6,000)	–11.9%	12.3%

G.5 Fine-Grained Crossover Buckets

Table G.6 shows the audience-size transition at finer resolution. The 50% win-rate crossover and the mean-revenue crossover both fall near 6,000 followers. Above 10,000 followers, CCRMS earns more in about three quarters or more of scenarios, rising to about 92% at 50,000–100,000.

Table G.6. Fine-Grained CCRMS Win Rate and Mean Advantage Near the Crossover Audience Size

Base audience	Win rate vs centralized	Mean advantage
1,500–2,000	8.7%	–32.8%
2,000–2,500	14.1%	–25.0%
3,000–3,500	25.8%	–14.7%
4,000–4,500	36.7%	–7.9%
5,000–6,000	48.6%	–1.7%
6,000–7,000	57.1%	+1.7%
8,000–9,000	68.5%	+7.1%
10,000–12,500	76.4%	+11.2%
20,000–30,000	86.6%	+19.1%
50,000–100,000	91.9%	+24.8%

G.6 Web3 Marketplace Sensitivity

Table G.7 varies the unsourced Web3 marketplace parameters independently to show how the CCRMS-versus-Web3 comparison changes under different assumptions.

Table G.7. CCRMS vs. Web3 Marketplace Under Varied Parameters

Web3 parameter changed	CCRMS vs Web3 (mean)	CCRMS win rate
Baseline (fee 0–2.5%, discovery +100–800, churn 5–10%, \$0.005/tx)	+4.6%	29.1%
Discovery +0–200 (same as CCRMS)	+9.4%	68.1%
Discovery doubled (+200–1,600)	–1.0%	17.3%
Churn 4–8% (same as CCRMS)	–6.0%	13.8%
Churn 8–15%	+33.9%	57.3%
Cost/tx \$0.001 (same as CCRMS)	+4.5%	28.8%
All non-fee dynamics same as CCRMS	–1.8%	45.6%

Across these cases, CCRMS ranges from –6.0% to +33.9% relative to Web3 marketplaces, and the sign depends mainly on the assumed churn difference, which has no published source. With identical non-fee dynamics, the fee difference alone leaves CCRMS 1.8% behind. The comparison is therefore undetermined within the assumption uncertainty.

G.7 Prototype as Built (No Secondary Stream)

The model credits creators on every platform with a secondary-sale stream (20% weight). The prototype’s `transfer_ownership` pays creators no resale royalty (Appendix E.7), and centralized streaming platforms have no resale market. Removing the stream for all platforms gives the configuration closest to the prototype as built. With the headline settings, the CCRMS mean advantage over the centralized configuration falls to 0.6% and its win rate to 18.1%; the 50% win-rate crossover rises to about 7,000 followers; and the advantage disappears at a fixed CCRMS fee of 5% (–1.2%) or at about 1.1 times the assumed centralized discovery bonus. Against Web3 marketplaces, CCRMS is 6.9% ahead on the mean. The full sweep results are in `results/monte-carlo-appendix-g-no-secondary.json`.

G.8 Reproduction

From `scripts/perf/` in the repository:

- `node monte-carlo-revenue.mjs` reproduces the headline results (Tables 7.8, 7.9, 7.10 and 7.11) and the sensitivity grid in Table G.3.
- `node monte-carlo-revenue.mjs --no-secondary` reproduces the prototype-as-built headline figures in Section G.7.
- `node monte-carlo-appendix-g.mjs` reproduces Tables G.4 to G.7; adding `--no-secondary` reproduces the Section G.7 sweeps.

The sweeps use $200 \text{ creators} \times 2,000 \text{ iterations}$ and seed 42, matching the sensitivity grid.

References

- 360iResearch. Blockchain for Content Monetization Market by Content Type, Blockchain Type, Monetization Model, Application, End-User: Global Forecast 2025–2030. Technical report, 360iResearch, March 2025. URL <https://www.giiresearch.com/report/ires1677173-blockchain-content-monetization-market-by-content.html>.
- F. Aguirre. RFC-0105: XCM Improved Fee Mechanism. Polkadot Fellowship RFCs, 2024. URL <https://github.com/polkadot-fellows/RFCs/pull/105>.
- Amazon.com, Inc. ACX Royalty Rates. Audiobook Creation Exchange (ACX), 2025. URL <https://www.acx.com/royalties>. Accessed May 2026. Exclusive distribution plan pays 40% royalty to rights holders; Audible retains 60%.
- Lance Anders and Shrug. ERC-4907: Rental NFT, an extension of EIP-721. Ethereum Improvement Proposals, no. 4907, March 2022. URL <https://eips.ethereum.org/EIPS/eip-4907>.
- L. Ante. Smart contracts on the blockchain – A bibliometric analysis and review. SSRN Electronic Journal, 2020. URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3576393.
- Apple Inc. Apple Music Insights Royalty Rate. Apple for Artists Support, 2025. URL <https://artists.apple.com/support/1124-apple-music-insights-royalty-rate>. Accessed May 2026. Apple Music pays approximately 52% to labels; platform retains approximately 48%.
- P. Banerjee et al. Reliable, Fair and Decentralized Marketplace for Content Sharing Using Blockchain. arXiv:2009.11033, 2020. URL <https://arxiv.org/pdf/2009.11033>.
- Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley, 4th edition, 2021.
- A. Bigiotti, L. Mostarda, A. Navarra, A. Pinna, D. Sestili, R. Tonelli, and M. Vaccargiu. Blockchain Agnostic Protocols: An Analysis of the State of the Art. *ACM Computing Surveys*, 58(2):1–36, 2025. doi: 10.1145/3758089.
- J. Burdges, A. Cevallos, P. Czaban, R. Habermeier, S. Hosseini, F. Lama, H. K. Alper, X. Luo, F. Shirazi, A. Stewart, and G. Wood. Overview of Polkadot and its Design Considerations. arXiv:2005.13456, 2020. URL <https://arxiv.org/abs/2005.13456>.

- I. Buzu. Blockchain, smart contracts and copyright management disruption. SSRN, 2021. URL https://papers.ssrn.com/sol3/Delivery.cfm/SSRN_ID3759260_code3591422.pdf.
- M. A. Cabot-Nadal, M. Payeras-Capellà, M. Mut-Puigserver, and A. Soto-Fernández. Improving the Token ERC-721 Implementation for Selective Receipt: Rejectable NFTs. In *2022 6th International Conference on System Reliability and Safety (ICSRS)*, pages 243–250. IEEE, 2022. doi: 10.1109/ICSRS56243.2022.10067494.
- Y. Cao, J. Cao, D. Bai, L. Wen, Y. Liu, and R. Li. MAP the Blockchain World: A Trustless and Scalable Blockchain Interoperability Protocol for Cross-chain Applications. In *Proceedings of the ACM Web Conference 2025 (WWW '25)*. ACM, 2025. doi: 10.1145/3696410.3714867.
- A. Catangiu. RFC-0100: New XCM Instruction InitiateAssetsTransfer. Polkadot Fellowship RFCs, 2024. URL <https://polkadot-fellows.github.io/RFCs/approved/0100-xcm-multi-type-asset-transfer.html>.
- M. Chegenizadeh, S. Rafati Niya, and C. J. Tessone. Multi-agent Privacy-Preserving Micro-payment Verification for Offline Resource-Constrained Devices Using Smart Contracts. In *Mathematical Research for Blockchain Economy (MARBLE 2024)*, Lecture Notes in Operations Research. Springer, 2024. doi: 10.1007/978-3-031-68974-1_7.
- K. Chen, L.-F. Lee, W. Chiu, C. Su, K.-H. Yeh, and H.-C. Chao. A Trusted Reputation Management Scheme for Cross-Chain Transactions. *Sensors*, 23(13):6033, 2023. doi: 10.3390/s23136033.
- M. Cherdakov, A. Kudashkin, Y. Madhwal, and Y. Yanovich. Web3 Based Digital Rights Management in the Music Industry. In *Proceedings of the 2023 6th International Conference on Blockchain Technology and Applications (ICBTA)*. ACM, 2023. doi: 10.1145/3651655.3651667.
- J. X. Chua. State of NFT marketplaces. Binance Research, March 2023. URL <https://research.binance.com/static/pdf/state-of-nft-marketplaces.pdf>.
- R. F. Ciriello, A. C. G. Torbensen, M. R. P. Hansen, and C. Müller-Bloch. Blockchain-based digital rights management systems: Design principles for the music industry. *Electronic Markets*, 33(1), 2023. doi: 10.1007/s12525-023-00628-5.
- Coinbase and x402 Foundation. x402 Protocol: HTTP-native blockchain micropayments. CoinPaprika, 2025. URL <https://coinpaprika.com/education/x402-ecosystem-instant-blockchain-payments-for-apis-and-ai-agents/>.
- cygaar. ERC-5643: Subscription NFTs. Ethereum Improvement Proposals, no. 5643, September 2022. URL <https://eips.ethereum.org/EIPS/eip-5643>.
- P. Darshan, J. S. Rohan, R. Rajesh, M. Ruchitha, S. Kamath, and M. N. Manas. Intellectual Property Rights and Entrepreneurship in the NFT Ecosystem: Legal Frameworks, Business Models, and Innovation Opportunities. arXiv:2507.00172, 2025. URL <https://arxiv.org/html/2507.00172v1>.
- C. Davik. The Art Of NFTs: Copyright, Contracts, And The Fallacy Of Ownership. *Georgia Law Review*, 2025.
- Z. Deng, C. Tang, T. Li, P. Ablä, Q. Chen, W. Liang, and D. He. Enhancing Blockchain Cross-Chain Interoperability: A Comprehensive Survey. arXiv:2505.04934, 2025. URL <https://arxiv.org/abs/2505.04934>.

- A. Dreyer et al. Can I mint an NFT with that?: Avoiding right of publicity and trademark litigation risks in the brave new world of NFTs. Skadden Publications, 2021. URL <https://www.skadden.com/-/media/files/publications/2021/05/canimintannftwiththatavoidingrightofpublicityandtr.pdf>.
- Eurojust. Non-Fungible Tokens and Intellectual Property Rights – Can the use of NFTs lead to IP infringements? Technical report, European Union Agency for Criminal Justice Cooperation, 2024. URL <https://www.eurojust.europa.eu/sites/default/files/assets/eurojust-nfts-intellectual-property-rights-flyer.pdf>.
- European Securities and Markets Authority. MiCA Regulation: Markets in Crypto-Assets. Technical report, ESMA, 2025. URL <https://www.esma.europa.eu/esmas-activities/digital-finance-and-innovation/markets-crypto-assets-regulation-mica>. Full enforcement by July 2026.
- A. Garba et al. A digital rights management system based on a scalable blockchain. Copenhagen Business School Research, 2021. URL https://research-api.cbs.dk/ws/portalfiles/portal/93314574/abba_garba_et_al_a_digital_rights_management_system_based_on_a_scalable_blockchain_publishersversion.pdf.
- J. Gatto et al. NFT License Breakdown: Exploring Different Marketplaces and Associated License Issues. Law of the Ledger, 2021. URL <https://www.lawofthelledger.com/wp-content/uploads/sites/15/2021/09/NFT-License-Breakdown-Article-0921.pdf>.
- Google LLC. YouTube Partner Program: How ad revenue is shared. YouTube Help Center, 2025. URL <https://support.google.com/youtube/answer/72902>. Accessed May 2026. YouTube retains 45% of ad revenue; creators receive 55%.
- P. Goyal et al. Secure Incentivization for Decentralized Content Delivery. In *USENIX Workshop on Hot Topics in Edge Computing (HotEdge 2019)*, 2019. URL https://www.usenix.org/system/files/hotedge19-paper-goyal_0.pdf.
- Grand View Research. Digital Rights Management Market Size, Share & Trends Analysis Report. Technical report, Grand View Research, 2025. URL <https://www.grandviewresearch.com/industry-analysis/digital-rights-management-market>.
- A. Guadamuz. The EU’s Artificial Intelligence Act and copyright. *The Journal of World Intellectual Property*, 28:213–219, 2025. doi: 10.1111/jwip.12330.
- R. Habermeier et al. Polkadot’s messaging scheme. Web3 Foundation Research, Medium, 2021. URL <https://medium.com/web3foundation/polkadots-messaging-scheme-b1ec560908b7>.
- B. Hanneke, O. Hinz, J. Pfeiffer, and W. M. P. van der Aalst. The Internet of Value: Unleashing the Blockchain’s Potential with Tokenization. *Business & Information Systems Engineering*, 66(4):411–419, 2024. doi: 10.1007/s12599-024-00883-6.
- A. R. Hevner, S. T. March, J. Park, and S. Ram. Design science in information systems research. *MIS Quarterly*, 28(1):75–105, 2004.

- ink! Alliance. Discontinuation of ink: Rust smart contract language. Polkadot Forum, 2026. URL <https://forum.polkadot.network/t/discontinuation-of-ink-rust-smart-contract-language/16849>. Posted January 27, 2026.
- V. Kumar, I. Budhiraja, A. Jabbari, et al. Efficient blockchain interoperability design for cross-chain transactions in future internet-of-value. *Peer-to-Peer Networking and Applications*, 18:110, 2025. doi: 10.1007/s12083-025-01941-w.
- S. Lahiri et al. Multidisciplinary perspectives on Non-Fungible Tokens (NFTs): a comprehensive meta-analysis. *Digital Economy and Sustainable Development*, 4:3, 2026. doi: 10.1007/s44265-026-00078-9.
- W. Li, Z. Liu, J. Chen, Z. Liu, and Q. He. Towards blockchain interoperability: a comprehensive survey on cross-chain solutions. *Blockchain: Research and Applications*, 6(3):100286, 2025.
- S. Y. Lin et al. A survey of application research based on blockchain smart contract. *Wireless Networks*, 28: 635–690, 2022. doi: 10.1007/s11276-021-02874-2.
- S. L. Madapati and N. R. Pradhan. Decentralizing video copyright protection: a novel blockchain-enabled framework with performance evaluation. *Frontiers in Artificial Intelligence*, 8, 2025.
- Magic Eden. Magic Eden Marketplace Fees. Magic Eden Help Center, 2025. URL <https://help.magiceden.io>. Accessed May 2026. Magic Eden charges a 2% marketplace fee on NFT sales.
- H. Mao, T. Nie, H. Sun, D. Shen, and G. Yu. A Survey on Cross-Chain Technology: Challenges, Development, and Prospect. *IEEE Access*, 11:45527–45546, 2023. doi: 10.1109/ACCESS.2022.3228535.
- D. Mareckova. *Blockchain and Collective Rights Management of Copyright and Related Rights at the Global Level: The Case of the Music Industry*. PhD thesis, OAPEN, 2024. URL https://library.oapen.org/bitstream/handle/20.500.12657/94575/oa_pdf-057-1731486746.pdf.
- F. Maric, B. Scholz, and P. Subotic. Formal Verification of a Fail-Safe Cross-Chain Bridge. In *6th International Workshop on Formal Methods for Blockchains (FMBC 2025)*, volume 129 of *OASICs*, 2025. URL <https://drops.dagstuhl.de/entities/document/10.4230/OASICs.FMBC.2025.8>.
- MarketsandMarkets. Digital Rights Management Market by Application. Technical report, MarketsandMarkets Research, July 2025. URL <https://www.marketsandmarkets.com/Market-Reports/digital-rights-management-market-152806525.html>.
- J. C. Mendoza-Tello, J. J. Sánchez-Lucas, and J. G. Orosco-Pantoja. A Model to Support the Music Streaming Service Using Blockchain-Based Smart Contracts. In *Applied Technologies (ICAT 2023)*, volume 2051 of *Communications in Computer and Information Science*. Springer, 2024. doi: 10.1007/978-3-031-58950-8_10.
- Mordor Intelligence. Digital Rights Management (DRM) Market – Size, Share & Industry Analysis. Technical report, Mordor Intelligence, January 2026. URL <https://www.mordorintelligence.com/industry-reports/digital-rights-management-drm-market>.
- D. Morháč, V. Valaštin, K. Košťál, and I. Kotuliak. ParaSpell XCM SDK: A New Protocol for Interoperability in Polkadot Paraverse. In *2023 Fifth International Conference on Blockchain Computing and Applications (BCCA)*, pages 569–576. IEEE, 2023a. doi: 10.1109/BCCA58897.2023.10338906.

- D. Morháč, V. Valaštín, K. Košťál, and I. Kotuliak. Enhancing XCMP Interoperability Across Polkadot Paraverse. In *2023 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pages 1–3. IEEE, 2023b. doi: 10.1109/ICBC56567.2023.10174872.
- D. Morháč, K. Košťál, V. Valaštín, et al. Unispell: universal adapter for interoperability in Polkadot Paraverse. *Cluster Computing*, 28:311, 2025. doi: 10.1007/s10586-025-05183-6.
- M. Munson. A Blockchain Controlled Content Fabric. Video Services Forum, 2019. URL https://static.vsf.tv/Meetings/2019/2019_DC/Munson_BlockchainFabric.pdf.
- M. Munson et al. Blockchain Content Fabric for Multi-Chain Content Ownership and Distribution. Technical report, Eluvio, 2022. URL <https://eluv.io/3e39198441b1d578bc0f.pdf>.
- M. Murray. NFT Ownership And Copyrights. *Indiana Law Review*, 56:367, 2022.
- Music Business Research. The Music Streaming Economy Part 14: The Artists’ Share of the Music Streaming Pie. Music Business Research Blog, 2024. URL <https://musicbusinessresearch.wordpress.com/2024/09/16/the-music-streaming-economy-part-14-the-artists-share-of-the-music-streaming-pie/>. Accessed May 2026.
- OpenSea. OpenSea Fees. OpenSea, 2025. URL <https://opensea.io/fees>. Accessed May 2026. OpenSea charges a 2.5% protocol fee on secondary sales; creator royalties are set separately by the collection creator.
- P. Orero et al. The Visible Subtler: Blockchain Technology for Right Management and Minting. *Open Research Europe*, 3:26, 2023.
- O. Osemwegie. Decentralized Media Distribution: Exploring blockchain’s role in copyright protection, monetization, and content ownership rights. *International Journal of Mechanical Technology and Marine Engineering*, 2025.
- Parity Technologies. ink! 3.0: Parity’s Rust-based language for WASM smart contracts gets a major update. Parity Blog, 2022a. URL <https://www.parity.io/blog/ink-3-0-paritys-rust-based-language-gets-a-major-update>.
- Parity Technologies. What is Parity’s ink!? Parity Blog, 2022b. URL <https://www.parity.io/blog/what-is-paritys-ink>.
- Parity Technologies. Polkadot Upgrade 2025: What You Need to Know. Parity Blog, 2025. URL <https://www.parity.io/blog/polkadot-upgrade-2025-what-you-need-to-know>.
- Parity Technologies. Enable Elastic Scaling. Polkadot SDK documentation, 2026. URL https://paritytech.github.io/polkadot-sdk/master/polkadot_sdk_docs/guides/enable_elastic_scaling_mvp/index.html. Accessed 5 October 2026.
- David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972. doi: 10.1145/361598.361623.

- S. S. Patil et al. Blockchain Based Multimedia Content Protection Using Encryption in DRM (Digital Rights Management). *International Journal of Innovative Research in Technology (IJIRT)*, 2022.
- K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee. A Design Science Research Methodology for Information Systems Research. *Journal of Management Information Systems*, 24(3):45–77, 2007. doi: 10.2753/MIS0742-1222240302.
- Polkadot Network. Polkadot Roundup 2025. Medium, 2025. URL <https://medium.com/polkadot-network/polkadot-roundup-2025-3c3c71c7e9c4>.
- Polkadot OpenGov. Referendum 1484: Increase validator set size to 600. Online, 2025. URL <https://polkadot.subsquare.io/referenda/1484>. Whitelisted Caller track; executed. Accessed September 2026.
- L. Quan, Y. Pan, and W. Zhong. Research and Application of Digital Collection Smart Contract Method Based on Blockchain. In *Proceedings of ICBIS 2024*, pages 34–43. Atlantis Press, 2024. doi: 10.2991/978-94-6463-419-8_6.
- I. S. Rao, M. L. M. Kiah, M. M. Hameed, and Z. A. Memon. Scalability of blockchain: a comprehensive review and future research direction. *Cluster Computing*, 27:5547–5570, 2024. doi: 10.1007/s10586-023-04257-7.
- Rated Labs. Rated Network Explorer: Ethereum Validator and Operator Data, 2026. URL <https://explorer.rated.network/>. Accessed August 2026.
- E. Reggianini. Standardizing On-chain IP Rights Management. Technical report, BlockStand, 2025. URL https://blockstand.eu/blockstand/uploads/2025/05/Standardizing-On-chain-IP-Rights-Management_Reggianini.pdf.
- K. Ren, N.-M. Ho, D. Loghin, T.-T. Nguyen, B. C. Ooi, Q.-T. Ta, and F. Zhu. Interoperability in Blockchain: A Survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(12):12750–12769, 2023. doi: 10.1109/TKDE.2023.3275220.
- A. Shah. Automating Royalties: A Framework of Smart Contracts for Cross-Platform Content Revenue. *International Journal for Multidisciplinary Research (IJFMR)*, 2025.
- R. Singh et al. Insights into research on blockchain for smart contracts: A bibliometric analysis. *Multimedia Tools and Applications*, 83:1–25, 2024. doi: 10.1007/s11042-023-15647-1.
- Snowfork. Snowbridge V2 Is Live! Polkadot Forum, 2025a. URL <https://forum.polkadot.network/t/snowbridge-v2-is-live/15844>. Posted November 7, 2025.
- Snowfork. Snowbridge 2025/2026 Stabilized. Polkasassembly Proposal #3313, 2025b. URL <https://polkadot.polkasassembly.io/post/3313>.
- Solana Compass. Solana Validators, 2026. URL <https://solanacompass.com/validators>. Accessed August 2026.
- Spotify AB. Understanding Spotify Royalties. Spotify for Artists Support, 2025. URL <https://support.spotify.com/us/artists/article/understanding-spotify-royalties/>. Accessed May 2026. Spotify pays approximately 70% of revenue to rights holders.

- Steem Team. Steem, an incentivized, blockchain-based, public content platform. Technical report, Steem, 2025. URL <https://steem.com/SteemWhitePaper.pdf>. Whitepaper.
- Story Foundation. Story Mainnet Launch. CoinDesk, 2025. URL <https://www.coindesk.com/tech/2025/02/13/story-protocol-launches-to-let-people-to-register-ip-and-get-paid-for-it/>. Posted February 13, 2025.
- Superfluid. Superfluid Documentation: Stream Money Every Second, 2026. URL <https://docs.superfluid.org/>. Accessed 5 October 2026.
- The DATA Foundation. We're becoming The DATA Foundation. Blog post, 2026. URL <https://www.datafdn.org/blog/were-becoming-the-data-foundation>. 25 June 2026. Accessed September 2026.
- Unique Network. Unique network documentation: Nested NFTs. Online, 2026. URL <https://docs.unique.network>. Accessed September 2026.
- Unlock Protocol. Unlock protocol documentation. Online, 2026. URL <https://docs.unlock-protocol.com>. Accessed September 2026.
- U.S. Department of Justice and Federal Trade Commission. Horizontal Merger Guidelines. Technical report, U.S. Department of Justice, 2010. URL <https://www.justice.gov/atr/horizontal-merger-guidelines-08192010>. Accessed September 2026.
- USPTO. Non-Fungible Tokens and Intellectual Property: A report to Congress. Technical report, U.S. Patent and Trademark Office, 2024. URL <https://www.copyright.gov/policy/nft-study/Joint-USPTO-USCO-Report-on-NFTs-and-Intellectual-Property.pdf>.
- J. Venable, J. Pries-Heje, and R. Baskerville. FEDS: A Framework for Evaluation in Design Science Research. *European Journal of Information Systems*, 25(1):77–89, 2016. doi: 10.1057/ejis.2014.36.
- M. Wamugo. Content monetization: non-fungible tokens as a new revenue stream for the media sector in Kenya. Master's thesis, Aga Khan University, 2024. URL https://ecommons.aku.edu/cgi/viewcontent.cgi?article=3232&context=theses_dissertations.
- Z. Wang, W. Feng, M. Huang, S. Feng, S. Mo, and Y. Li. Blockchain-based information security protection mechanism for the traceability of intellectual property transactions. *Sensors*, 25(10):3064, 2025.
- R. Watson. Blur proposal suggests adding trading fees and eliminating creator fee guarantees. The Block, November 2024.
- Web3 Foundation. Asynchronous Backing (Pipelining). Polkadot Wiki, 2026. URL <https://wiki.polkadot.com/learn/learn-async-backing/>. Accessed 5 October 2026.
- G. Wood. Polkadot: Vision for a heterogeneous multi-chain framework. Technical report, Web3 Foundation, 2016. URL <https://polkadot.network/whitepaper>. White Paper.
- G. Wood. XCM: The cross-consensus message format. Polkadot Blog, 2022. URL <https://www.polkadot.network/blog/xcm-the-cross-consensus-message-format>.

- G. Wood. RFC-1: Agile Coretime. Polkadot Fellowship RFCs, 2023. URL <https://polkadot-fellows.github.io/RFCs/approved/0001-agile-coretime.html>.
- G. Wood. JAM: The Join-Accumulate Machine (Gray Paper). Technical report, Web3 Foundation, 2024. URL <https://wiki.polkadot.com/learn/learn-jam-chain/>. Version 0.7.1+, updated through 2025.
- R. Xie and Y. Liu. Research on copyright protection of digital works based on multi-blockchain. *Recent Advances in Computer Science and Communications*, 15(8):1126–1134, 2022. doi: 10.2174/2666255815666220204092947.
- R. Xie and M. Tang. A digital resource copyright protection scheme based on blockchain cross-chain technology. *Heliyon*, 10(16):e34781, 2024. doi: 10.1016/j.heliyon.2024.e34781.
- X. Yi, Y. Zhou, Y. Lin, et al. Digital rights management scheme based on redactable blockchain and perceptual hash. *Peer-to-Peer Networking and Applications*, 16:2630–2648, 2023. doi: 10.1007/s12083-023-01552-3.
- Z. Yin et al. Bool Network: An Open, Distributed, Secure Cross-chain Notary Platform. IACR ePrint 2022/1290, 2022. URL <https://eprint.iacr.org/2022/1290.pdf>.